

Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

School of Engineering

Named Data Networking For The ESP32 Microcontroller Series

Matthew Power

Supervisor: Prof. Stefan Weber

April 16, 2026

A dissertation submitted in partial fulfilment
of the requirements for the degree of
MAI (Computer Engineering)

Declaration

I hereby declare that this dissertation is entirely my own work and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at http://www.tcd.ie/calendar.

I have completed the Online Tutorial on avoiding plagiarism 'Ready Steady Write', located at http://tcd-ie.libguides.com/plagiarism/ready-steady-write.

I consent to the examiner retaining a copy of the thesis beyond the examining period, should they so wish (EU GDPR May 2018).

I agree that this thesis will not be publicly available, but will be available to TCD staff and students in the University's open access institutional repository on the Trinity domain only, subject to Irish Copyright Legislation and Trinity College Library conditions of use and acknowledgement.

Signed: Matthew Power

Date: April 13, 2026

Abstract

Information Centric Networking (ICN) offers a range of alternatives to the current Internet architecture. These approaches have varied over the last few years, but have all agreed on one fundamental idea: the Internet should rely on naming data for forwarding information as opposed to relying on naming devices (IP addresses). This research project explores the domain of ICN, specifically focusing on a leading approach, Named Data Networking (NDN), and its various implementations on resource-constrained microcontrollers.

NDN implementations for microcontroller architectures are feasible, with a few key adaptations. However, as of this dissertation, they are reliant on specific microcontrollers, are claimed to be in a broken state, have been left unmaintained for several years, or are reliant on frameworks not purpose-built for their architecture, thereby missing out on optimisations and adaptations suitable for in-depth usage and testing of an NDN stack.

The ESP32 series is a wide range of microcontrollers that are well-suited to budget-limited, resource-constrained deployments. These have experienced a surge in popularity in consumer and professional work, but NDN testing has not been a focal point on ESP32 devices. For these reasons, this dissertation aims to assess the feasibility of an NDN approach tailored to the ESP-IDF framework on ESP32 devices. Results of this testing provided key insights into microcontroller deployment and a novel adaptation that enabled NDN applications to communicate with other NDN-enabled devices.

The ESP-IDF framework enabled flashing the same application across many devices in the ESP32 series, thereby providing strong interoperability. Multiple applications were adapted from previous work, and new ones were created using this project's adaptation. Tests were carried out on resource usage, caching, and power consumption, with the lightest applications using between 700 - 900KB of flash, and the adaptation layer itself using between 17KB to 30KB of flash depending on the application. RAM usage for NDN features was approximately 10KB, with utilisation also varying on application, cache sizes, and target board. Power usage with constant forwarding averaged 107 mW to 114 mW.

Acknowledgements

I would like to thank all the people who helped me and encouraged me throughout my years at Trinity, in particular my father Paul Power who gave me much support until the very end.

Thanks to my partner Lucy O'Brien who gave me a place to stay when needed, Tom Moroney who gave me a laptop when mine failed and Jake Bolton for inspiring me to continue into the tech workforce by showing me what it can buy (useless items).

Thank you to Prof. Stefan Weber who is consistently a great source of intelligent, thorough knowledge and for all advice he gave me around this work, and for being a fantastic lecturer in the IoT module allowing me to create one of my favourite projects.

Contents

1 Introduction **1**

1.1 Context . . . 1

1.2 Objectives and Scope . . . 2

2 Background **3**

2.1 Named Data Networking . . . 3

2.1.1 Interest and Data Packets . . . 3

2.1.2 Interest Forwarding . . . 7

2.1.3 Receiving and Caching Data . . . 8

2.1.4 Data Authenticity and Integrity . . . 9

2.1.5 Data Encryption . . . 11

2.2 Named Data Networking Forwarding Daemon . . . 12

2.3 Named Link State Routing - NLSR . . . 13

3 State of the Art **15**

3.1 NDN on IoT . . . 15

3.2 CCN-lite . . . 17

3.3 NDN-Lite . . . 18

3.3.1 NDN-riot . . . 18

3.3.2 NDN-iot-package-over-posix . . . 18

3.3.3 NDN-iot-package-over-nordic-sdk . . . 19

3.3.4 esp8266ndn . . . 19

3.4 Hardware Platforms . . . 19

3.5 MCU Frameworks and Operating Systems . . . 21

3.6 Problem Statement . . . 22

4 Design **23**

4.1 Overview . . . 23

4.1.1 ESP-IDF . . . 25

4.2 NDN-Lite Port to ESP-IDF . . . 28

4.2.1 Face selection . . . 28
4.2.2 Library selection . . . 28
4.3 Application Development . . . 31
4.3.1 Secure Sign-on Protocol . . . 32
4.3.2 Simple Manual Verification . . . 33
4.3.3 Application Details . . . 34
4.3.4 Naming Scheme . . . 36
4.4 Summary . . . 36

5 Implementation **38**
5.1 Build System Integration . . . 38
5.2 Platform Adaptation Implementation . . . 39
5.2.1 Clock and Timer . . . 39
5.2.2 Wi-Fi and Network Initialisation . . . 40
5.2.3 Random Number Generation . . . 40
5.2.4 UDP Face . . . 41
5.2.5 Cryptographic Backend . . . 41
5.2.6 Compilation . . . 41
5.3 Basic Applications . . . 43
5.4 Smart Home Sensor Application . . . 47
5.4.1 Device Advertisement . . . 47
5.4.2 Bootstrapping Exchange . . . 48
5.4.3 Data Production and Signing . . . 49
5.4.4 Manager and Dashboard . . . 49
5.5 Issues and Limitations . . . 52
5.6 Summary . . . 52

6 Evaluation **54**
6.1 Flash Usage . . . 54
6.2 RAM Usage . . . 58
6.2.1 Content Store Caching . . . 60
6.3 Latency . . . 61
6.3.1 Content Store Retrieval Times . . . 65
6.4 Optimisation . . . 67
6.5 Power Usage . . . 67
6.5.1 ADALM1000 . . . 67
6.5.2 Loopback Power Use . . . 68
6.5.3 Consumer and Producer Power Use . . . 70
6.5.4 Cryptography Power Use . . . 72

6.5.5 Battery Life Estimation . . . 75
6.5.6 Packet Inspection . . . 76
6.6 Summary . . . 77

7 Conclusion **79**
7.1 Limitations . . . 80
7.2 Future Work . . . 80
7.2.1 Chunking and Throughput . . . 81
7.2.2 New Face Implementations . . . 81
7.2.3 SSP . . . 81
7.2.4 Real Sensor Data . . . 81
7.2.5 Topology Benchmarks . . . 82
7.2.6 Stronger base applications and throughput testing . . . 82

A1 Appendix **92**
A1.1 idf.py menuconfig . . . 92
A1.2 idf.py menuconfig Mbed TLS options . . . 93
A1.3 ADALM1000 Power measurement script . . . 95
A1.4 NDN-Lite manager . . . 97

List of Figures

2.1 Structure of an NDN Interest packet showing required and optional fields 5
2.2 NDN-TLV encoded Interest packet with Nonce and InterestLifetime fields 5
2.3 Structure of an NDN Data packet showing required, optional, and cryptographic fields . . . 7
2.4 NDN-TLV encoded Data packet with mandatory SignatureInfo and SignatureValue fields . . . 7
2.5 Interest forwarding and Data retrieval across an NDN network using the CS, PIT, and FIB . . . 8
2.6 Data return path through intermediate node with Content Store caching and forwarding to PIT entries . . . 9
2.7 Signed Interest command authentication in an NDN IoT network: unsigned Interest accepted (left) versus rejected by signature validation (right) 10
2.8 Multiple face abstraction showing that NFD interprets each differing face as the same entity class for packet forwarding. . . . 13

3.1 Relationship between NDN IoT implementations surveyed in this chapter. 15

4.1 ESP32P4NRW32 chip variant with only HP Core System and External PSRAM shown . . . 25
4.2 Overview of the ESP-IDF layered architecture and its available component categories. Applications are built on top of configurable components, which abstract the underlying ESP32 hardware. . . . 26
4.3 Random Number Generator source on the ESP32-P4 . . . 29
4.4 SSP sign-on protocol between a device and controller, adapted from Li et al. (2019). . . . 33
4.5 Simple Manual Verification bootstrapping protocol. A device broadcasts an advertisement Interest; the operator approves or rejects via the manager dashboard, after which public keys are exchanged. . . . 35

5.1 Default ESP-IDF project structure. . . . 38
5.2 Altered ESP-IDF project structure. . . . 39

5.3 Compile-time selection between Mbed TLS and default cryptographic backends. The ESP32 backends are loaded when `NDN_LITE_SEC_BACKEND_ESP32_MBEDTLS` is defined. Both use the same ESP32 RNG implementation 42
5.4 Architecture of the NDN-Lite ESP-IDF adaptation layer showing the relationship between applications, adaptation components, CMake build configuration, and the underlying NDN-Lite library 42
5.5 Component registration for the adaptation layer (left) and a typical application (right). The application only requires the adaptation component. . 43
5.6 Serial output of the udp-basic-consumer-producer loopback test showing Wi-Fi connection, Interest forwarding, and Content Store cache hits. *Note: The AP SSID and password are no longer the same.* 44
5.7 UART configuration for the serial console interface. 45
5.8 Serial output of the udp-basic-producer showing UART-based FIB setup for unicast Interest reception and Content Store insertion 45
5.9 Corresponding device's serial output of the udp-basic-consumer showing UART-based FIB configuration and successful Data retrieval from a remote producer 45
5.10 Serial output of an ESP32 node during smart home application startup showing ECDSA key generation, multicast face creation, and prefix advertisement 48
5.11 Manager log showing device discovery, route registration, and NFD FIB updates upon receiving an advertisement Interest 48
5.12 Full NDN-Lite Manager dashboard with data retrieval panel, NFD faces, routes, and activity log, see Figure A1.4 in the Appendix for a larger image 49
5.13 NDN-Lite Manager dashboard showing four differing ESP32 devices with trust status, prefix assignments, and registration state 50
5.14 ESP32-P4 is rejected and data is returned by the ESP32-C6 on the prefix /home/temperature 51
5.15 ESP32-C6 returns Data for Interest that has prefix /home/temperature/living-room 51

6.1 Heap consumption by stage on the udp-basic-consumer-producer application across ESP32-P4, ESP32-C6, and ESP32-WROOM-32E, the ESP32-P4 devkit has the lowest due to offloading Wi-Fi functionality to the co-processor 59
6.2 Heap consumption by stage for udp-group-producer (a) and udp-group-consumer (b) on the ESP32-P4 comparing Mbed TLS and TCC/MECC backends 60

6.3 Heap consumption during Content Store fill on the ESP32-P4 with a maximum of 50 entries at 172 bytes each 61
6.4 ECDSA sign and verify latency across ESP32 variants 62
6.5 AES-CBC-128 encrypt and decrypt latency across ESP32 variants 63
6.6 SHA-256 sign latency across ESP32 variants comparing Mbed TLS and TC and MECC backends, these latency differences are extremely small and therefore cryptographic back-end choice based on these results is not necessary 64
6.7 SHA-256 verify and NDN overhead latency across ESP32 variants comparing Mbed TLS and TC and MECC backends, results are nearly the same for all back-end choices 65
6.8 Basic application forwarder polling loop 65
6.9 Content Store round-trip time per request on the ESP32-P4 with udp-basic-consumer-producer over 100 iterations showing CS hits and periodic cache expiry misses, due to being on the same device, the timing is synchronous. 66
6.10 RTT between two ESP32-P4's one running udp-basic-consumer, the other running udp-basic-producer. 100 iterations showing CS hits and periodic cache expiry misses and retrieval times varying greatly due to forwarding logic. 66
6.11 Build optimisation comparison on the ESP32-P4 for udp-basic-consumer-producer showing binary image size and median verify latency. 67
6.12 Current draw measurement circuit setup using ADALM1000 and ESP32C6-Devkit-C1 with exposed J5 header and 1 Ω resistor 68
6.13 Current measurement at device boot, before Wi-Fi initialisation on udp-basic-consumer-producer, the program was paused in between stages in order to better capture current draw at each stage 69
6.14 Current measurement on udp-basic-consumer-producer during Wi-Fi initialisation 69
6.15 Current measurement on udp-basic-consumer-producer post Wi-Fi initialisation before any packet forwarding or receiving has begun, the Wi-Fi module will pull around 80mA during Wi-Fi RF events which is consistent with the ESP32-C6 datasheet 70
6.16 Current draw when udp-basic-consumer-producer is running, there are no Wi-Fi TX events due to the device forwarding packets to itself. The average power is 150mW as there is very little delay. Interest and Data are being forwarded constantly. This means that ECDSA signing and verification, as well as AES encryption and decryption, occur back-to-back. . . 70

6.17 Sixty-second window of current measurement on udp-group-consumer with visible Wi-Fi TX events for each Interest forwarded, not every TX event was captured due to the fact that figures were captured live, slightly reducing sampling rate. In total 30 Interest were satisfied 71
6.18 Sixty second window of current measurement on udp-group-producer with similarly visible Wi-Fi TX events for each Data returned 71
6.19 Sixty-second window of current measurement on udp-group-consumer when the CS is turned on, and Data FreshnessPeriod is set to 5000ms. This plot shows a reduction in Wi-Fi TX events but a larger amount of ECDSA verification due to Data being available in the CS 72
6.20 ESP32-C6 current draw during 500 iterations of ECDSA key pair generation without Wi-Fi activity 73
6.21 ESP32-C6 current draw during 500 iterations of ECDSA signature operations 73
6.22 ESP32-C6 current draw during 500 iterations of ECDSA verification operations 74
6.23 ESP32-C6 current draw during 500 iterations of AES-CBC encryption, these operations happen in a matter of microseconds, meaning many of them are missed by the ADALM1000, max values give an idea of the cost of one operation 74
6.24 ESP32-C6 current draw during 500 iterations of AES-CBC decryption . . 74
6.25 Advertising Interest packets and Data response captured 76
6.26 Advertising Interest reported from the ESP32-WROOM-32E 76
6.27 Interest and Data matching in Name, in accordance with NDN TLV v0.3 76
6.28 Interest with CanBePrefix parameter set and Data returned with the longer child prefix name 77
6.29 Top: Interest breakdown (left) and no-prefix (right). Bottom: Data breakdown. Both Interests return the same Data 77

A1.1 ESP-IDF menuconfig example, parameters could be set in here manually and saved to an sdkconfig file. This file could also be manually created and edited without entering this menu 92
A1.2 Mbed TLS hardware acceleration parameters can be altered in the menuconfig, however when using compatible devices these hardware acceleration parameters will be activated by default when using Mbed TLS 93
A1.3 The ADALM1000 board setup with a 1 Ω shunt resistor and an ESP32-C6-DevkitC-1 with its J5 header removed to access the current measurement pins 94
A1.4 Full NDN-Lite Manager dashboard showing data retrieval panel, NFD faces, routes, and activity log. 98

List of Tables

3.1 ESP32 variant specifications 20

4.1 Development boards selected for this project 24
4.2 Cryptographic hardware acceleration by ESP32 variant 27
4.3 NDN-Lite adaptation layer mappings to ESP-IDF 30

5.1 UDP example applications from ndn-iot-package-over-posix 43

6.1 ESP32-P4 Flash Memory Usage Summary for udp-basic-consumer 55
6.2 ESP32-P4 Flash Usage by Component (Top 9 by Size) 55
6.3 ESP32 WROOM-32E Flash Usage by Component (Top 9 by Size) 56
6.4 Total Binary Image Size by Device for udp-basic-consumer 57
6.5 Networking Library Flash Usage: ESP32-C6 vs ESP32-WROOM-32E . . 57

Nomenclature

BLE	Bluetooth Low Energy
CS	Content Store
ECDSA	Elliptic Curve Digital Signature Algorithm
FIB	Forwarding Information Base
HMAC	Hash-based Message Authentication Code
ICN	Information-Centric Networking
IoT	Internet of Things
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
MTU	Maximum Transmission Unit
NDN	Named Data Networking
NFD	Named Data Networking Forwarding Daemon
NLSR	Named-data Link State Routing
NVS	Non-Volatile Storage
PIT	Pending Interest Table
PSRAM	Pseudo Static Random Access Memory
RIB	Routing Information Base
RNG	Random Number Generator
RTOS	Real-Time Operating System
SHA	Secure Hash Algorithm
SoC	System on Chip
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
TLV	Type–Length–Value
UDP	User Datagram Protocol

1 Introduction

1.1 Context

Content-Centric Networking (CCN) (Jacobson et al., 2009) was among the early proposed data-centric solutions and has evolved into what is now known as NDN (Zhang et al., 2010), which has a solid research base and numerous successful demonstrations. However, current implementations of the NDN architecture primarily focus on applications that can utilise resources typical of devices such as servers, desktops, laptops, and other personal computing hardware that are considered unconstrained for typical day-to-day networking tasks. This has allowed NDN to grow and be tested in a multitude of ways that realise the concepts it introduces, with two main examples being the Named Data Networking Forwarding Daemon (NFD) (Afanasyev et al., 2021) as well as Named Link State Routing (NLSR) (Hoque et al., 2013); however, these are both limited to the previous devices mentioned.

Modern microcontrollers (MCUs) have become more powerful while maintaining a small, budget-friendly form factor, making them a strong choice for data collection systems in many professional and academic environments. Although they provide significant utility, they often operate on the edge, leaving them prone to issues such as lossy network connections, disconnections due to environmental conditions, and data loss when a device goes offline. However, these issues in data collection and distribution are where the core principles of NDN can be used to provide solutions. For these reasons, there is a real need for implementations that enable NDN applications on these devices.

Although some frameworks enable such implementations, many do not fully leverage modern MCU architectural features. This project aims to understand the current support for deploying NDN applications on MCUs, to investigate how NDN can be implemented on constrained MCUs, and to characterise the resource and power usage of such an implementation.

1.2 Objectives and Scope

To understand the full extent of NDN deployment on MCU devices, the following objectives are listed in chronological order to aid understanding of the work carried out throughout this project.

- Initial review of NDN architecture and its use on unconstrained hardware.
- Survey of MCU devices that may support NDN implementations.
- Review of previous implementations on applicable MCU devices.
- An implementation based on previous work.
- Creation of light test applications based on such an implementation.
- Development of a heavier application as an NDN MCU proof of concept using the light applications.
- Testing of base networking activity, such as resource usage, power usage and data retrieval speed.

With all objectives met, the work resulted in a successful implementation that can be used for further NDN research and testing on MCUs. The solution adhered to guidelines and principles established by previous NDN implementations to ensure it is future-proof, modular, lightweight, and therefore portable, and it showed that it is not constrained to communicating only with other MCUs but can also communicate with an NDN application running on unconstrained hardware (NFD).

2 Background

This section introduces NDN, explains its underlying concepts, and describes the contemporary applications that implement it. Then, a review of MCUs and their supporting frameworks that best support NDN implementation.

2.1 Named Data Networking

Modern networking assigns every connecting device a unique IP address to route data to other machines. This model successfully supports the entire Internet; however, it contrasts with the predominant driver of traffic, which is data retrieval. To the average user, where the requested data comes from is largely irrelevant. NDN conceptually shifts this by assigning each piece of data a hierarchical name composed of components separated by `/`, similar to a file path (e.g. `/cso/databases/housing/graph`), where each component is a child of the preceding one in the hierarchy (Zhang et al., 2014). In NDN, when a device requests data, it uses such a name, and any node that contains the matching data can then route it back to the original requester without knowing the original node's location. NDN eliminates three major problems that IP routing has had to work around, namely NAT traversal, address space exhaustion, and address management (Zhang et al., 2014) by introducing the following concepts:

2.1.1 Interest and Data Packets

Interest

The means by which NDN nodes make these requests is via *Interest packets*, which are typically referred to as "Interest". All NDN packets conform to the NDN-TLV specification, currently at version 0.3 (Named Data Networking, 2025). A node that sends an Interest is called a consumer, and this packet requires only a name in its encoding, however it often carries more than this. In NDN, consumers always initiate an exchange with an Interest, as unsolicited data is against NDN specification. Below are the explanations of the possible optional values to assist in forwarding and routing.

Mandatory

Name

Hierarchical components separated by "/". Must contain at least one component, or the Interest will be discarded.

Core

Nonce

A randomly generated fixed 4-byte value combined with the name to uniquely identify an Interest, used to detect and discard looping Interests.

InterestLifetime

Time-to-live of the Interest in milliseconds. Defaults to 4000ms if omitted. Once this time has elapsed, it is considered expired and discarded.

HopLimit

A 1-byte unsigned integer in the range [0, 255]. Decremented at each hop. An Interest with a HopLimit of 0 may only be satisfied locally and must not be forwarded.

CanBePrefix

When present, parent name elements can allow a prefix match. When absent, only exact name matches are accepted.

MustBeFresh

A zero-length flag. When set, only Data within its FreshnessPeriod will be returned. Without this flag, stale cached Data may be returned.

Metadata

ForwardingHint

Contains a list of delegation names providing alternative forwarding paths to reach the requested Data.

ApplicationParameters

Arbitrary bytes that parameterise the request. Requires an Interest parameters digest component in the name to ensure uniqueness when present. May optionally contain an *InterestSignature*, which consists of *InterestSignatureInfo* storing metadata about the signature including the SignatureType and KeyLocator, and *InterestSignatureValue* holding the cryptographic signature bytes. When present, the Interest is considered a Signed Interest. Discussed further in Section 2.1.4.

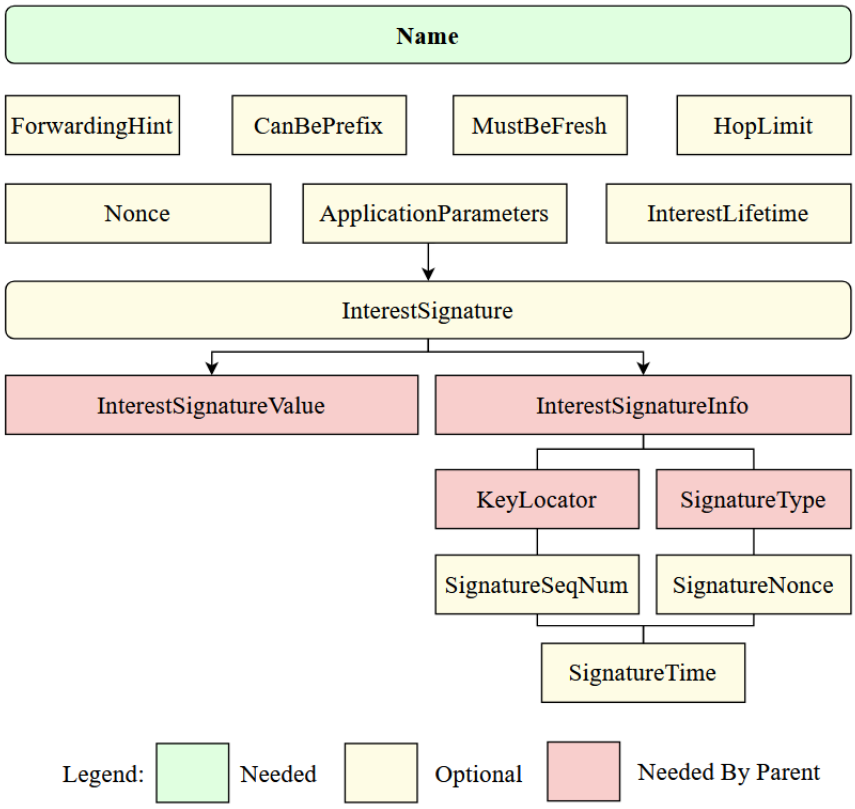


Figure 2.1: Structure of an NDN Interest packet showing required and optional fields

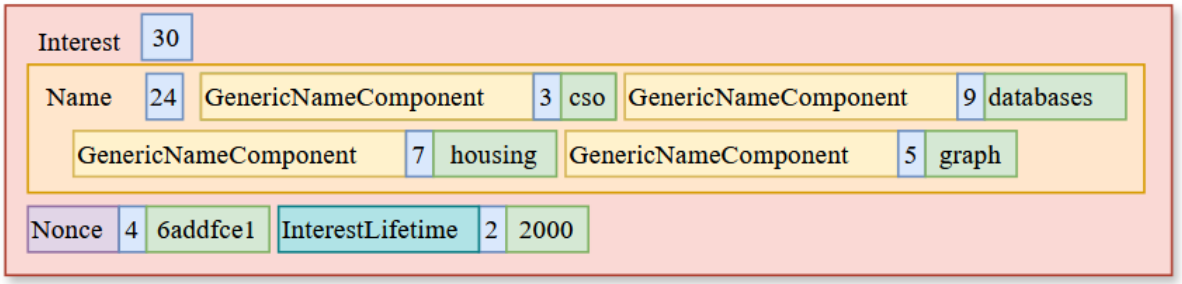


Figure 2.2: NDN-TLV encoded Interest packet with Nonce and InterestLifetime fields

Data

A node that replies with Data is called a producer. A producer that receives an Interest and holds the matching data identified by the prefix will return a *Data packet* or just Data for short. Data must have a name and a DataSignature to be valid (Zhang et al., 2014). Nodes may be a producer and consumer simultaneously, sending Interest to retrieve Data and replying to Data upon receiving Interests. For constrained devices, focus was placed on implementing applications that used as little resources as possible, therefore the fundamental difference between the two is of higher importance in such deployments. An example of such is an Internet of Things (IoT) sensor network of nodes that only replies to Interests; all of the replying nodes would be considered data *producers*.

Mandatory

Name

Same structure as Interest Name. Must contain at least one component.

DataSignature

A mandatory field consisting of two parts. *SignatureInfo* stores metadata about the signature and *SignatureValue* holds the cryptographic signature bytes computed over the packet. Both are discussed in Section 2.1.4.

Core

Content

The data payload itself, encoded as an arbitrary sequence of bytes. This element is optional, but usually will be present.

Metadeta

The following three fields are contained within a *Metainfo* field.

FreshnessPeriod

Time in milliseconds after arrival, after which the Data is marked non-fresh. A value of zero or absent causes the Data to be marked non-fresh immediately after satisfying pending Interests. Non-fresh Data remains valid but will not satisfy Interests with `MustBeFresh` set.

ContentType

A positive integer indicating the content type. Types include:

BLOB (0)	arbitrary payload
LINK (1)	list of delegations, pointers to where data may be
KEY (2)	cryptographic public key
NACK (3)	application-level negative acknowledgement. Defaults to BLOB if omitted.

FinalBlockId

A name component identifying the last block in a sequence of fragmented Data. Should be present in the final block and may appear in earlier fragments to signal the end of the sequence to consumers.

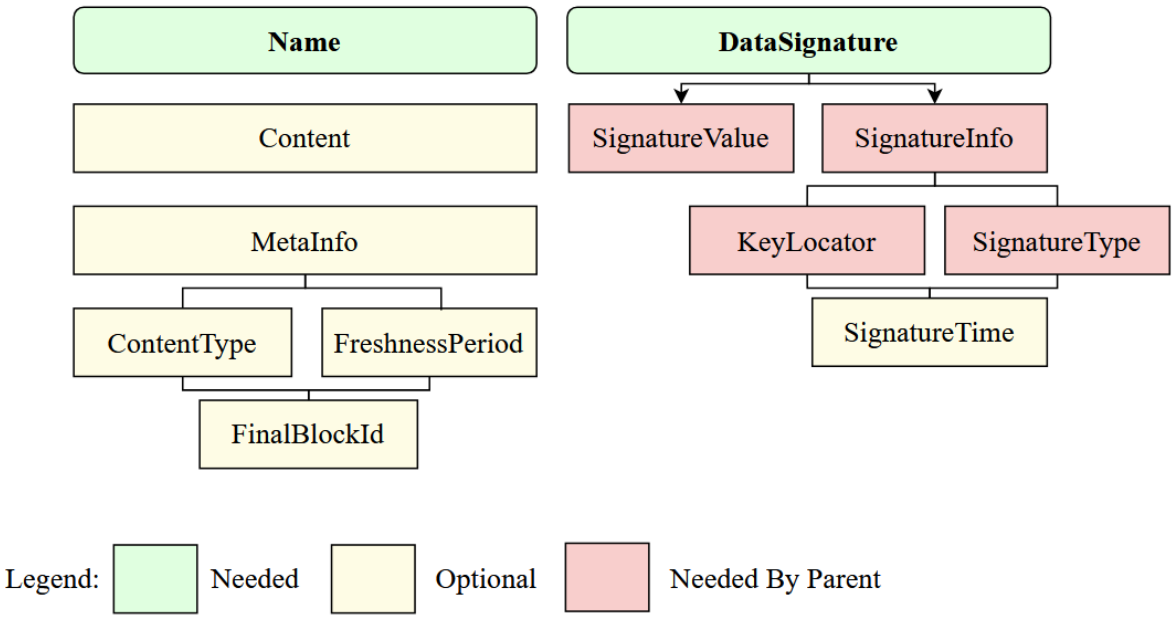


Figure 2.3: Structure of an NDN Data packet showing required, optional, and cryptographic fields

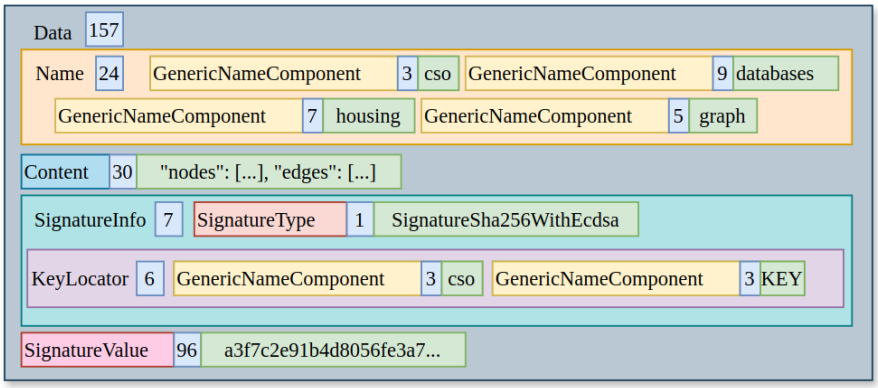


Figure 2.4: NDN-TLV encoded Data packet with mandatory SignatureInfo and SignatureValue fields

2.1.2 Interest Forwarding

Nodes in NDN rely on the following fundamental concepts:

- Forwarding Information Base (FIB) maps name prefixes to faces and is checked when Interests are to be forwarded.
- Pending Interest Table (PIT) records faces and prefixes for Interests that could not be satisfied.
- Content Store (CS) caches Data packets at each node.

When a consumer sends an Interest, it is forwarded into the network. Each router checks whether the name field in the Interest matches Data stored in the CS. If a copy has been

cached, it will be returned immediately along the reverse path, following entries in each node's PIT. Otherwise, the router consults its FIB for a prefix match and forwards the Interest on the corresponding face, creating a PIT entry that records the incoming face and the Interest name. This is how Data can follow the reverse path back the original consumer. This process will be repeated, device by device, until either a node with a matching Data is reached or the Interest expires, at which point each node removes the corresponding PIT entry.

If an Interest arrives at a node that already has a PIT entry for the same name, indicating that another consumer has already requested that Data, the new incoming face is added to the existing entry and the Interest is not forwarded further.

This aggregation behaviour is one of the scalability advantages NDN has over IP (Zhang et al., 2014): in traditional IP networking, simultaneous requests for the same content generate independent flows that can overwhelm the network if demand grows faster than it can be satisfied.

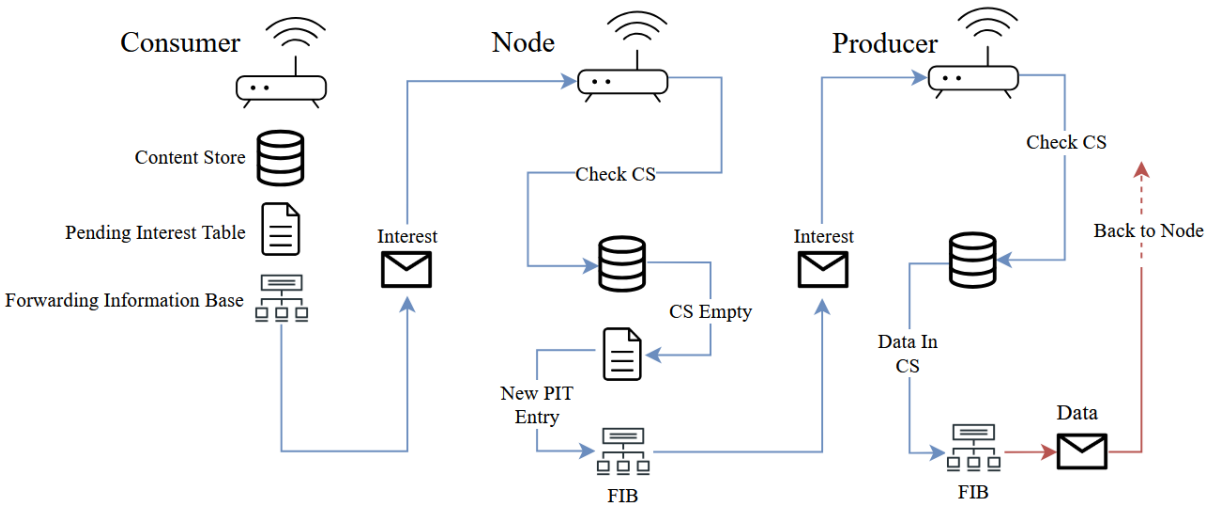


Figure 2.5: Interest forwarding and Data retrieval across an NDN network using the CS, PIT, and FIB

2.1.3 Receiving and Caching Data

When the Interest reaches the correct producer with matching named data, it will forward the Data by checking the PIT to determine which face the Interest came from. As the Data is forwarded back through each node, nodes will remove the corresponding PIT entry, continuing from each node until returning Data to the consumer that first sent the Interest. For PIT entries that consist of a single prefix and multiple faces, the Data will be forwarded to each face.

If the nodes along this path can support a Content Store, they can cache the data at each node as it travels from the producer to the consumers. Afterwards, if another consumer

requests the same data, it may retrieve it from any of these nodes (Zhang et al., 2014). An NDN network of nodes that lose connection and reappear allows the distribution of content without relying on any single node to rejoin.

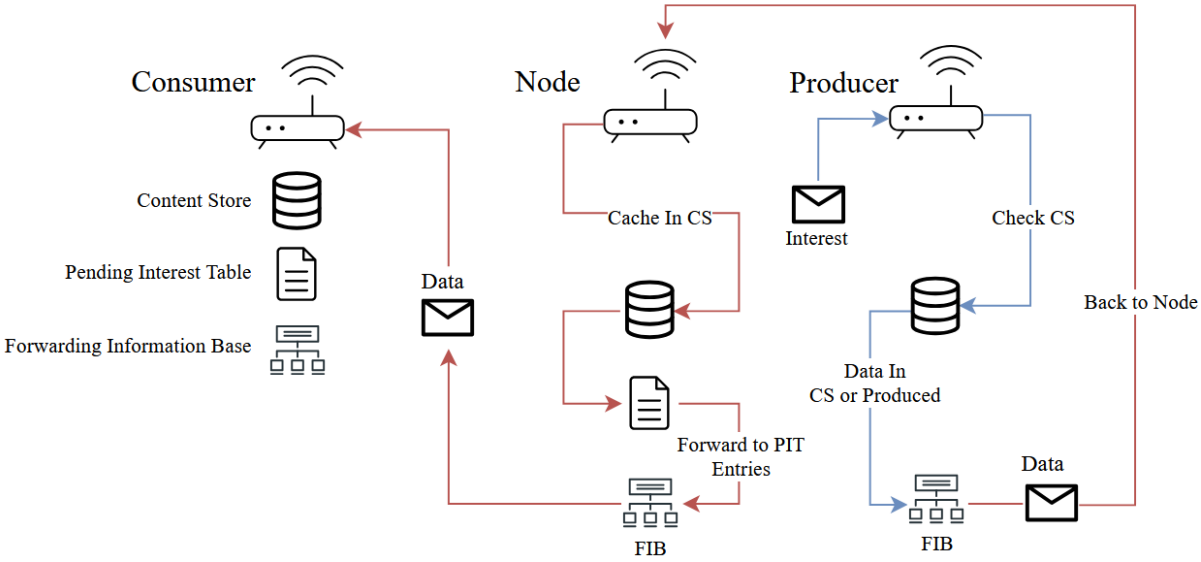


Figure 2.6: Data return path through intermediate node with Content Store caching and forwarding to PIT entries

2.1.4 Data Authenticity and Integrity

Regular networking, such as TCP/IP, secures the channel in which processes operate by using protocols such as TLS (Rescorla, 2018). If that channel is interfered with or the data leaves it, it is no longer secure. A cached copy of data lacks proof of authenticity, leaving it vulnerable to alteration by malicious attackers. NDN prevents this by securing each individual data packet itself through a signature at the time of production (Shang et al., 2016). Therefore, data can travel around regardless of which node or device it is being forwarded from or which device is caching it, and data integrity can be assumed.

To ensure this, every NDN data packet must be cryptographically signed, and this cannot be opted out of (Zhang et al., 2018b). This creates a network with security by design, where every piece of content has a verifiable origin. Because the signature and name are used together to verify the data, if a third party tries to interfere with it, its signature will no longer be able to authenticate that it came from the original producer.

When signatures are generated at the time of production using the producer's private key (Zhang et al., 2018b), they are stored in the DataSignature field as the SignatureValue parameter. While Data packets are always signed, Interest packets may also carry a signature when the use case demands it, stored in the InterestSignatureValue. One such

case is for sending commands, which Burke et al. (2013) describes as necessary for tasks such as command authentication.

Consider a homeowner, Jim, who uses an NDN IoT network to remotely control his heating system while he is frequently away from home. Normal operation involves Jim sending an Interest to his thermostat, encoding a command, and the thermostat responding with a Data packet confirming the change. His neighbour Bob, who lives directly above him, discovers that Jim's heating warms his own apartment and decides to exploit this. Because Jim's thermostat is not configured to require or validate signed Interests, Bob can send his own Interest packets to the thermostat and raise the temperature freely. Had Jim's thermostat been implemented to require signed Interests, it would verify the signature on any incoming command Interest against a set of trusted keys, specifically those belonging to Jim's namespace. Bob, holding no such key, would have his Interests discarded before any command is executed.

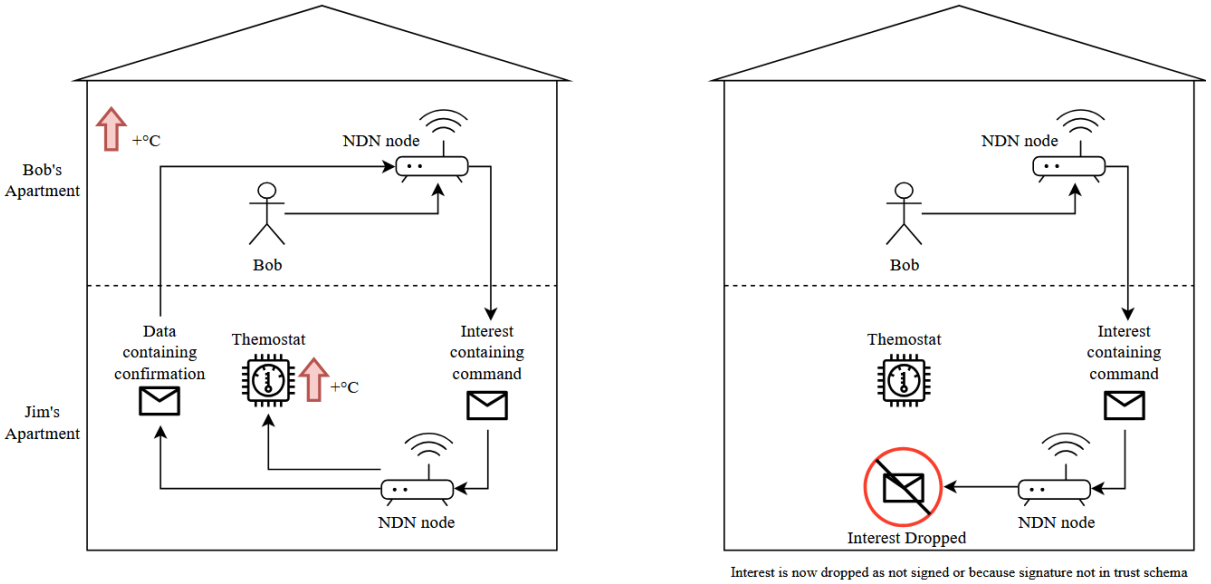


Figure 2.7: Signed Interest command authentication in an NDN IoT network: unsigned Interest accepted (left) versus rejected by signature validation (right)

To understand how these signatures are verified, it is necessary first to understand how entities and certificates are managed in NDN. Entities in an NDN network will own one or more names and prove ownership of each name through an NDN certificate. This binds the name to a cryptographic public-private key pair (Zhang et al., 2018b). Certificates within NDN can be treated as Data, allowing them to be requested and cached in content stores. Certificate names will follow a structured naming convention of the form `/<prefix>/KEY/<key-id>/<issuer-info>/<version>`. The prefix in this instance identifies the entity that owns the key, and the remaining name components identify the specific key instance, the issuer and the version. In Jim's network, the thermostat would have a certificate like so: `/home/jim/thermostat/KEY/001/jim/001`, clearly identifying

the key as belonging to Jim's namespace.

When a consumer receives a Data packet and needs to verify it, it does so using the *SignatureInfo* field embedded in the packet. This contains a *KeyLocator* identifying the name of the signing key (Zhang et al., 2018b). The consumer can then use this to construct an Interest and retrieve the corresponding certificate from the network. The public key within that certificate can then be used to verify the *SignatureValue*. As the certificate itself is a signed Data, it must also be verified. The consumer will continue to fetch certificates using the same method until the chain terminates at a locally installed trust anchor. A Data packet will be considered valid only if every certificate in the chain has a valid signature and satisfies the consumer's trust policies. In the case of Jim, the thermostat application would follow this chain from the device certificate to Jim's own certificate, which serves as the trust anchor for his home network.

All signature verification in NDN terminates at an established trust anchor. In TCP/IP, this anchor is typically a commercial Certificate Authority (CA) such as Let's Encrypt or DigiCert, whose root certificate comes pre-installed in operating systems and browsers. The issue with this model is that there is implicit trust being placed in third-party applications. This became apparent when Let's Encrypt (Cimpanu, 2017) issued 14,566 valid SSL certificates to sites impersonating PayPal, indicating that CAs may not guarantee legitimacy. NDN addresses this by removing this reliance entirely. By following the Simple Distributed Security Infrastructure (SDSI) model (Rivest and Lampson, 1996), each system on the network can establish its own trust anchor bound by its own namespace Zhang et al. (2018b).

Knowledge of the trust anchor is insufficient in proving authenticity. There must also be rules that determine which keys can sign what data. The NDN solution is trust schemas, which define relationships between data names and signing key names in a systematic way (Yu et al., 2015). The consumer can assess this trust schema to see if a signing key has the permission to sign a packet before performing any cryptographic verification. For example, in Jim's network, a trust schema could restrict thermostat command acknowledgements to only be signed under the `/home/jim/thermostat` namespace. This means even a valid signature from outside the namespace will be rejected.

2.1.5 Data Encryption

Unauthorised nodes may still read content from a Data packet with just the above implementation. In a network where any node might cache and produce data, and that data must not be seen by a third party, NDN employs data encryption through Name-based Access Control (NAC). This will encrypt data on a per-namespace basis (Zhang et al., 2018a). Instead of securing the channel through which data travels, NAC encrypts

the content of the Data packets themselves just as with Data signatures. Only authorised consumers with the correct decryption keys will access the content regardless of which node it was retrieved from. As with certificates, the keys used in this process are distributed as named NDN data packets.

2.2 Named Data Networking Forwarding Daemon

The Named Data Networking Forwarding Daemon (NFD) is the reference implementation of the NDN protocol and has been one of the primary platforms for NDN research and application development since its release in 2014 (Afanasyev et al., 2021) (latest revision was in 2021). It runs as a daemon on Linux, macOS, and Android, and its architecture enables it to operate on top of existing network infrastructure rather than replacing it.

NFD implements the NDN forwarding plane using the concepts previously introduced, such as the PIT, CS and FIB. It exposes a pluggable forwarding strategy layer, allowing different forwarding strategies to be configured on a per-namespace basis (Afanasyev et al., 2021) and relies on routing strategies to forward Interest. Default strategies include the Best Route strategy, which forwards each Interest to the lowest-cost upstream face, and the Multicast strategy, which can forward an Interest to all available upstream faces. Multicast routing strategies like this are particularly useful in IoT deployments where producer locations may not be known in advance, and service discovery via multicast Interest can be utilised.

NFD abstracts all transport mechanisms to a single structure, a face. This allows networking interfaces such as Ethernet, UDP, TCP tunnel, Unix domain sockets, or Bluetooth to be represented and treated equally in the forwarding pipeline (Afanasyev et al., 2021). Local applications can communicate with NFD via a Unix socket face, making local process communication seemingly equivalent to remote NDN communication.

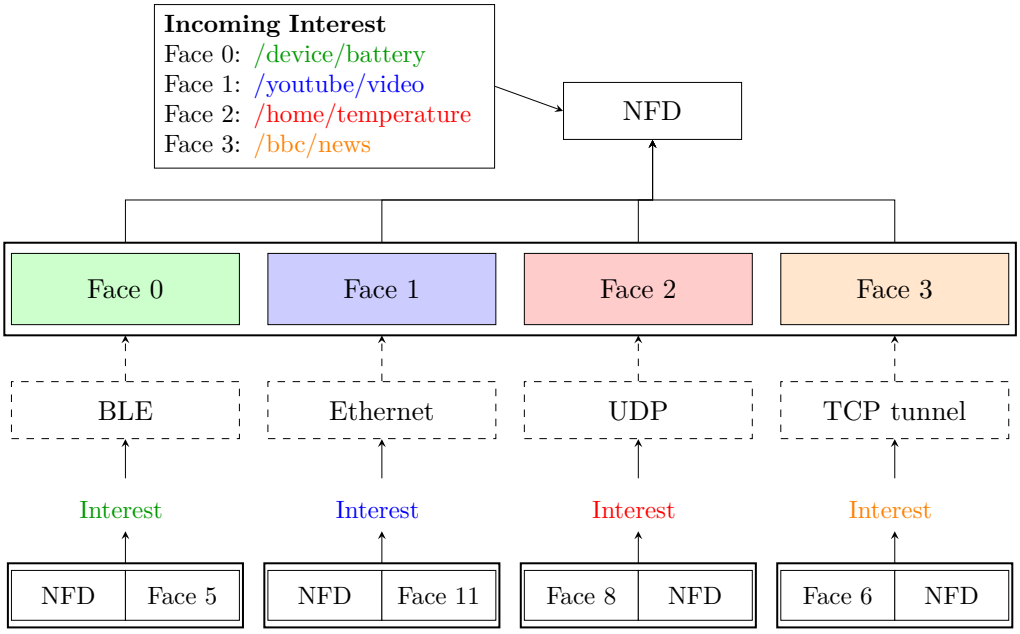


Figure 2.8: Multiple face abstraction showing that NFD interprets each differing face as the same entity class for packet forwarding.

NFD's Forwarding Information Base can be populated through a Routing Information Base (RIB), which acts as an intermediary between routing sources and the forwarding plane (Afanasyev et al., 2021). An RIB stores routes learned from sources such as routing protocols (e.g., Named Link State Routing, NLSR) and application prefix registrations. When a route is registered or removed, the RIB manager invokes an FIB Updater, which processes route inheritance flags and generates the corresponding FIB updates.

While NFD is well-suited to general-purpose computing environments, its resource usage makes it unsuitable for constrained IoT devices. NFD's installed footprint on PC is 124 MB with an additional 144 MB dependency on the ndn-cxx library (Named Data Networking, 2026a), bringing the combined storage up to approximately 268 MB. There have been successful attempts to refactor NFD, achieving a 6 MB executable (Newberry et al., 2021), the memory usage still exceeds constrained use cases. Constrained IoT devices selected and used in this project are several orders of magnitude below typical NFD's requirements, and this incompatibility motivated the development and adaptation of NDN concepts specifically targeting constrained IoT hardware, as discussed later in Section 3.1 onwards.

2.3 Named Link State Routing - NLSR

NLSR is the primary routing protocol deployed alongside NFD that populates the RIB of an NFD instance (Hoque et al., 2013). While NFD's FIB can be configured manually using the `nfdc` management tool, this approach does not scale in dynamic multi-node

networks where topology changes occur frequently. NLSR provides an automated routing service that discovers reachable name prefixes across the network and computes the best forwarding paths to them. This can keep NFD's FIB up to date without manual intervention.

NLSR follows a different approach compared to IP link-state protocols such as OSPF (Moy, 1997). Updates to nodes throughout the network are propagated via Interest and Data rather than a separate protocol. This is done through Link State Advertisements (LSAs), which are stored and retrieved as NDN Data packets. Since every NDN Data packet has a cryptographic signature, every LSA can be authenticated. Routers receiving a routing update can verify the claimed origin that generated it.

To calculate the best-cost routes, NLSR runs a modified Dijkstra's algorithm that produces a ranked list of multiple next hops for each name prefix (Hoque et al., 2013). These routes are registered with the RIB of an NFD, which processes them alongside any other route sources and generates the corresponding FIB entries that NFD uses for Interest forwarding (Afanasyev et al., 2021).

NLSR uses two types of LSAs to build a network understanding. The first is Adjacency LSAs, which can advertise active links between neighbouring routers, and the second is Prefix LSAs, which advertise name prefixes registered at a router. Both take the hierarchical naming form of `/<network>/<site>/<router>/NLSR/LSA`. This allows the origin of any LSA to be verified against the naming hierarchy. To coordinate the sending of LSAs, NLSR originally used ChronoSync (Zhu and Afanasyev, 2013) but has migrated to using Psync (Zhang et al., 2017). Each router maintains a hash of the database it uses to hold link information, called the Link State Database (LSDB), and neighbouring routers periodically exchange these hashes using Interest and Data packets. When a mismatch between hashes is detected, a router with an outdated LSDB will send an Interest for the missing LSA by name and can update its LSDB once the Data has been returned.

Like NFD, NLSR is designed for general-purpose computing environments and is not suitable for deployment on constrained IoT devices. NLSR has the same dependency on ndn-cxx and requires a POSIX environment with an NFD instance running to communicate with. Not accounting for storage size, NLSR involves cryptographic signature verification of LSAs and ongoing hash comparisons across its LSDB, which are computationally expensive.

3 State of the Art

This chapter surveys existing NDN implementations for IoT devices and the hardware and software platforms on which they depend. Figure 3.1 shows the key projects that entail NDN IoT implementations such as NDN-Lite which serves as the core lightweight library, with platform-specific adaptations targeting RIOT-OS, POSIX, and Nordic SDK environments, while CCN-lite and esp8266ndn represent independent implementations. The gap this section identifies, and the project's overall focus, is determined to be an NDN-Lite adaptation for ESP-IDF.

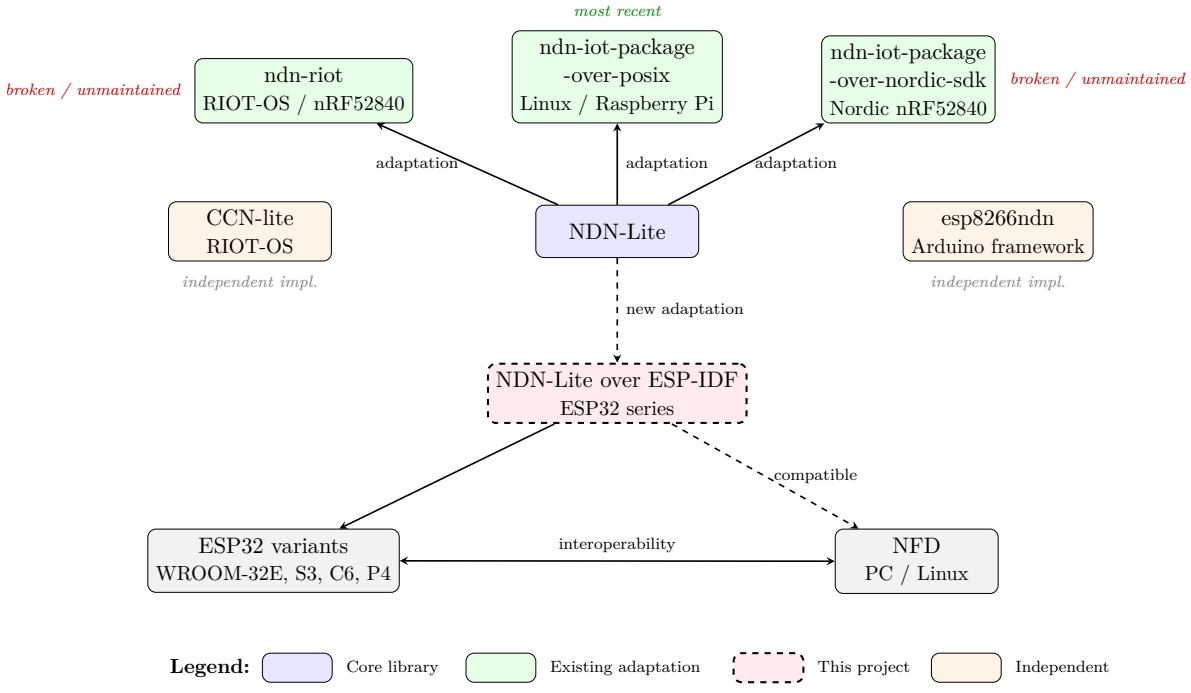


Figure 3.1: Relationship between NDN IoT implementations surveyed in this chapter.

3.1 NDN on IoT

The Internet of Things (IoT) can be defined as the resource-limited devices that enable the interconnectedness of services through networks, which often interact with the real world (Zhang and Tao, 2021). Many environments and applications benefit from using

IoT or have demonstrated real-world success, such as smart homes (Li et al., 2011), healthcare (Catarinucci et al., 2015), and smart grids (Guan et al., 2017). Industry has indicated that the total number of IoT devices was approximately 21.1 billion in 2025 (IoT Analytics, 2025) with continued projected growth. Deployment of IoT devices with embedded hardware often restricts memory capacity to kilobytes and computational power to MegaFLOPS, resulting in power consumption in the milliwatt range (Baccelli et al., 2014) vastly contrasting to general computing power.

Constrained IoT devices are formally classified by RFC 7228 (Bormann et al., 2014) into three categories based on available memory and storage.

- Class 0 devices have less than 10 KiB of RAM and 100 KiB of flash and are so constrained that they need support from external devices for communication.
- Class 1 devices sit in the 10 KiB of RAM and 100 KiB of flash range and can run protocols specifically designed for constrained environments, such as CoAP.
- Class 2 devices contain approximately 50 KiB of RAM and 250 KiB of flash and support most standard protocol stacks while still benefitting from lightweight implementations.

Higher-tier constrained devices beyond Class 2 provide hundreds of kilobytes of RAM and several megabytes of flash and are becoming increasingly popular for embedded AI applications (Moin et al., 2022). These are still much below the requirements of NFD and NLSR, and it is often between Class 2 and above that lightweight NDN libraries target. The following sections discuss implementations developed for such devices.

IoT devices will typically be powered either directly from a mains connection or with battery power and usually rely on publish-subscribe protocols such as MQTT (OASIS, 2014) to relay information to a remote device or the cloud. Issues arise with these systems when devices must communicate over low-power, lossy wireless networks (Homaei et al., 2025). Protocols such as the IPv6 adaptation layer 6LoWPAN (Alexander et al., 2012) and the Constrained Application Protocol (CoAP) (Shelby et al., 2014) have been developed to adapt IP-based communication to these situations. However, comparisons of NDN with MQTT and CoAP on constrained hardware have demonstrated that NDN outperforms these protocols in certain scenarios. Gündoğan et al. (2018) conducted an experimental evaluation on Class 2 devices across single-hop and multi-hop IEEE 802.15.4 networks with up to fifty nodes. In single-hop experiments, NDN, MQTT and CoAP could perform at a similar level, but when experiments extended to multi-hop testing, performance began to diverge. A packet loss rate of up to 30% with CoAP was observed, and testing showed a high number of retransmissions compared to NDN. NDN nodes

were able to outperform CoAP, losing only 5% of its chunks due to how its hop-by-hop forwarding nature and its utilisation of the Content Store for in-network caching.

The NDN flash usage was reported to be the leanest at around 30KiB which was 40% less than the equivalent MQTT and CoAP stacks due to using CCN-Lite, their reference IoT implementation of NDN (Baccelli et al., 2014). The study found that the RAM usage was also lower at around 8,700 Bytes compared to 30 KiB for the IP-based protocol stacks with an extra 10KiB being used for the Content Store in the NDN tests.

Further evidence for use of NDN in IoT environments can be seen from later experiments by Gündoğan et al. (2021), who demonstrated the possibility of distributing firmware updates across constrained devices. Using a thirty node multi-hop network with devices containing 64KB of RAM and 256KB of flash, the system distributed updates of 128KB across topologies of up to seven hops. NDN's Content Store enabled retransmissions to be resolved locally, rather than having the transmission traverse the full request path to the original gateway. A cascading retrieval strategy was used that allowed each node to update itself and then forward chunks downstream. With this method, nodes could update in three minutes, and when disabling the Content Store, the same update process would take over two hours, demonstrating the in-network caching advantages.

A survey of existing NDN implementations for IoT devices was conducted, with preference on projects that have proven deployment capabilities and have somewhat recent development. These will be discussed in the next section.

3.2 CCN-lite

CCN-Lite is a lightweight C implementation of NDN concepts designed to run on constrained IoT hardware. It was among the first NDN implementations demonstrated on IoT devices, with Baccelli et al. (2014) porting it to the RIOT-OS operating system and deploying it across a sixty-node building automation testbed. Their results show the contrast in memory footprint compared to IP-based alternatives. NDN required 16,628 bytes of ROM and 5,112 bytes of RAM on RIOT hardware, compared to 53,412 bytes of ROM and 27,739 bytes of RAM for the equivalent RPL + 6LoWPAN stack and also showed 50% less retransmissions in multi-consumer scenarios when using caches.

This study demonstrated that NDN concepts could be feasibly implemented within the memory constraints of MCU devices, but because CCN-Lite was primarily developed as a research prototype to demonstrate the possibilities of embedded NDN rather than as a maintainable framework, it has received little to no maintenance in the last few years, and does not fully conform to the current NDN-TLV 0.3 specification (Named Data Networking, 2025).

3.3 NDN-Lite

NDN-Lite is a lightweight NDN library designed for cross-platform IoT deployment applications on constrained hardware, originally introduced as part of the (Named Data Networking of Things) NDNoT framework (Zhang et al., 2018a). NDN-Lite's library modularity allows developers to create adaptations for any device that supports raw C. This is achieved through an abstraction layer that lets device-specific functions and methods be wrapped using the NDN-Lite API. NDN-Lite provides a set of IoT-oriented modules and example applications, primarily demonstrated with their POSIX implementation (Named Data Networking for IoT, 2024). Existing adaptations are available in the Named Data Networking for IoT collection hosted on GitHub (Named Data Networking for IoT, 2026a), and their state was reviewed for possible deployment.

3.3.1 NDN-riot

NDN-riot provides an NDN-Lite adaptation for RIOT-OS, replacing an earlier standalone implementation which had documented memory management limitations (Named Data Networking for IoT, 2018). It supports IEEE 802.15.4 communication and has been deployed on Nordic nRF52840 hardware in a smart-home context, as demonstrated by the Sovereign framework (Zhang et al., 2022). However, this library has been reported as broken in the NDN-Lite overview documentation (Named Data Networking for IoT, 2026b), which, although it may be outdated, was not verified for confirmation due to the commit history showing that this repository has not been updated in eight years, whereas RIOT-OS has been actively maintained, meaning a divergence in implementation standards are likely to have taken place.

3.3.2 NDN-iot-package-over-posix

NDN-iot-package-over-posix provides an NDN-Lite adaptation for POSIX platforms, with deployments mainly documented on Linux, and Raspberry Pi devices being a suitable IoT candidate (Named Data Networking for IoT, 2024). It contains working examples of UDP and Unix socket face implementations, and has the most documentation of the collection with it being explicitly recommended as the most stable and recently maintained adaptation. A dedicated website is referenced for getting started and example usage (Named Data Networking for IoT, 2020) with sample results which include compatibility between NFD and NDN-Lite network nodes.

3.3.3 NDN-iot-package-over-nordic-sdk

NDN-iot-package-over-nordic-sdk is an NDN-Lite adaptation for Nordic Semiconductor nRF52840 development boards, supporting BLE and IEEE 802.15.4 faces (Nordic Semiconductor, 2018). This adaptation has also been reported as broken, and, like the RIOT-OS adaptation, it has been inactive for a number of years.

3.3.4 esp8266ndn

While esp8266ndn is not part of the official NDNoT collection repository, it is referenced in the documentation alongside the above adaptations and was developed by Shi (2025), a key contributor to them. The library targets the ESP8266 (Espressif Systems, 2014) but also supports the ESP32 series (Espressif Systems, 2016) by using the Arduino framework (Arduino, 2026a). It provides face implementations for Wi-Fi, UDP, Bluetooth, and Ethernet on its target devices, along with examples, including data retrieval and pinging. Due to using the Arduino framework it has good interoperability between a number of boards with the documentation describing the full range: the ESP8266, ESP32 series (3.x core), nRF52, and RP2040.

3.4 Hardware Platforms

The survey of NDN IoT adaptations in the previous sections established that deployments were carried out on targeted boards, such as Nordic nRF boards via RIOT-OS, or on Raspberry Pi devices with the POSIX adaptation, but no specific adaptation exists for the ESP32 series using its native ESP-IDF framework. The argument for implementing an NDN deployment using NDN-Lite devices from the ESP32 range, specifically with ESP-IDF, is motivated by several factors.

The ESP32 series is a range of low-cost system-on-chip (SoC) MCUs developed by Espressif Systems (Espressif Systems, 2026g), first released in 2016 . Espressif has reported cumulative shipments exceeding one billion units (Espressif Systems, 2023), and according to a third-party report (Niwa, 2022), Espressif held the leading position in the MCU Wi-Fi chipset market with a 27% share in 2022, making it one of the most widely deployed Wi-Fi-enabled platforms in IoT.

The series offers good peripheral and integration support for many processors capable of Wi-Fi and Bluetooth, supported by community-maintained open-source libraries available on Espressif's GitHub collection (Espressif Systems, 2026h). The range of memory accessible to each device spans from hundreds of kilobytes to megabytes of RAM, and often includes at least 1-16 megabytes of flash. This places most of these devices in the higher-tier constrained devices class identified in Section 3.1.

Other platforms, such as the Arduino boards, are widely used in IoT, but their base models lack integrated wireless connectivity, and their memory profiles, ranging from 2 KB SRAM on the classic Uno R3 (Arduino, 2026b) to 32 KB on the newer UNO R4 (Arduino, 2023), fall below the 30 KB RAM and 70 KB ROM footprints reported from previous NDN-Lite deployments (Zhang et al., 2022), suggesting limited headroom for an NDN stack alongside application logic and a Wi-Fi driver. Notably, the UNO R4 Wi-Fi variant achieves its wireless connectivity by incorporating an ESP32-S3-MINI-1-N8 (Espressif Systems, 2022d) as a coprocessor, which itself contains 520KB of RAM, more than ten times that of the host processor, and is inaccessible during programming.

Boards like the Raspberry Pi 4 and Pi 5 provide up to 16 GB of LPDDR4X RAM and quad-core ARM Cortex-A processors running at up to 2.4 GHz (Raspberry Pi Ltd, 2019), (Raspberry Pi Ltd, 2023), which allows them to run NFD directly under Linux, placing them far outside the constrained-device category addressed by this project.

The Raspberry Pi Pico W is a closer comparison point, providing 264 KB of SRAM and 802.11b/g/n Wi-Fi on an RP2040 microcontroller (Raspberry Pi Ltd, 2021). However, the original Pico W lacks hardware cryptographic acceleration, and an external Infineon CYW43439 chip handles its Wi-Fi (Infineon Technologies, 2020) rather than being integrated into the main processor, the new Pico 2 W (RP2350) (Raspberry Pi Ltd, 2024) implements hardware cryptography acceleration and offers backward compatibility between itself and its predecessor, but development will be limited between just these two devices.

Much of the ESP32 series has Wi-Fi directly on chip, hardware acceleration, and they also span a broader product line across multiple architectures and performance tiers. Meaning that porting NDN-Lite to these devices can enable new, in-depth NDN applications that cover many aspects of IoT at a greater scale than the other devices mentioned.

ESP32 Variants for IoT Applications

Table 3.1: ESP32 variant specifications

ESP32	**WROOM-32E**	**S3**	**C6**	**P4**
CPU Architecture	Xtensa LX6	Xtensa LX7	RISC-V	RISC-V
Cores	2	2	1 (+LP)	2 (+LP)
Clock (MHz)	240	240	160	400
SRAM (KB)	520	512	512	768
Price	€3	€4	€4	€11

Table 3.1 details some specifications of four ESP32 variants. Please note that prices were obtained from suppliers as of March 2026 and refer to the module chips, not development kits.

These variants were released spanning different years and selected to cover a broad range of boards with diverse use cases. The first being:

The ESP32-WROOM-32E (WROOM-32E) is a module based on the original ESP32 SoC and is one of the earlier members of the family available since around 2020. This board remains widely deployed across general-purpose IoT applications (Espressif Systems, 2016).

The ESP32-S3 (S3) reached general availability in 2022, targets edge computing applications and can run lightweight embedded ML tasks such as object detection and voice recognition (Espressif Systems, 2022e).

The ESP32-C6 (C6), released in 2022, specialised in newer wireless connectivity protocols, combining Wi-Fi 6 with IEEE 802.15.4 support, the latter enabling Zigbee and Thread protocols suited to low-power mesh deployments (Espressif Systems, 2022b).

The ESP32-P4 (P4), announced in 2023 and available from 2024, targets applications that require higher computational capabilities, such as human-machine interfaces and edge AI. As it lacks an integrated radio, wireless connectivity requires a companion chip, typically an ESP32-C6 (Espressif Systems, 2024).

3.5 MCU Frameworks and Operating Systems

Several development frameworks exist that provide deployment of RTOS's on the ESP32 series. These all offer a range of hardware accessibility, resource overhead and development support for usage on devices.

- FreeRTOS (Amazon Web Services, 2026), is a lightweight open source RTOS currently under AWS ownership that takes a minimal kernel-only approach. This makes it applicable to extremely low-overhead, minimal setup embedded applications. It lacks device driver and I/O support but still supports memory management and multi-threading, and is compatible with many CPU architectures.
- Zephyr is another open-source RTOS and development framework and is maintained by the Linux Foundation. It supports a wide range of architectures, including Xtensa and RISC-V (Linux Foundation, 2026). In contrast to FreeRTOS, it provides a configurable device-driver environment with extensive examples, allowing application code to be deployed across many devices, which is achieved through a hardware abstraction methodology. It has strong support for Bluetooth but relies on hardware-specific drivers for Wi-Fi rather than a built-in library.
- Arduino is widely adopted in IoT due to its accessibility and library ecosystem (Arduino, 2026a), which does not deploy an RTOS but a bare-metal firmware with

code execution consisting of a single setup loop followed by a "superloop". As previously stated, the Arduino framework can be utilised to deploy the esp8266ndn library on a range of ESP32 devices (Shi, 2025), making it the only framework with existing NDN library support for the ESP32 series. Arduino abstracts away lower-level hardware access by wrapping user code in a single FreeRTOS task, discouraging direct use of ESP-IDF and FreeRTOS APIs and reducing developers' control over task scheduling and hardware configuration.

- ESP-IDF is the official development framework for the ESP32 series (Espressif Systems, 2026a). It extends FreeRTOS with an integrated Wi-Fi and Bluetooth stack, as well as additional ESP32-specific functionality, and is therefore compatible with libraries such as lwIP for networking and Mbed TLS (Mbed TLS, 2026) for cryptographic operations. Using ESP-IDF enables simple access to coprocessors on development kits that contain chips such as the ESP32-P4.

3.6 Problem Statement

The previous section outlines how NDN offers advantages over IP-based protocols for IoT deployments and how multiple solutions have existed for deploying NDN applications on IoT. However, these are now a) unmaintained and broken, b) lacking in quantitative measurements or c) restricted by their RTOS or framework.

There are no NDN-Lite adaptation targets for the ESP32 series and its native ESP-IDF framework; existing adaptations support different hardware platforms or rely on the Arduino framework to abstract hardware access provided by ESP-IDF. The only existing ESP32-capable library, esp8266ndn, does not provide quantitative measurements of resource usage or power consumption for any of the target ESP32 hardware.

The specific goals of this project are therefore:

1. Port NDN-Lite to the ESP32 series using ESP-IDF directly, extending NDN-Lite's platform coverage to a widely deployed IoT hardware ecosystem.
2. Evaluate the range of devices that can intercommunicate using this adaptation, including operation with greater NDN applications such as NFD.
3. Get quantitative metrics of NDN communication on ESP32 hardware.
4. Develop a broader application using this port and understand the limits of the adaptation and deploy it in real time.

4 Design

This chapter presents the design choices for implementing NDN on IoT devices by first introducing specific development kits (devkits) that support NDN applications and comparing their features.

The available networking interfaces on these devkits are then examined, and the libraries capable of implementing them are identified. This includes a mixture of FreeRTOS compatible libraries and those specific to ESP-IDF.

Deployment options are then described with the overall design strategy for extending initial applications to additional use cases, such as a smart home network. This includes the issues that IoT devices face in bootstrapping and securing their communications, by comparing available academic solutions with a simpler, less secure alternative.

Finally, the behaviour of the smart home application and the NDN conventions followed are detailed.

4.1 Overview

Creation of an NDN implementation on IoT infrastructure requires a few main tasks, namely:

- Choosing the operating system and specific board for initial development so that compatibility between as many devices as possible was ensured. ESP-IDF was outlined in the previous sections to be capable of such a task.
- Specifying the faces that are to be used for forwarding between devices to maximise cross compatibility.
- Creation of a new implementation or use of pre-existing libraries.
- Specification of libraries for realising such an implementation. These include:
  - Networking
  - Cryptography

  - Random Number Generation
  - Clock and timing
- Post completion of initial implementations, the specification of further application possibilities that can be implemented.

Each ESP32 device introduced in the previous section (Table 3.1) can support lightweight NDN stacks. They vary widely in processing power and memory specifications, consistent with what might be observed in a real-world NDN IoT scenario. These differences can allow a network comprising differing features and use cases, such as nodes with varying Content Store sizes. Their successful use in the project would result in an implementation that enables NDN communication across diverse devices, thereby maximising interoperability. Such a diverse range of resources would allow devices sitting in between this range to theoretically partake in the network as long as their applications are deployable through ESP-IDF.

The ESP32-P4 (P4) was selected as the first chip for possible implementation due to its extensive resource pool which depend on the specific devkit used. The P4 can be configured to expand its total amount of RAM. This is possible through the use of external Pseudo Static RAM (PSRAM) (Espressif Systems, 2026p), which is similar to Dynamic RAM (DRAM) in that it must be refreshed at a set interval by a memory controller, or the data begins to degrade (Jacob et al., 2010). The difference is that PSRAM contains its own memory management and can therefore refresh itself; this makes it functionally equivalent to the P4 as Static RAM (SRAM), hence the "Pseudo Static" name.

As PSRAM exists off-chip, and is accessed via SPI or Octal SPI, it can be slower than onboard SRAM on the P4. Therefore, it may be better suited to use cases that involve holding large data structures and supporting complex applications, such as embedded AI deployments (Espressif Systems, 2026l). An NDN-specific use case may be a large Content Store. The chosen P4 development kit was an ESP32-P4-Module-DEV-KIT-A, which uses the ESP32-P4NRW32 chip variant (Waveshare, 2026).

Table 4.1: Development boards selected for this project

Development Board	**Flash**	**SRAM**	**PSRAM**	**Wi-Fi**	**Bluetooth**
ESP32-DevKitC	4 MB	520 KB	None	802.11b/g/n	Classic + BLE
ESP32-S3-EYE	8 MB	512 KB	8 MB	802.11b/g/n	BLE 5.0
ESP32-C6-DevKitC-1	8 MB	512 KB	None	802.11b/g/n/ax	BLE 5.3
ESP32-P4-DEV-KIT-A	16 MB	768 KB	32 MB	via C6	via c6

The next board selected was an ESP32-C6-DevkitC-1 (Espressif Systems, 2022a). This contains an ESP32-C6-WROOM-1 with 8 MB of flash and 512KB of SRAM. This was chosen as it is the default devkit available from Espressif, whereas the P4 does not have any native devkits.

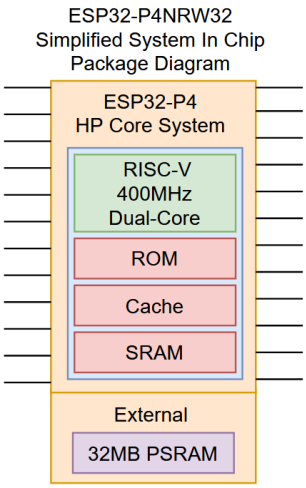


Figure 4.1: ESP32P4NRW32 chip variant with only HP Core System and External PSRAM shown

An ESP32-S3-EYE (Espressif Systems, 2022c) with 16 MB of flash, 512KB of SRAM and 8MB PSRAM was chosen. The applications for this devkit include image capture and processing, and it includes an LCD screen. This was chosen because its use cases are more diverse than those of the other, more IoT-targeted boards.

Finally, an ESP32-DevKitC (Espressif Systems, 2017), which hosts an ESP32-WROOM-32E with 4 MB of flash and 520KB of SRAM. This board is a predecessor to the ESP32-C6-Devkit-C1, and it is the most resource-constrained of the selection. Applications designed with these constraints in mind will show what a proven implementation may be limited to.

4.1.1 ESP-IDF

ESP-IDF (Espressif IoT Development Framework) is the official development framework for the ESP32 series, built on top of FreeRTOS and providing a comprehensive set of libraries for embedded applications (Espressif Systems, 2026a). The framework provides a component-based architecture in which each functional module, such as networking, cryptography, storage, and peripheral drivers, is an independently configurable component that applications can use. ESP-IDF is built on FreeRTOS, with an integrated Wi-Fi and Bluetooth stack, a networking layer via lwIP, cryptographic support through its fork of Mbed TLS, and driver libraries for peripherals such as UART, SPI, I2C, and GPIO. Application configuration is managed through a Kconfig-based menuconfig system that allows compile-time selection of features, memory allocations, and hardware-specific parameters. The CMake-based build system compiles only the components required by a given application, thereby minimising binary size for constrained deployments. Figure 4.2 illustrates the layered architecture of ESP-IDF and highlights the possible components

and the structure of ESP-IDF.

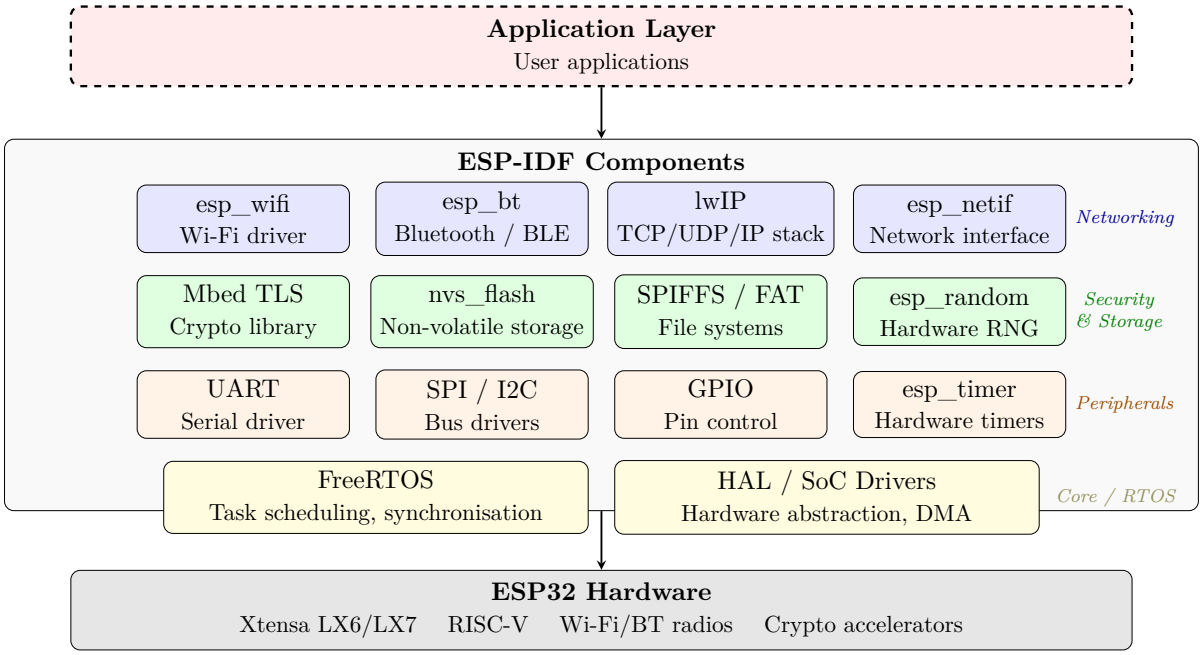


Figure 4.2: Overview of the ESP-IDF layered architecture and its available component categories. Applications are built on top of configurable components, which abstract the underlying ESP32 hardware.

Cryptography Support

The evidence for creating an NDN-Lite implementation specifically for the ESP32 range and the ESP-IDF framework extends beyond the reasoning discussed in previous sections. Many of these boards provide dedicated hardware for cryptographic acceleration when using the Mbed TLS library.

Espressif maintain their own public fork of the Mbed TLS library (Espressif Systems, 2026i). By browsing the datasheets for their various boards, it can be seen which of them support these cryptographic optimisations. For example, the P4 provides the following hardware accelerators when Mbed TLS is configured correctly (Espressif Systems, 2026m):

- An ECDSA accelerator supporting the P-256 curve (National Institute of Standards and Technology (US), 2023), which is the mandatory curve for NDN's Signature-Sha256WithEcdsa signature type (Named Data Networking, 2024b).
- A SHA accelerator that supports SHA-256.
- An AES accelerator with a a configurable anti-side-channel attack feature.
- An ECC accelerator that performs hardware-accelerated point operations (addition, multiplication, verification).

- An HMAC accelerator for HMAC-SHA-256.
- An RSA accelerator that supports modular exponentiation and multiplication.

Hardware acceleration for cryptography varies by devkit, and support for operations varies significantly. For example, the WROOM-32E and the S3 lack support for ECDSA (Espressif Systems, 2026f) (Espressif Systems, 2022c).

Some of these accelerators can be utilised for NDN packet signing and encryption operations and may provide an advantage over boards that lack such hardware. RTOSs that are not specific to ESP32s will also lack immediate driver support for accessing them. The same holds for cryptographic library selection; however, it is noted that WolfSSL has officially added support for these peripherals on ESP32 devices (wolfSSL, 2024), but this support must be explicitly selected and enabled.

Table 4.2: Cryptographic hardware acceleration by ESP32 variant

Accelerator	ESP32-P4	ESP32-C6	ESP32-S3	WROOM-32E
SHA	✓	✓	✓	✓
AES	✓	✓	✓	✓
RSA	✓	✓	✓	✓
HMAC	✓	✓	✓	–
ECC	✓	✓	–	–
ECDSA	✓	✓	–	–

ESP-IDF Project Configuration

ESP-IDF uses a CMake-based build system. An application will have a components directory that can be used to create customised project dependencies; these can be reused for other projects and enable modularity. These will be included in the main application program if specified in the `CMakeLists.txt` that exists next to the root code main file. Components that are native to ESP-IDF can also be specified in this file and will be pulled from the ESP-IDF core. Component versions and dependencies can be specified in an `idfcomponents.yml` file allowing project dependencies to be replicated across many projects.

ESP-IDF provides an idf.py script that gives extensive tooling for board development, configuration and flashing. Using ESP-IDF's `idf.py menuconfig` command allows application and chip-specific parameters to be set and saved. This includes the cryptographic settings that apply to each chip, as well as additional parameters, including the Wi-Fi SSID, Password, PSRAM allocation, main task stack sizes, partition table allocations, and more, which will be discussed in the Implementation section.

A use case in this project for this is to create a custom configuration tailored to NDN-Lite or an application. This could enable dynamic changes, such as Content Store sizes, application-specific Wi-Fi parameters, MTU sizes, the selection of the cryptographic back-end, and more, all of which may be device-specific.

4.2 NDN-Lite Port to ESP-IDF

4.2.1 Face selection

The NDN-Lite face selected for adaptation was UDP over Wi-Fi. This was to maximise IoT interoperability among the selected devkits and to ensure compatibility with a device such as a PC running NFD. UDP supports the following forwarding modes:

- Unicast
  - A unicast face can maintain a direct mapping between itself and another face, allowing point-to-point communication. This can be utilised in scenarios where the source and destination are both known; however, it will require a protocol or user input to register such a connection.
- Multicast
  - A multicast face can allow Interest flooding across an NDN network, allowing Interest to be delivered to multiple faces. Multicast was chosen for implementation for discovery and experimental interoperability.

Bluetooth and Bluetooth Low Energy (BLE) were not selected for this implementation, although all the selected devkits support BLE natively; it offers lower throughput and shorter range than Wi-Fi (Popli et al., 2019). Another key reason was the need for compatibility with NFD, for which Wi-Fi UDP is best suited in early adoption, as the devices used to develop the implementation had network access via Wi-Fi or Ethernet but lacked Bluetooth. However, for an adaptation that may require optimisation for power use, BLE would be the preferred choice.

4.2.2 Library selection

esp_wifi.h

ESP-IDF's Wi-Fi library (Espressif Systems, 2026r) can be used to configure and connect to local Wi-Fi networks. The `esp_netif` (Espressif Systems, 2026d) library interfaces the Wi-Fi driver with the lwIP stack, handling address assignment and other protocol requirements. Pairing this with **esp_nvs.h** (Espressif Systems, 2026n) allows Wi-Fi credentials to be persisted to flash, so that, in the event of a disconnection, reconnection

can occur without manual intervention. This is important for IoT deployments with unattended or battery-powered devices that must wake from deep sleep and reconnect to Wi-Fi using the same credentials, without wasting resources on repeated connection attempts.

lwIP

To implement the UDP face, lwIP (Goldschmidt, 2026) was selected as the network stack. lwIP is bundled with ESP-IDF and mirrors the standard BSD socket API used in POSIX systems, allowing the use of the same function calls. lwIP is already a dependency of ESP-IDF's Wi-Fi driver, therefore it is a necessary contributor.

esp_timer.h

This library could be used to implement Espressif's specific software timers (Espressif Systems, 2026e). NDN-Lite requires this for features such as the PIT for detecting expired Interests, Content Store data freshness expiry, the Forwarder for tracking the time of sending Interests, Interest encoding, which creates the nonce based on a seed from the timer and much more.

esp_random.h

A Random Number Generator (RNG) is required for cryptographic key generation, nonce generation and encryption IV's. Espressif also provide a built-in library for this (Espressif Systems, 2026o). The source of RNG is board-specific in the ESP32 series; for example, the P4 uses numbers generated by thermal noise in its SAR Analogue-to-Digital Converter (ADC) and clock-mismatch effects derived from circuit instability.

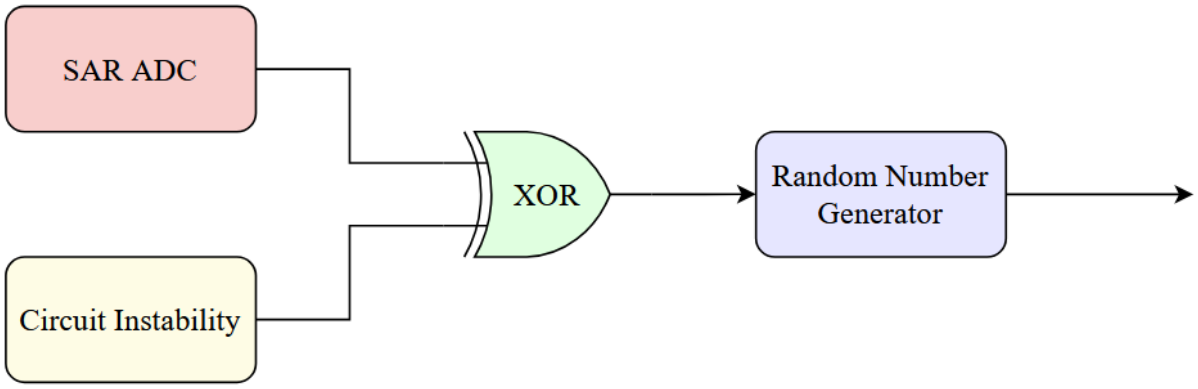


Figure 4.3: Random Number Generator source on the ESP32-P4

vTaskDelay

As ESP-IDF is built on FreeRTOS, multiple options, such as `vTaskDelay()` (Espressif Systems, 2026j), are available for task scheduling and yielding; other NDN-Lite adaptations demonstrate task yielding to allow periodic operations to execute between forwarding. These other tasks, such as Wi-Fi/essential networking, may need to be performed between the main task loop.

esp_driver_uart

The need for user input is application-dependent; for very simple applications, it was decided that users may need to provide run-time input to manually set the IP address, port configuration, and prefix for creating faces, since there will be no routing protocol. The Espressif library `esp_driver_uart` (Espressif Systems, 2026q) allows the configuration of a UART input, such as the use of parity bits, altering baud rate and other UART necessities. Only boards with UART capabilities can be used with this application, and the selected devkits all have this functionality.

Tinycrypt and micro-ecc

NDN-Lite comes pre-packaged with TinyCrypt (Intel Corporation, 2019) and micro-ecc (MacKay, 2024) as its cryptographic backend. Tinycrypt provides a lightweight implementation of SHA-256, AES, and HMAC. micro-ecc handles elliptic curve operations for ECDSA signing and verification. These libraries were initially used for simplicity and ease of implementation, but were replaced with Mbed TLS for the reasons mentioned in Section 4.1.1. To test whether Mbed TLS outperforms them, a compilation option can be implemented that selects either the new Mbed TLS or the default TinyCrypt and micro-ecc backend. Upon review, it was found that TinyCrypt is no longer actively maintained and has been archived by Intel, introducing security risks which could be exploitable for applications relying on this library, and although micro-ecc has not been archived and is still open to contributions, it has had no activity in two years. This pushed the project to rely on Mbed TLS as its main cryptographic backend, but with the possibility to alternate to understand any differences.

Table 4.3: NDN-Lite adaptation layer mappings to ESP-IDF

Abstraction	**ESP-IDF Component**	**Rationale**
Network face	ESP Wi-Fi + lwIP UDP sockets	Integrated stack, standard socket API
Clock/timer	esp_timer	Hardware timer, microsecond resolution
RNG	esp_random	Hardware entropy source call
Sleep/scheduling	FreeRTOS	Native to ESP-IDF's RTOS layer
Console I/O	esp_driver_uart	Serial access for input
Cryptography	Mbed TLS	Hardware Acceleration and ESP support

4.3 Application Development

The initial implementation of the NDN-Lite library will create and test some basic applications that align with the examples in NDN-iot-package-over-posix. This can enable comparisons of basic packet transfer between the two adaptations and facilitate faster adoption.

After NDN packet transfer has been successfully implemented using the libraries selected in the previous section, a more comprehensive example of a smart home network can be created to demonstrate real implementation capabilities. This would involve multiple nodes operating on each selected devkit to demonstrate the port's interoperability.

There were a few key design considerations that were decided upon:

- In order to implement ease of interoperability of many differing devices, seamless operation was incentivised, this means creating applications that are not device specific.
- Each device in the network needs to follow NDN guidelines, such as packets being compatible with NDN-TLV v0.3, nodes can only send Data when an Interest is received, and Data must have a signature.
- As this is a smart home network, a manager or dashboard can be implemented to monitor the network's current status. This can display various elements such as data for requesting services or general device status.

The NDN-iot-package-over-posix repository describes an implementation of these abilities (Named Data Networking for IoT, 2021b). Achievement of this process is through a node running on a general-purpose PC with access to NFD, paired with a Python-based controller. This is set up with a UI that provides access to a publish-subscribe API for various services, such as operating a light switch or adjusting its brightness.

The process they outline for introducing new nodes into the NDN network addresses a problem that many IoT devices across different networking protocols face: the issue of security bootstrapping. A device seeking access to a home IoT network requires two things:

- A trust anchor certificate enabling it to verify the authenticity of other nodes in the network.
- A personal certificate signed by a trust anchor selected by the home network, for others to use to recognise the device.

In typical NDN deployments on general-purpose hardware, trust credentials will have been installed by the user, administrator or a trusted application. A server running

NFD will have trust anchors and signed certificates installed directly, and application-level trust schemas can determine whether a signed Data should be accepted. This is similar to how browsers currently use trusted domains. In an NDN deployment there is no need for a CA for verification when connecting to domains because each packet includes a signature and a key locator that can be traced back to a root trust anchor, as outlined in Section 2.1.4. In both cases, the devices have the resources and storage to perform such operations, typically in real time by a human operator. This is not the case for NDN IoT deployments, where there is no shell access, no interactive interface, and limited resources. Additionally, because IoT devices are often deployed at varying scales, a selected solution that provides automatic credential provisioning must be both scalable and lightweight.

4.3.1 Secure Sign-on Protocol

Li et al. (2019) introduce such a solution using a Secure Sign-On Protocol (SSP). This protocol automates credential provisioning through two rounds of Interest and Data exchange between an NDN IoT device and a controller. This is modelled as a system of four actors: the home network, the device, a controller with sufficient resources for credential management, and a human who initiates the process.

Before signing on, the human must participate in a pre-shared secret phase between the device and the controller, which is established by flashing a private key and a symmetric key onto the device during manufacturing. The public key corresponding to the device's private key, along with the symmetric key, is then transferred to the controller via a QR code, which the operator must manually scan and upload.

The device will initially broadcast a sign-on request as a signed Interest, signed with its pre-installed private key, and containing a challenge. The controller can then verify this Interest using the public key uploaded via the QR code.

If successful, the controller sends back a trust anchor certificate signed with the symmetric key and its own challenge. The device can then verify the response and send another signed Interest that is a certificate request, initiating a new key-sharing interaction.

The controller will respond to this request by generating new keys and a signed certificate, and send the private key and certificate back to the device. This final interaction is signed with the pre-shared symmetric key, and the private key within is encrypted with a temporary key derived from a Diffie-Hellman exchange between the device and the controller using the earlier challenges.

If the pre-shared secrets (QR code) are later leaked, the content of the packets in the second round remains secure under this Diffie-Hellman encryption, as the exchange cannot

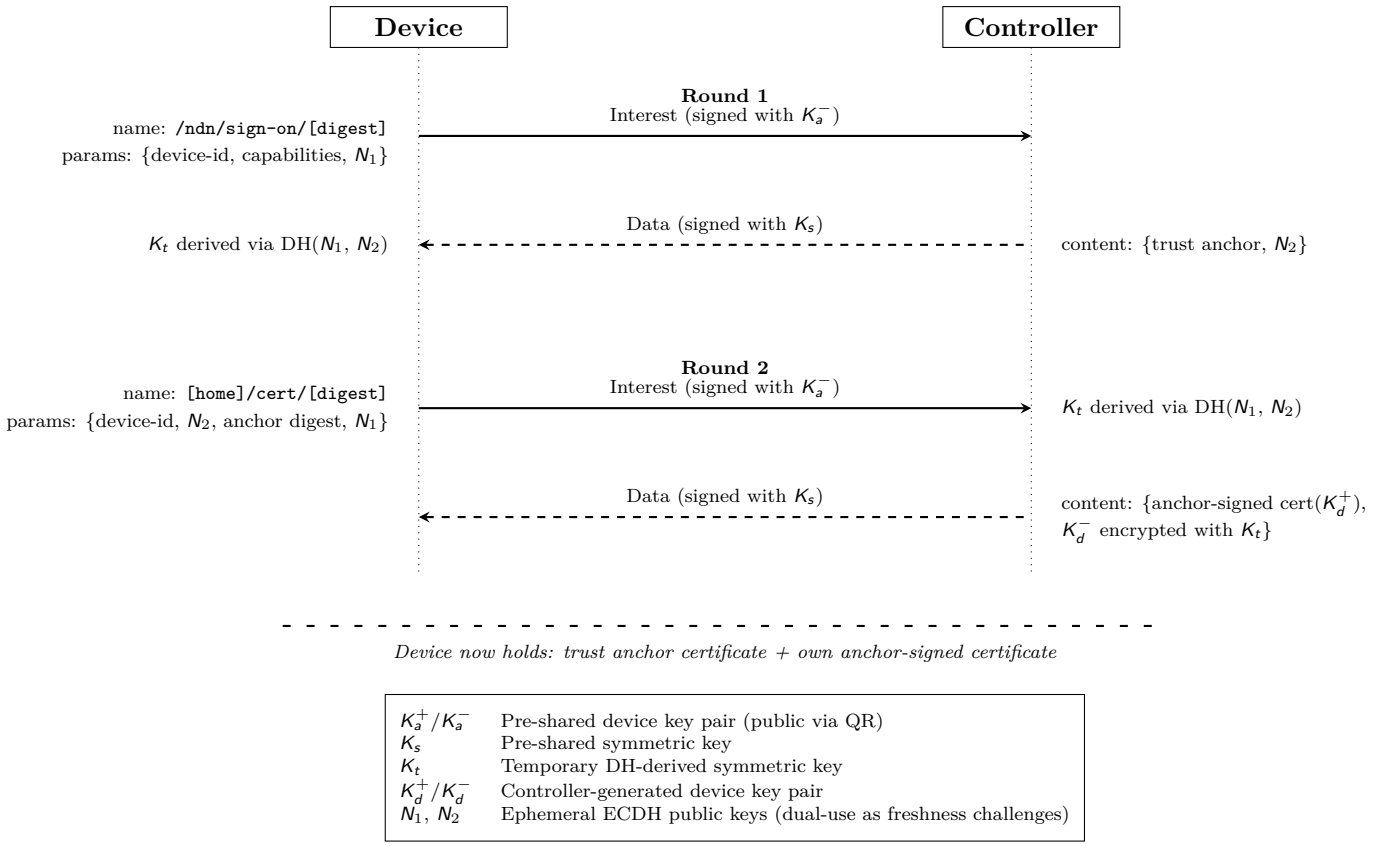


Figure 4.4: SSP sign-on protocol between a device and controller, adapted from Li et al. (2019).

be reconstructed. After this process, the device has a trust anchor certificate and its own anchor-signed certificate, thereby granting it secure access to the NDN IoT network. This is the method that NDN-iot-package-over-posix directly employs to ensure that secure bootstrapping is enabled for each device for which it manually generates key pairs. However, the application developed in this project does not implement this method.

Li et al. (2019) state that devices capable of generating their own key pair do not require a controller to transmit an encrypted private key in the last step, and that devices with permanent storage can retain credentials after reboots, thereby eliminating subsequent sign-ons for such capable devices.

The devices used in this project are all capable of NVS flash techniques and can self-generate key pairs. This presents the opportunity to reduce the SSP exchange by removing encryption of the private key in the last round. However, another bootstrapping technique was considered.

4.3.2 Simple Manual Verification

Another bootstrapping method could be deployed to reduce physical interaction with boards. An accept button on a human interface can allow nodes to access a network

instead of the manual pre-sharing of secrets step. Devices that haven't been on-boarded will send an advertising Interest that contains the following:

- Name
- Public key, stored within the ApplicationParameters section
- Device ID, derived from the chip model and MAC address
- As the scope of this project only implements UDP over Wi-Fi faces, they will also include an IP address

The advertising Interest can be signed and sent over a multicast face that the manager, running on a PC with NFD can use to observe such packets. If a device is not recognised, the user will be notified. The human is presented with the option to accept or reject the device. Upon acceptance, the manager stores the device's public key and registers the appropriate NFD route. The manager will then send back a signed Data with its public key, so the ESP32 node may be able to verify Interests from that point forward if implementation provides a method for doing so. Interest sent by the manager can return signed Data from devices that were accepted on the network and can be verified by the manager using the stored public key. This can be achieved using the Mbed TLS library and the hardware accelerators on the ESP32 devices as all devices in the network will need to generate Data with verifiable ECDSA signatures in accordance with the NDN-TLV v0.3 specifications.

This implementation will come at a cost; it aims to provide simpler onboarding for demonstration and ease of use purposes, but it can be abused. One such scenario is an attacker gaining network access and recreating one of these nodes. The attacker can broadcast their own Interest, which could be accidentally accepted by a user if they do not realise it is from an outside node, thereby approving their device and accepting their Data packets as legitimate. Another critical vulnerability arises when an attacker acts as a manager and first sends its public key to a node before the original manager does. This would allow an outsider to send Interests to the nodes and receive data, gaining access to node functionality similar to the attack described in Section 2.1.4. Attacks like these are why IoT devices struggle to bootstrap securely and must adhere to strict security policies and protocols, such as the one outlined by SSP.

4.3.3 Application Details

Each ESP32 node in the network can operate as a sensor data producer. Devices will advertise their name for that data on a prefix following a hierarchical naming scheme, such as `/home/temperature/living-room/<device>` with an Interest. In practice, the sensor readings that will be returned will be simulated for simplicity.

Devices will advertise their prefixes at a set rate. This will be used by the manager to determine whether a device is still active and can set routes accordingly. After a set period without advertisements, the manager can remove the route. This is not necessary in an NDN, where data location is not necessary, but is beneficial in a smart-home IoT network, where device availability may need to be checked.

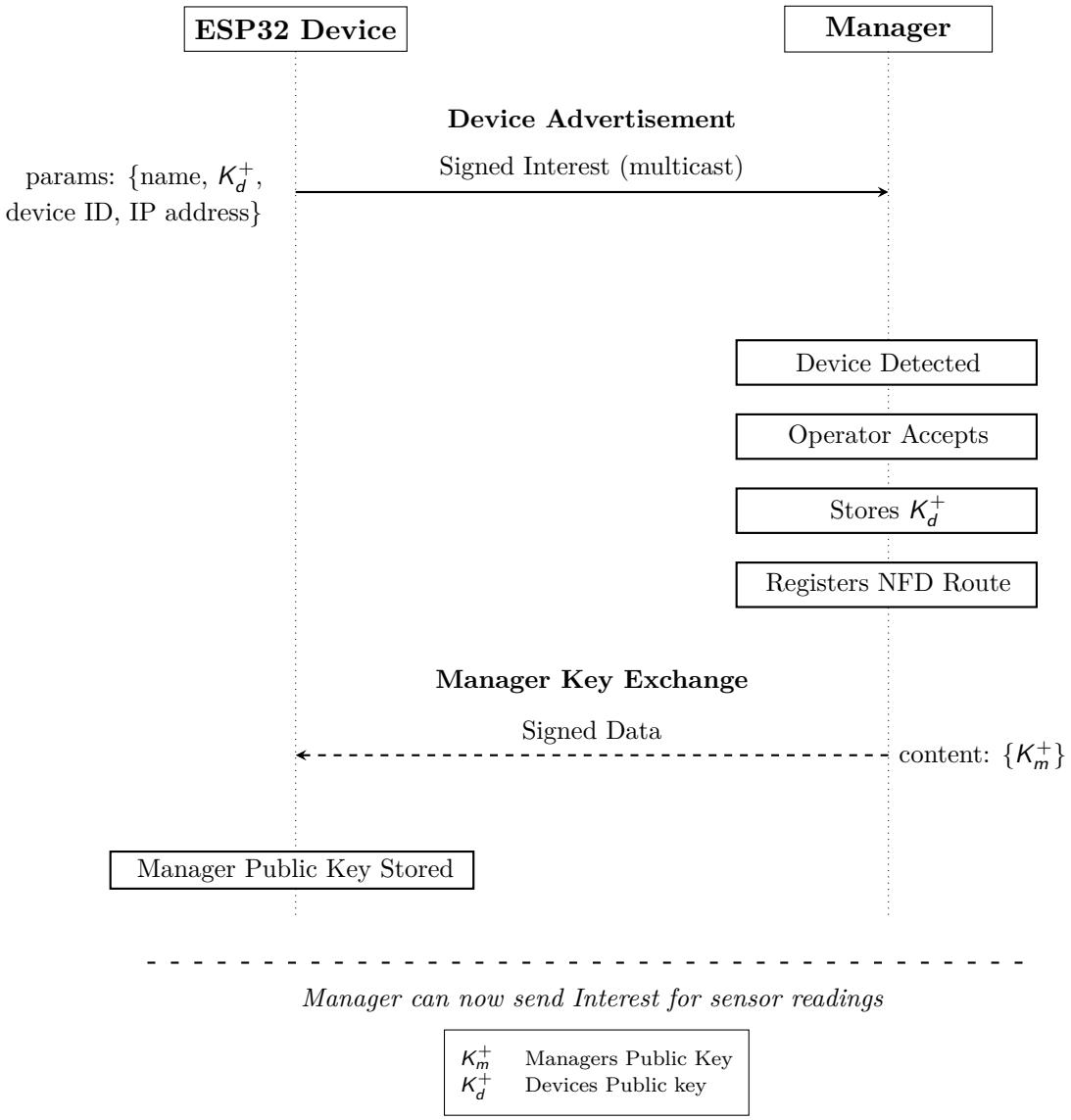


Figure 4.5: Simple Manual Verification bootstrapping protocol. A device broadcasts an advertisement Interest; the operator approves or rejects via the manager dashboard, after which public keys are exchanged.

The manager will run as a Python script on a standard PC with access to NFD, utilising a REST API to feed an HTML dashboard that provides a user interface. This will have live network status updates, the ability to request data, the request of specific prefixes and approval or rejection of nodes.

4.3.4 Naming Scheme

Following NDN conventions, the application uses a hierarchical naming scheme (Named Data Networking, 2019). The first component identifies the network (`home`), The second identifies the data type (`temperature`, `humidity`), and the third identifies the device location (`living-room`, `bed-room`).

This structure allows consumers to specify which room a reading must come from, or if a general reading of the house's temperature will suffice. This is enabled through the CanBePrefix option for Interest that was outlined in Section 2.1.1. For example, a consumer sending Interest to `/home/temperature` with the CanBePrefix flag will activate Data responses from all nodes that contain that prefix in their Data response functionality, whether this is the Content Store or real time Data production. The response will come from the first face that responds, and the other Interests will be dropped. Alternatively, if an Interest is sent for `/home/temperature/living-room`, the Data will be retrieved only from a node that has that exact named Data.

4.4 Summary

This concludes the Design chapter, which details the major design decisions for the NDN-Lite ESP-IDF port and the steps for creating a functional implementation. Four development kits were selected; the Waveshare ESP32-P4-Module-DEV-KIT-A, the ESP32-C6-DevKitC-1, ESP32-S3-EYE and the ESP32-DevkitC, each varying in their IoT use cases and containing different architectures.

The NDN-Lite adaptation layer was mapped to the various necessary components. This included lwIP and esp_wifi for UDP networking over Wi-Fi, esp_timer for timing, esp_random for retrieving a hardware entropy source, FreeRTOS vTaskDelay for scheduling, esp_uart_driver for sending keyboard input to the devices, and Mbed TLS for cryptographic implementation. Both a UDP unicast and multicast face were selected to ensure compatibility with NFD on PC and communication between ESP32s.

An NDN smart home network was designed to demonstrate interoperability across multiple devices, and the key decisions underlying this design were explored. Devices advertise prefixes via multicast and respond to Interests with Data that contain simulated readings, using a hierarchical naming scheme. A Python-based manager running on a PC alongside NFD will register and deregister routes based on the nodes' online status. A dashboard provides a human interface for viewing this information in real time, and the stack comprises a front-end HTML and CSS layout that relies on a back-end Python script.

Security bootstrapping comprises nodes broadcasting their information via Interests, the user interface awaiting human input to accept the device, and credentials being returned upon acceptance, thereby allowing the device to participate in the network. Although this approach prevents basic external nodes from participating, it still introduces security vulnerabilities relative to more thorough onboarding methods, such as the SSP method, and, in a new implementations and deployments, is not recommended as the go-to solution unless security is to be disregarded.

5 Implementation

This chapter details the key steps required to implement the design solutions. Beginning with the first major step of integrating NDN-Lite into the ESP-IDF framework by creating the project structure in accordance with the ESP-IDF format, while also conforming to the other NDN-Lite project structures. The outline of the platform adaptation steps taken to meet the requirements of the selected libraries and the tests involved, followed by an explanation of the Smart Home Sensor Application.

5.1 Build System Integration

The ESP-IDF environment includes multiple releases, each tagged with compatibility for its respective chip series. The release chosen for this project was v5.5, as it was the latest stable release at the time the project began (Espressif Systems, 2026b), and it was compatible with the four selected chips on the devkits.

Initially, the ESP-IDF project structure followed the default format as shown in Figure 5.1.

```
ndn-iot-package-over-esp-idf/
├── build/
├── main/
│   ├── CMakeLists.txt
│   ├── main.c
│   └── idf_component.yml
├── CMakeLists.txt
├── sdkconfig
├── dependencies.lock
├── components/
│   └── component1/
│       ├── CMakeLists.txt
│       ├── Kconfig
│       └── src1.c
└── managed_components/
```

Figure 5.1: Default ESP-IDF project structure.

NDN-Lite was integrated as a custom ESP-IDF component by placing it into the `components/ndn-lite/` directory. For this to be used in the project, an accompanying `CMakeLists.txt` was created which allowed the project to declare all the custom source files, include paths and parameters set for the library.

To keep NDN-Lite separate and enable its use by multiple applications within the project, the project structure was modified. This was a change in the location of the components directory. By creating a general components directory, multiple applications could easily reference it and pull the necessary NDN-Lite files individually.

```
ndn-iot-package-over-esp-idf/
├── components/
│   ├── ndn-lite/
│   └── adaptation/
├── application_1/
├── application_2/
└── application_3/
```

Figure 5.2: Altered ESP-IDF project structure.

This alteration and introduction of the adaptation component allowed the project to create or introduce necessary changes to the NDN-Lite library that make it compatible with ESP32 devices without changing the NDN-Lite library contents. This separate adaptation layer is consistent with the NDN-Lite documentation. Each application references this components directory and builds the necessary files that are specified by a `CMakeLists.txt` inside the adaptation component. This component structure can target all four selected ESP32 variants by using the `idf.py set-target` parameter and no source code modifications when switching deployment between targets.

Within the adaptation library a `CmakeInputs` directory was created with multiple cmake files to allow specification of what should be compiled by the master `CMakeLists.txt` file. This pointed to source files from NDN-Lite and the created adaptations that will be discussed in the next section.

5.2 Platform Adaptation Implementation

Below are the specific adaptations for the ESP32 devices that were implemented using the selected libraries from Section 4.2.1.

5.2.1 Clock and Timer

NDN-Lite requires three clock functions to be implemented by each platform adaptation:

- `ndn_time_now_ms` for getting a clock timestamp in milliseconds.
- `ndn_time_now_us` for timestamps in microseconds.
- `ndn_time_sleep_ms` for introducing blocking delays.

Using the `esp_timer.h` library, `esp_timer_get_time()` returns time in microseconds since boot. Implementing this involved wrapping `ndn_time_now_us` and casting its default type, `int64_t`, to the NDN-Lite standard `ndn_time_us_t`. For `ndn_time_now_ms`, the same was done except it was divided by 1000 to get a reading in milliseconds and cast to `ndn_time_ms_t`. The FreeRTOS `vTaskDelay()` function was used to implement the sleep function `ndn_time_sleep_ms()`. `vTaskDelay` requires that the delay be in ticks. Therefore, the approach for this conversion involved taking in the requested delay in milliseconds and converting it to ticks with `pdMS_TO_TICKS()` and passing it to `vTaskDelay`.

5.2.2 Wi-Fi and Network Initialisation

For Wi-Fi connection and face implementation, the network connection functions and subsystems needed to be added. This was done by replicating the Wi-Fi example in ESP-IDF (Espressif Systems, 2026c) with the addition of lwIP, `esp_netif` and a few helper functions specific to this project. These included `wifi_get_ip_raw()` and `wifi_get_ip_string()` that could be used in the automatic retrieval of the devices IP and for debugging. The station program begins by checking the NVS flash partition to see if Wi-Fi credentials have previously been stored, if not it will initialise the Wi-Fi station process that starts a series of networking setup process using the `esp_netif` library. This will check the Wi-Fi configuration parameters that can be set in the `idf.py menuconfig` setup such as SSID, password and more and use them to initiate a Wi-Fi connection.

5.2.3 Random Number Generation

Random number generation could be implemented through use of the `esp_random.h` library which includes the `esp_fill_random()` function. This relies on various entropy sources dependent on the devkit selected. The WROOM-32E, C6, and S3 use the RF subsystem as a secondary source of entropy, and combine this with their default sources when the Wi-Fi subsystem is activated. This can be done through various API calls dependant on the networking interface to be used, in this case it is Wi-Fi therefore `esp_wifi_start()` from the esp_wifi.h library will activate this source. The `esp_fill_random()` is used by taking in the pointer of the buffer to be filled and its length, which is then wrapped with `ndn_lite_esp32_rng()`.

5.2.4 UDP Face

The UDP face implementation allows boards to use NDN-Lite for forwarding over Wi-Fi with lwIP's BSD socket API, and because this API mirrors the POSIX socket interface, this face implementation could be adapted from the existing ndn-iot-package-over-posix UDP face implementation with minimal modifications. The core functions for socket operations, such as creating sockets, binding them, non-blocking configuration, joining multicast groups, sending datagrams and receiving, all have identical function calls to the BSD socket API, which the POSIX adaptation uses, allowing for the creation of unicast and multicast UDP faces.

The main modifications were to the header structure and build integration, achieved by repackaging the face as its own header (`esp32-face.h`) and linking it to the lwIP and Wi-Fi components. A debugging output was created that could be enabled or disabled at compile time to provide verbose debug information during development.

5.2.5 Cryptographic Backend

Initially TinyCrypt and micro-ecc were used as the cryptographic backend, with seeding from the RNG implementation as previously described. At startup `ndn_lite_startup()` is called, which subsequently calls `ndn_security_init()`. This would begin loading the SHA, ECC, HMAC and AES implementations from TinyCrypt and micro-ecc. To implement Mbed TLS cryptography, function calls had to be created so that they could be routed through backend loaders and wrapped by the NDN-Lite API. These were:

- ndn-lite-default-sha-impl.c
- ndn-lite-default-aes-impl.c
- ndn-lite-default-ecc-impl.c
- ndn-lite-default-rng-impl.c
- ndn-lite-default-hmac-impl.c

Each of these were replicated using Mbed TLS with a new security configuration file in the adaptation folder. Compiler definitions were created that allowed switching between the old and new cryptographic libraries for testing and later evaluation.

5.2.6 Compilation

All of these adaptations led to an NDN-Lite compatibility layer for ESP-IDF, with the structure of the layer being compiled shown in Figure 5.3.

```
#ifdef NDN_LITE_SEC_BACKEND_ESP32_MBEDTLS
    ndn_lite_esp32_mbed_tls_sha_load_backend();
    ndn_lite_esp32_mbed_tls_aes_load_backend();
    ndn_lite_esp32_mbed_tls_ecc_load_backend();
    ndn_lite_esp32_mbed_tls_hmac_load_backend();
    ndn_lite_esp32_rng_load_backend();
#else
    ndn_lite_default_sha_load_backend();
    ndn_lite_default_aes_load_backend();
    ndn_lite_default_ecc_load_backend();
    ndn_lite_default_hmac_load_backend();
    ndn_lite_esp32_rng_load_backend();
#endif
```

Figure 5.3: Compile-time selection between Mbed TLS and default cryptographic backends. The ESP32 backends are loaded when `NDN_LITE_SEC_BACKEND_ESP32_MBEDTLS` is defined. Both use the same ESP32 RNG implementation

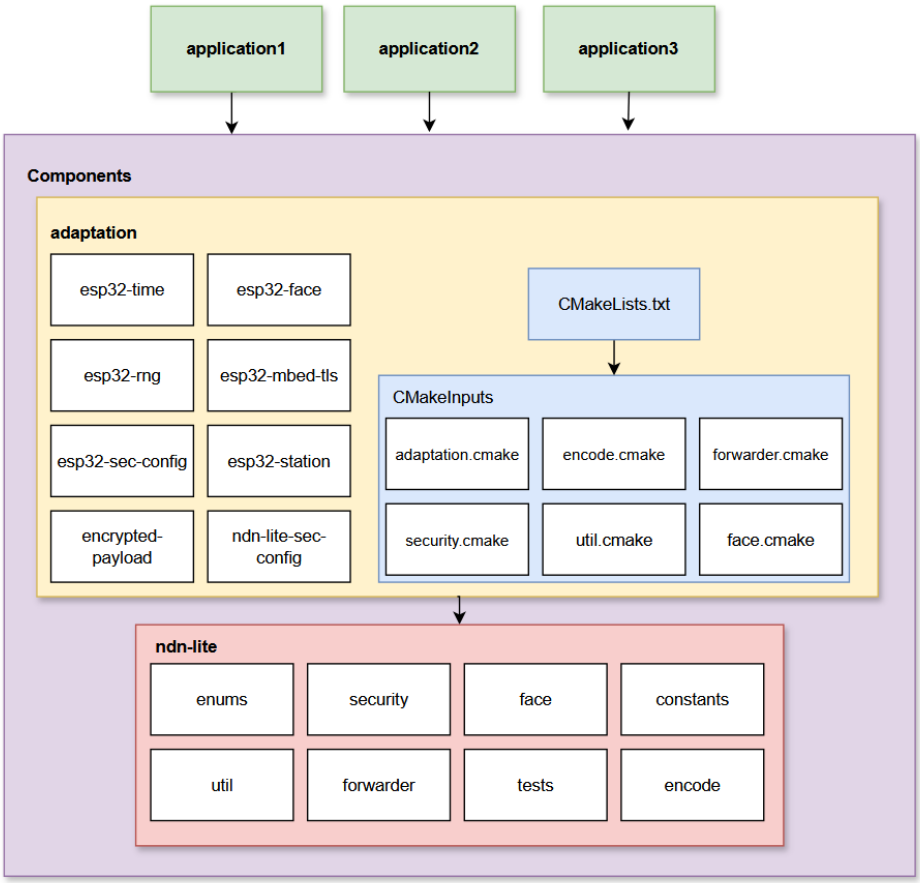


Figure 5.4: Architecture of the NDN-Lite ESP-IDF adaptation layer showing the relationship between applications, adaptation components, CMake build configuration, and the underlying NDN-Lite library

As the adaptation acts as a layer between the boards and NDN-Lite, the `CMakeLists.txt` for the adaptation registers the required components using `idf_component_register`. This allows the inclusion of all the necessary source files that are set in the CMakeInputs directory, the necessary directories and all the required components. When developing an NDN-Lite application on ESP32s, only the adaptation layer needs to be referenced when using this implementation because of these structural design choices.

Adaptation Component

```
idf_component_register(
    SRCS
        ${ALL_SOURCES}
    INCLUDE_DIRS
        ${ALL_INCLUDE_DIRS}
    REQUIRES
        esp_timer freertos lwip
        esp_netif esp_wifi
        nvs_flash mbedtls
)
```

Application

```
idf_component_register(
    SRCS
        "main.c"
    INCLUDE_DIRS
        "."
    REQUIRES
        adaptation
)
```

Figure 5.5: Component registration for the adaptation layer (left) and a typical application (right). The application only requires the adaptation component.

5.3 Basic Applications

The ndn-iot-package-over-posix repo introduces a few basic examples that enable Interest and Data transfer. The following were selected as a base implementation using the adaptation layer and named similarly.

Table 5.1: UDP example applications from ndn-iot-package-over-posix

Application	Description
`udp-basic-producer`	Publishes Data packets over unicast UDP for Interests
`udp-basic-consumer`	Sends Interests and receives Data packets over unicast UDP
`udp-group-producer`	Publishes NDN Data packets to a multicast group over UDP
`udp-group-consumer`	Joins a UDP multicast group and receives Data packets

Due to the portability of NDN-Lite, the selection of lwIP and using C as the primary development language, the basic applications logic could be reused with minimal changes to their code with primary modifications being the header includes, build integration, and the replacement of POSIX-specific initialisations with the ESP-IDF alternatives described in Section 5.2.

Preliminary Single Device Testing

An initial application was developed that could express Interest and produce Data on the same device which was an amalgamation of udp-basic-producer and udp-basic-consumer, for preliminary testing before setting up inter-board connections. This was named udp-basic-consumer-producer. Two unicast faces were created on the same device using ports 6363 and 6364, with the consumer's outbound port mapped to the producer's inbound port and the producer's outbound port mapped to the consumer's inbound port. The device's IP address was retrieved after the Wi-Fi stack was initialised using the `wifi_get_ip_raw()` function. Although this was used, it was not necessary; the loopback IP (127.0.0.1) could also be used. Two callback functions were replicated from ndn-iot-package-over-posix's implementation (Named Data Networking for IoT, 2024): **on_data** and **on_interest**. After initialisation of Wi-Fi, the NDN-Lite stack and faces, a while loop is entered. This continually calls the `ndn_forwarder_process()` function, which reads any data that arrives on the UDP face, triggering either **on_interest** for Interest packets or **on_data** for Data packets.

```
I (5695) RPC_WRAP: ESP Event: Station mode: Connected
I (5695) esp_wifi_remote: esp_wifi_internal_reg_rxcb: sta: 0x4005448c
--- 0x4005448c: wifi_sta_receive at /home/matt/esp/esp-idf/components/esp_wifi/src/wifi_netif.c:38
I (6735) esp_netif_handlers: sta ip: 192.168.0.94, mask: 255.255.255.0, gw: 192.168.0.1
I (6735) wifi station: got ip:192.168.0.94
I (6735) wifi station: connected to ap SSID:Three_9951C5 password:3nXywxw53522y35
Security backend initialised
Forwarder initialised
On interest
DEBUG: fwd_data_pipeline, L466: [FORWARDER] (fwd_data_pipeline) No cs entry found, inserting new one
On data
It says: I'm a Data packet.
DEBUG: ndn_forwarder_express_interest, L269: [FORWARDER] (ndn_forwarder_express_interest) Prefix match in content store found
On data
It says: I'm a Data packet.
DEBUG: ndn_forwarder_express_interest, L269: [FORWARDER] (ndn_forwarder_express_interest) Prefix match in content store found
On data
It says: I'm a Data packet.
DEBUG: ndn_forwarder_express_interest, L269: [FORWARDER] (ndn_forwarder_express_interest) Prefix match in content store found
```

Figure 5.6: Serial output of the udp-basic-consumer-producer loopback test showing Wi-Fi connection, Interest forwarding, and Content Store cache hits. *Note: The AP SSID and password are no longer the same.*

Inter Device Testing - Unicast

A connection between the P4 devkit and the WROOM-32E devkit was established, with the P4 acting as the consumer and the WROOM-32E as the producer. The same faces were created as before, but on separate devices. As these were unicast faces with different IP addresses, the faces on each device needed to be created manually. This could be hardcoded by assigning static IP addresses using `esp_netif_set_ip_info()`; however, manual input was retained to allow prefixes to be entered at runtime. This shows the advantage of routing protocols such as NLSR which can automatically find other nodes in the network and create routes for the FIB. In this case, the FIB would be updated by passing the prefix and face with `ndn_forwarder_add_route_by_name(&face->intf, &name_prefix);`. The `esp_uart_driver` was used to be able to manually input the

ports, IP and matching prefix in each device to create the FIB entry at runtime. Both the devkits had the possibility to receive UART input through the same cable that powers each board. The UART connection was configured as shown in Figure 5.7.

```
uart_config_t uart_config = {
    .baud_rate  = 115200,
    .data_bits  = UART_DATA_8_BITS,
    .parity     = UART_PARITY_DISABLE,
    .stop_bits  = UART_STOP_BITS_1,
    .flow_ctrl  = UART_HW_FLOWCTRL_DISABLE,
    .source_clk = UART_SCLK_DEFAULT,
};
```

Figure 5.7: UART configuration for the serial console interface.

When paired with a parser, this configuration could enable boards to receive keyboard input and create a face based on the received data over UART. After connecting both devices to the same network and creating FIB entries on each of them for the other device, Interest-Data exchanges were observed with subsequent Content Store hits as before.

```
I (12613) esp_netif_handlers: sta ip: 192.168.0.232, mask: 255.255.255.0, gw: 192.168.0.1
I (12613) w-V Enter: <local-port> <remote-ip> <remote-port> <name-prefix>
Raw input: '6364 192.168.0.94 6363 /test'
Local port: 6364
Remote IP: 192.168.0.94
Remote port: 6363
Name prefix: /test
Security backend initialised
Forwarder initialised
DEBUG: fwd_on_incoming_interest, L384: [FORWARDER] (fwd_on_incoming_interest) No prefix match in content store found
On interest
DEBUG: fwd_data_pipeline, L466: [FORWARDER] (fwd_data_pipeline) No cs entry found, inserting new one
DEBUG: fwd_on_incoming_interest, L386: [FORWARDER] (fwd_on_incoming_interest) Prefix match in content store found
```

Figure 5.8: Serial output of the udp-basic-producer showing UART-based FIB setup for unicast Interest reception and Content Store insertion

```
I (12613) esp_netif_handlers: sta ip: 192.168.0.94, mask: 255.255.255.0, gw: 192.168.0.1
I (12613) ▯▯▯Enter: <local-port> <remote-ip> <remote-port> <name-prefix>
Raw input: '6363 192.168.0.232 6364 /test'
Local port: 6363
Remote IP: 192.168.0.232
Remote port: 6364
Name prefix: /test
Security backend initialised
Forwarder initialised
DEBUG: fwd_data_pipeline, L466: [FORWARDER] (fwd_data_pipeline) No cs entry found, inserting new one
On data
It says: I'm a Data packet.
DEBUG: ndn_forwarder_express_interest, L269: [FORWARDER] (ndn_forwarder_express_interest) Prefix match in content store found
On data
It says: I'm a Data packet.
```

Figure 5.9: Corresponding device's serial output of the udp-basic-consumer showing UART-based FIB configuration and successful Data retrieval from a remote producer

Inter Device Testing - Multicast

Multicast faces were set on both of the boards in new applications, udp-group-consumer and udp-group-producer, with the multicast group IP and port set to match NFD's,

which is "224.0.23.170" and 56363. The UART input was still used to set the prefix at runtime, and once this was done, the devkits could send Interest and receive Data. When Interest was expressed from a PC via NFD on the same network, the producer boards could also reply with Data.

Cryptography

Initially, these examples signed and verified Data packets using a SHA-256 digest. Once basic implementations were verifiably transferring packets, ECDSA key generation, AES encryption/decryption, Interest forwarding, and Data signing and verification were added to the `udp-basic-consumer-producer` program using the NDN-Lite wrapped Mbed TLS backend.

Because this example runs on a single board, both keys are available locally. The forwarding and receiving process is as follows:

1. **Key generation.** An ECDSA P-256 key pair is created with:

   ```
   ndn_ecc_make_key(&ecc_pub_key, &ecc_prv_key, NDN_ECDSA_CURVE_SECP256R1, 1)
   ```

2. **Encryption.** The Data content is encrypted with AES-128-CBC before signing via:

   ```
   ndn_aes_cbc_encrypt((const uint8_t*)str, strlen(str) + 1, encrypted_buf, &enc_size,
   aes_iv, &aes_key)
   ```

3. **Data signing.** Inside the `on_interest` callback, the outgoing Data packet is signed with the ECDSA private key using:

   ```
   ndn_data_tlv_encode_ecdsa_sign(&encoder, &data, &identity_name, &ecc_prv_key)
   ```

4. **Verification.** When the Data packet arrives, the `on_data` callback verifies the ECDSA signature using:

   ```
   ndn_data_tlv_decode_ecdsa_verify(&data, rawdata, data_size, &ecc_pub_key)
   ```

5. **Decryption.** The content is then decrypted with the AES symmetric key using:

   ```
   ndn_aes_cbc_decrypt(data.content_value, data.content_size, dec_buf, &dec_size, aes_iv,
   &aes_key)
   ```

This program could allow the testing of cryptographic operations such as their runtime performance, power usage and differences in metrics across selected devkits including the results between using the Mbed TLS and TinyCrypt/micro-ecc backends, this is explored in Section 6.3 and 6.5. Although AES encryption was not used in the smart home system, this program shows that encryption and decryption are possible in further implementations using this adaptation. The IV and AES key were hardcoded for benchmarks but

should be dynamically generated in a real deployment.

5.4 Smart Home Sensor Application

The smart home application described in Section 4.4 relies on a controller/manager just as the ndn-iot-package-over-posix example does. A manager was deployed on a PC using a script `ndn-manager.py`, which relied on a series of asynchronous functions that were non blocking to aid in parsing incoming Interest, extracting information, validating keys and communicating with a REST API for front end updates. Helper functions were created that utilise the `nfdc` command (Named Data Networking, 2024a) that comes with NFD for route management and logging.

On the ESP32 node, the fundamental logic for creating the basic applications was reused to turn all of the nodes into Data producers with the ability to advertise their prefixes for the manager to see, and with advertising Interest signed and verified using ECDSA.

5.4.1 Device Advertisement

After booting, each device will connect to the Wi-Fi network set in an `sdkconfig` file. It will then generate an ID using its chip model and MAC address. Following this it will create an ECDSA key pair, and creates a multicast face on the NDN multicast group (224.0.23.170, port 56363) as was previously done in the basic group application. The device also registers a callback for its data prefix so that incoming Interests trigger a Data response, the values used for this are hardcoded to match real sensors such as "temperature" and locational values such as "living-room", these values may be input using a more sophisticated control suite in a real scenario, for example if a device was to be moved to another room, its prefix could be altered on the dashboard and the device would update itself.

The device can then send an advertisement Interest with the hierarchical name `/home/SD/ADV/<device_location>/<device_id>`, signed with its ECDSA private key to the multicast face. The Interest lifetime is set to one second so that PIT entries do not linger for unnecessary amounts of time. By using the ApplicationParameters field, the Interest can carry a structured payload, inside this was packed the following values: 4-byte big-endian freshness value, the device's full prefix, base prefix, an IP address as a pipe-delimited string, and the raw 64-byte secp256r1 public key encoded as the 32-byte X coordinate concatenated with the 32-byte Y coordinate.

Advertisements were sent every 30 seconds. On the manager side, an AdvListener class contains the necessary functions that are called by an asynchronous `async def run(self):` function that will parse these advertisements and extract the device meta-

data and public key that was packaged in ApplicationParameters. This will then update a device registry with the information retrieved. If no advertisement has been received from a device within 90 seconds it will be marked as offline and removed from the NFD FIB.

```
I (12616) NDN: === NDN Device: esp32p4-40 ===
Security backend initialised
Forwarder initialised
I (12636) NDN: ECDSA key pair generated, identity: /home/living-room/esp32p4-40
I (12636) NDN: Multicast face created
I (12636) NDN: Registered: /home/SD
I (12646) NDN: Registered: /home/temperature (handles all sub-prefixes)
I (12656) NDN: ADV sent: /home/SD/ADV/living-room/esp32p4-40 [prefix=/home/temperature/living-room base=/home/temperature ip=192.168.0.232]
DEBUG: fwd_data_pipeline, L466: [FORWARDER] (fwd_data_pipeline) No cs entry found, inserting new one
I (12896) NDN: Controller public key received and stored
I (42676) NDN: ADV sent: /home/SD/ADV/living-room/esp32p4-40 [prefix=/home/temperature/living-room base=/home/temperature ip=192.168.0.232]
```

Figure 5.10: Serial output of an ESP32 node during smart home application startup showing ECDSA key generation, multicast face creation, and prefix advertisement

```
20:01:45 [MANAGER] No devices online
20:01:46 [MANAGER] 127.0.0.1 [08/Apr/2026:19:01:46 +0000] "GET /stats HTTP/1.1" 200 3(
20:01:46 [MANAGER] 127.0.0.1 [08/Apr/2026:19:01:46 +0000] "GET /devices HTTP/1.1" 200
20:01:46 [MANAGER] 127.0.0.1 [08/Apr/2026:19:01:46 +0000] "GET /nfd/faces HTTP/1.1" 2(
20:01:46 [MANAGER] 127.0.0.1 [08/Apr/2026:19:01:46 +0000] "GET /nfd/routes HTTP/1.1" 2
20:01:50 [MANAGER] Route remove: /home/temperature → udp4://224.0.23.170:56363
20:01:50 [MANAGER] Route add: /home/temperature → udp4://224.0.23.170:56363
20:01:50 [MANAGER] ROUT esp32p4-40   | /home/temperature → 192.168.0.232
```

Figure 5.11: Manager log showing device discovery, route registration, and NFD FIB updates upon receiving an advertisement Interest

5.4.2 Bootstrapping Exchange

New devices are discovered through these advertisements and initially, the manager stores their information with a trust status of `unverified`. A device card will be shown on the dashboard along with this trust status and an option to either approve or reject the device is presented.

When approved, the manager will register an NFD route mapping the device's base prefix to the multicast face using `nfdc` and this will allow Interest to be forwarded to the device. The public key extracted from the advertisement is held in the device store and verified with an ECDSA validator for subsequent data fetches. Upon receiving a Data packet, the validator verifies the signature using the stored public key before the data is accepted, and if this fails the data will be dropped and not read. Full certificate issuance, where the manager would sign and return a certificate binding the device's name to its public key, is not fully implemented, currently when ESP32 nodes send signed advertising Interest, the manager will respond with a Data containing its public key, but later exchanges do not involve signed Interest meaning mutual authentication is incomplete in practice.

5.4.3 Data Production and Signing

ESP32 nodes that receive Interest from the manager after bootstrapping will trigger an `on_interest` callback when the Interest matches the registered prefix. When the Interest is accepted, the device creates a JSON payload which contains the fake sensor reading and unit, along with the sensor device identifier. The Data packets are named with prefixes defined in the Section 4.3.4 such as, `/home/temperature/living-room` and parent prefixes in Interest can trigger a Data return as long as it contains the same root prefix and the `CanBePrefix` encoding exists. This may be used in the scenario that temperature or readings don't necessarily need to be from a specific room. The FreshnessPeriod parameter is set to 10 seconds, so that cached copies will no longer satisfy `MustBeFresh` Interests and old readings will not be kept for unnecessary lengths of time.

The Data packet is signed with ECDSA using the device's private key and its identity name using `ndn_data_tlv_encode_ecdsa_sign()`, which encodes the TLV packet with the signature appended. The signed packet is then referenced by `ndn_forwarder_put_data()`, which returns it to the face the Interest arrived on.

5.4.4 Manager and Dashboard

The manager utilises the `aiohttp` (Svetlov, 2026) library which is a third party library for Python built upon the `asyncio` (Python Software Foundation, 2026) library, and allows the creation of asynchronous functions. This is beneficial in a networking application that will constantly await requests and responses. When these processes are wrapped with this asynchronous logic, if the manager sends an Interest, it will not block all other processes while awaiting a Data response or timing out. It can manage other requests simultaneously such as polling the device registry or sending another Interest.

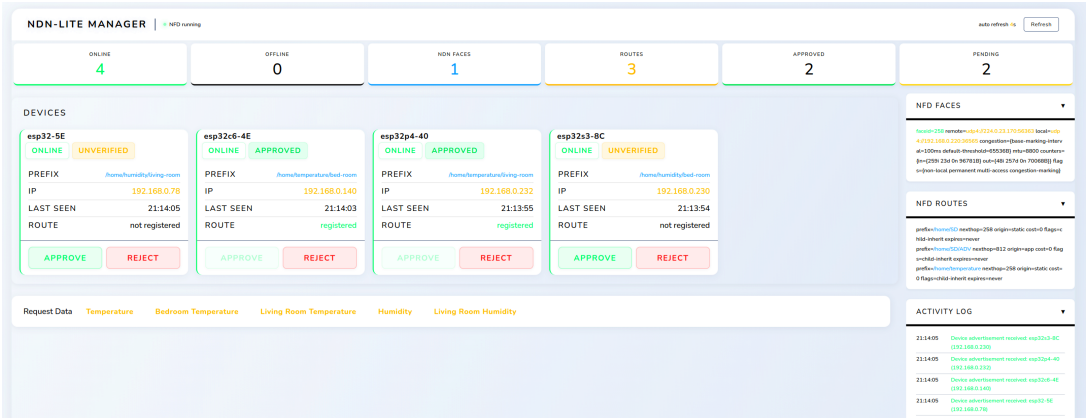


Figure 5.12: Full NDN-Lite Manager dashboard with data retrieval panel, NFD faces, routes, and activity log, see Figure A1.4 in the Appendix for a larger image

Manager interaction with the NFD instance is performed with the `python-ndn` (Named Data Networking, 2026b) library which is what was utilised to send and receive packets to and from ESP32 nodes. Once devices are discovered and are populating the device registry using the ADVlistener class, device information is logged to a `devices.json` file through a `DeviceStore` class, which will enter each device's identifier, location, IP address, prefix, trust status, and raw public key. This JSON file persists upon closing and restarting `ndn-manager.py` so that any previously known devices are shown, and will be set to offline until a fresh ADV Interest is received.

The REST API exposes endpoints for retrieving all device information, approving and rejecting devices, fetching data from full named data or just the parent prefix. The dashboard uses these endpoints and displays the current status of ESP32 nodes using a combination of HTML and CSS.

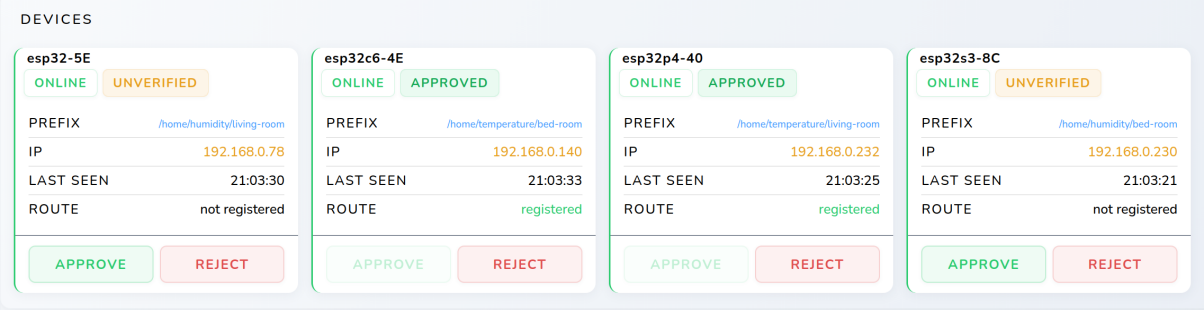
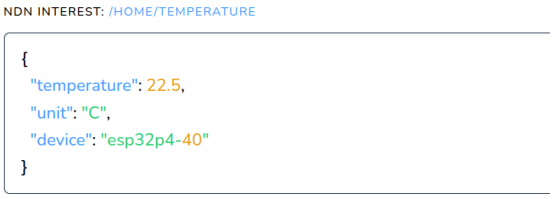
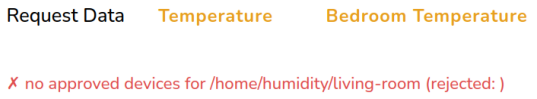


Figure 5.13: NDN-Lite Manager dashboard showing four differing ESP32 devices with trust status, prefix assignments, and registration state



(a) Data returned for Interest with prefix /home/temperature



(b) No Data returned for Interest with name /home/humidity/living-room due to no trusted devices in the network with that prefix

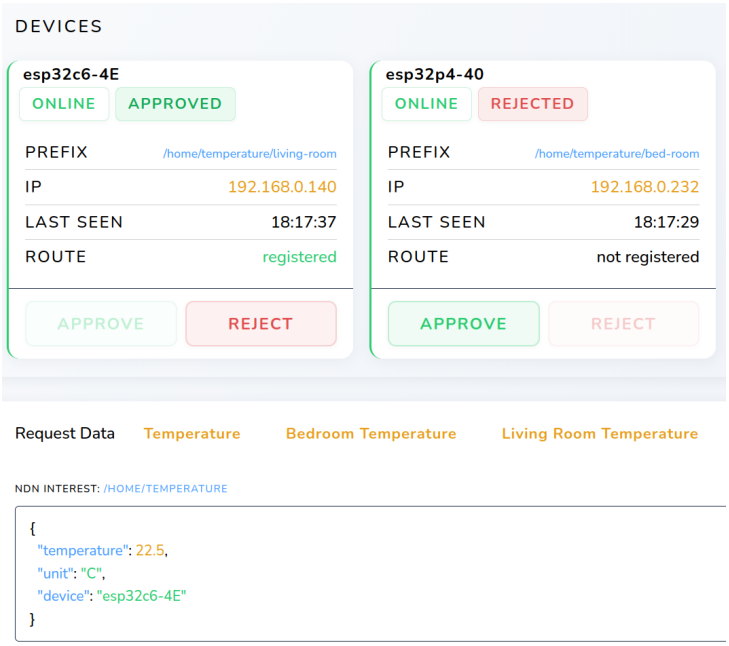


Figure 5.14: ESP32-P4 is rejected and data is returned by the ESP32-C6 on the prefix /home/temperature

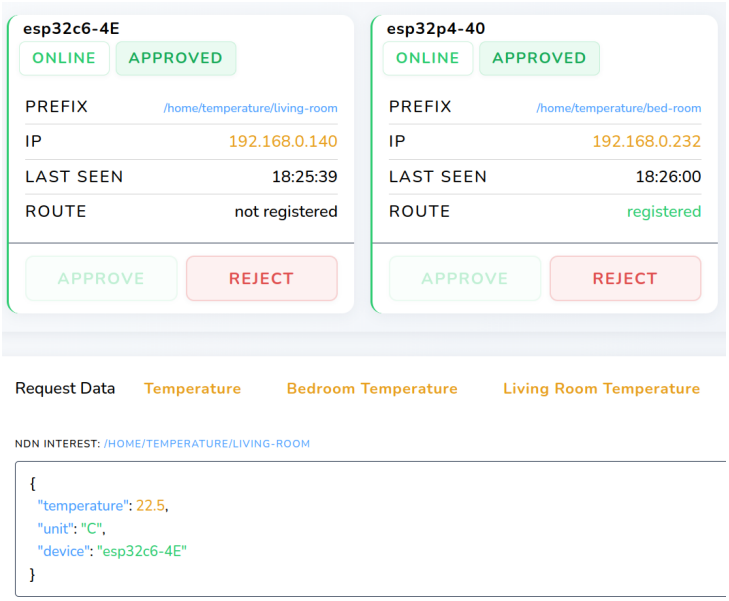


Figure 5.15: ESP32-C6 returns Data for Interest that has prefix /home/temperature/living-room

5.5 Issues and Limitations

NFD was installed using the instructions given at its repository, however it would not correctly startup and when checking the status using `systemctl status nfd`, this showed that NFD was trying to access a `/run/nfd` directory that was not there due to not successfully being created in the installation. This directory was created and permissions were given for it to be created and used across rebooting NFD which solved the issue.

Data encryption was not performed, therefore outsiders with access to the network can view traffic exposing the data sensors possibly sensitive information. A more secure implementation would use a bootstrapping and security algorithm such as SSP to prevent this as they implement secure key exchange.

Initial issues in implementation introduced set backs, one such problem was an incorrect compilation setup. It was found that the NDN-Lite library has two methods of searching its FIB for prefix matching. These were `name-splay` and `name-tree` and the former was being compiled.

Upon review it was found that both `name-splay` and `name-tree` are *Trie* data structures (Fredkin, 1960). Tries are commonly used for autocompletion and spell checking tasks through the storing and retrieval of strings with a tree like structure (Sarker et al., 2020). The splay method differs from the default tree method by introducing splay trees at each nodes child which can self adjust for quicker access to recently accessed names. This implementation was however unfinished leading to no prefix matching for forwarding, therefore the `name-tree` implementation was selected for compilation. The splay implementation could be fixed and deployed for benchmarking against the original as future work.

Another issue was that there needed to be a compilation flag to enable the hardware RNG signing, if `DFEATURE_PERIPH_HWRNG` was not set, a function would be called: `uECC_sign_deterministic` which generates a nonce deterministically. ECDSA security depends on this nonce being unpredictable; deterministic signing may produce weaker nonces on constrained hardware so this compile flag must be set as described in the NDN-Lite documentation (Named Data Networking for IoT, 2021a).

5.6 Summary

This section introduced steps that were taken to successfully implement NDN networking onto the ESP32 series using ESP-IDF. The build system configuration showed that multiple applications could use a single components file, this made it simple to test between consumer and producer setups without replicating the components folder multiple times.

However, in a stand-alone ESP project, this components folder should be left in its typical place.

All of the wrapper functions that NDN-Lite needed to be populated in order for the adaptation were documented, and the selected libraries for; clock and timing, Wi-Fi, RNG, face implementation and cryptography were implemented. These were all contained within a single adaptation folder.

The basic applications that successfully tested NDN-Lite's abilities correctly used the adaptation layer, with demonstrated Interest and Data exchange on a single device, between two devices with a unicast face, and multiple devices with multicast faces. Parameters could be input to the device at runtime using the UART driver and connection which solved basic FIB entry issues.

Finally, a simulation of a smart home sensor network was created that used the basic application principles. This could deploy on multiple ESP32 devkits and communicate with a manager that was processing events through a PC running NFD and could update the NFDs routes and FIB for prefix matching or remove information if devices no longer appeared online. Live network information was retrieved from the manager and shown by a dashboard that was consuming the REST API endpoint. This allowed a basic onboarding system that nodes could join and subsequently respond to manager Interest.

6 Evaluation

This chapter details the performance, resource utilisation, and power characteristics of the NDN-Lite ESP32 port. As multiple applications were implemented, their behaviour varied greatly depending on their use case. Metrics were collected from the basic applications described in Chapter 5 to understand these differences, which could be examined by enabling and disabling security, testing with loopback scenarios and inter-board benchmarking.

Resource profiling was carried out using native ESP-IDF tools, which allowed in depth component analysis. A cross-examination was performed to parametrise the overhead introduced by NDN-Lite, the cryptographic backend, and the Wi-Fi networking stack, as they were identified as the largest contributors to binary size.

Runtime behaviour was characterised using metrics related to Interest and Data packet transmission and Content Store caching. Cryptographic performance was evaluated by comparing the TinyCrypt and micro-ecc backends with the new Mbed TLS backend using the compile-time definitions outlined in Section 5.4.

Power consumption was measured with a specialised current measurement device that allowed extensive logging, this captured events that happened in short time frames such as Wi-Fi activity, cryptographic operations and forwarding events. Current draw metrics across idle, active forwarding, and cryptographic operations were also collected.

All of these metrics combine to detail the overall performance and suitability of NDN-Lite for deployment on ESP32 hardware.

6.1 Flash Usage

Resource usage and constraints depend heavily on the devkit used, and testing on boards with different architectures can have varying results. ESP-IDF provides the following tools to decompose a project by size and specific RAM usage: `idf.py size`, for an individual component breakdown: `idf.py size-components`, and for a more in-depth file breakdown: `idf.py size-files`.

The ESP32-P4 flash size was profiled with the udp-basic-consumer application first using `idf.py size`:

Table 6.1: ESP32-P4 Flash Memory Usage Summary for udp-basic-consumer

Memory Type/Section	**Used [bytes]**
Flash	693,550
.text	477,742
.rodata	215,304
.appdesc	256
.init_array	248

Total image size: 778,926 bytes (.bin may be padded larger)

The profile shows that the total image size amounts to approximately 779KB. A flash size under 1MB is desirable for constrained devices, and this could be reduced even further depending on how necessary the biggest contributors are which could be found with `idf.py size-components`:

Table 6.2: ESP32-P4 Flash Usage by Component (Top 9 by Size)

Archive	**Flash Total**	**.text**	**.rodata**
libespressif___esp_hosted.a	155,883	89,464	66,415
libesp_app_format.a	116,151	554	115,333
liblwip.a	112,434	108,738	3,696
libc.a	54,031	50,684	3,347
libmbedcrypto.a	38,410	35,628	2,782
libesp_hw_support.a	28,032	25,170	2,842
libesp_driver_uart.a	22,976	22,186	782
libadaptation.a	20,341	20,248	93
libhal.a	11,096	10,954	142

The first of the top contributors that exceeds 100KB is libespressif___esp_hosted.a. This library is used by processors to link communication between them and a coprocessor. In the case of the P4 devkit, this is because it lacks Wi-Fi capabilities and uses this library to link networking between the C6 coprocessor and itself. The next library, libesp_app_format.a, is an artefact of linker string merging during compile time for string literals across many components. This is for optimisation, as described in the ESP-IDF size guide (Espressif Systems, 2026k). This optimisation can be turned off to better see the split of component usage, but they note that the binary will grow in size and therefore it remained on for this project. The final contributor is liblwip.a, which is the lwIP library used for networking.

These results show that the greatest components are related to networking and device-specific communication, not due to the NDN-Lite library, which is contained within libadaptation.a. Some smaller optimisations could be made for the base applications using the profile results, such as the removal of libesp_driver_uart.a, which would free around 23KB if input were instead hardcoded.

For comparison, a profile was taken of the application when it may be deployed on a WROOM-32E, as this has its own Wi-Fi capabilities: The results show a similar binary

Table 6.3: ESP32 WROOM-32E Flash Usage by Component (Top 9 by Size)

Archive	**Flash Total**	**.text**	**.rodata**
libnet80211.a	139,116	125,674	13,442
liblwip.a	89,154	85,483	3,671
libmbedcrypto.a	72,883	65,872	7,011
libwpa_supplicant.a	64,125	62,537	1,588
libesp_app_format.a	58,824	417	58,407
libc.a	53,982	50,892	3,090
libpp.a	46,576	41,609	4,967
libphy.a	34,138	34,138	0
libadaptation.a	17,914	17,758	156

size of around 810KB. The most notable comparison is between the Wi-Fi communication libraries on the WROOM-32E, libnet80211.a consumes 139KB of flash to drive the on-chip Wi-Fi hardware directly, while on the P4, libespressif___esp_hosted.a consumes an almost similar 155KB to facilitate communication with the C6 coprocessor, demonstrating that offloading Wi-Fi to a coprocessor does not provide any meaningful flash space reductions as the layer required to communicate with the coprocessor is roughly the same size as a native networking driver.

Another difference is libmbedcrypto.a, which consumes 72,883 bytes on the WROOM-32E, compared to 38,410 bytes on the P4, nearly double. As discussed above, this may be due to the WROOM-32E's native support for WPA2/WPA3 authentication, which pulls in additional Mbed TLS functionality. The presence of libwpa_supplicant.a further shows this with a flash size of 64,125 bytes and libpp.a at 46,576 bytes, which implements the WPA supplicant and Wi-Fi MAC layer, respectively. Neither of which appears in the P4 build, as the coprocessor handles these responsibilities. The libmbedcrypto.a size difference may also be due to the WROOM-32E relying on software cryptographic implementations, whereas the P4 was found to rely on hardware acceleration for the same operations, as discussed in Section 6.3. The adaptation library, libadaptation.a is 17,914

bytes on the WROOM-32E, consistent with its footprint in other builds, confirming that the NDN-Lite layer itself contributes little compared to the larger networking and driver libraries.

Table 6.4: Total Binary Image Size by Device for udp-basic-consumer

Device	**Image Size (bytes)**
ESP32-P4	778,926
ESP32-WROOM-32E	810,973
ESP32-S3-EYE	805,617
ESP32-C6	901,994

The binary size comparison in Table 6.4 shows that the C6 is an outlier with an extra 90 - 100KB in size. Upon inspection of the components, the networking libraries: libnet80211.a, libpp.a, liblwip.a and libwpa_supplicant.a on the C6 are larger when compared to the other chips.

Table 6.5: Networking Library Flash Usage: ESP32-C6 vs ESP32-WROOM-32E

Archive	**C6 Flash (bytes)**	**WROOM-32E Flash (bytes)**
libnet80211.a	169,546	139,116
liblwip.a	109,223	89,154
libwpa_supplicant.a	78,320	64,125
libpp.a	65,157	46,576
Total	**422,246**	**338,971**

The C6's larger binary size difference is due to the extra wireless capabilities that Espressif introduce for this particular SoC. As the WROOM-32E supports only Wi-Fi 4 and Bluetooth Classic/LE, it results in a smaller networking stack. The C6 includes Wi-Fi 6 (802.11ax), BLE 5.3, and the IEEE 802.15.4 protocol (Espressif Systems, 2022b). The ESP-IDF target support package for the C6 will compile all of these drivers regardless of application usage, as reflected in libnet80211.a library (169,546 bytes vs 139,116 bytes on the WROOM) and the libpp.a library (65,157 bytes vs 46,576 bytes), together accounting for the majority difference in total binary size between the two devices.

Cryptography Flash Sizes

A comparison of flash usage was performed using the default libraries TinyCrypt and micro-ecc (TC and MECC) vs MBed TLS. This was possible due to the compile-time definitions described in Section 5.2.5. On the P4 with the udp-basic-consumer application, the flash size was smaller when using TC and MECC (736,184 Bytes) and when using MBed TLS (778,922 Bytes), an increase of 42,738 bytes. It can be seen that in

the `idf.py size-components` when using TC and MECC, the adaptation library they reside in grows by approximately 17KB (37,678 from 20,726) as they are packaged inside NDN-Lite. TC and MECC are expected to be lighter as they were chosen by NDN-Lite developers due to their small form factor, so they could be prepackaged as a default backend in the case that another backend could not be implemented on new native device adaptations, whereas the Mbed TLS fork contains device driver implementations to work with hardware accelerators across many boards.

6.2 RAM Usage

RAM usage was measured using another set of ESP-IDF-provided tools; the first, `esp_get_free_heap_s` provides a runtime snapshot of the available free heap. The second was `esp_get_minimum_free_heap_si` which logs the minimum amount of free heap since the program started, providing a watermark of the peak. Used together, they give insight into the runtime live and peak memory usage during operation. Gathering metrics was done on the following applications.

- udp-basic-consumer-producer
- udp-group-consumer ↔ udp-group-producer

To help in gathering these metrics, they were modified to perform one hundred Interest sending and Data retrieval interactions with and without cryptography using Mbed TLS and then with TC and MECC. An automatic script would print the collated results from the free heap size and the minimum free heap size. These programs would then be flashed to each board, and the process repeated until the ram usage for each application was quantified across the selected devkits.

The points at which these metrics were taken are the following:

- When the device boots
- Wi-Fi connection and IP acquisition
- NDN-Lite initialisation and ECDSA key pair generation
- UDP face construction and forwarder route registration
- Interest forwarding and Data receiving

Three of the devkits were individually tested using the udp-basic-consumer-producer application with Mbed TLS and ECDSA enabled. Then two ESP32-P4 devkits were used for wireless communication benchmarks using udp-group-consumer and producer. The main task size was initially set to 16KB, which contains all the initialisation and forwarding logic. It was found that the main task used at most 10KB, as determined

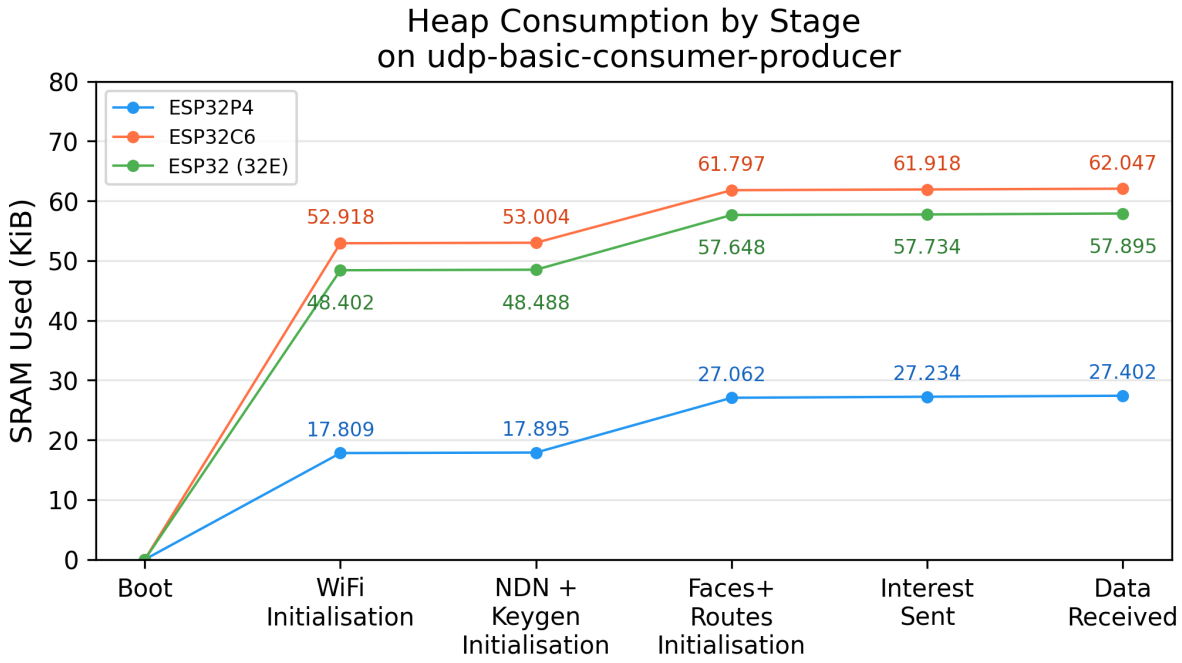


Figure 6.1: Heap consumption by stage on the udp-basic-consumer-producer application across ESP32-P4, ESP32-C6, and ESP32-WROOM-32E, the ESP32-P4 devkit has the lowest due to offloading Wi-Fi functionality to the coprocessor

by the FreeRTOS `uxTaskGetStackHighWaterMark()` function; therefore, this could have been reduced to around 11KB-12KB to sit safely above the 10KB peak usage.

The total available heap per devkit was calculated by using the amount of free heap on boot, (eg. `[HEAP] boot free=508808 min=508808`). Memory usage for each process is determined based on this metric. Unlike the static memory profiles, the results show differences between running the Wi-Fi stack on a coprocessor as opposed to having on-chip access, with more resources required for both the C6 and the WROOM-32E when initialising the Wi-Fi stack (50KiB) as opposed to on the P4 (18KiB) a reduction of roughly 64%, resulting in less than half the heap being consumed for the P4. The NDN-Lite stack itself only required around 10KiB of heap during runtime in these tests. ECDSA signing and verification were performed on the same chip, as shown in Figure 6.1, and across other chips via inter-board communication between two P4s, as shown in Figure 6.2. When splitting ECDSA signing and verification, slightly more resources are used on the consumer side for verification; this is extremely small (P4 producer, Data signed: 24.98 and P4 consumer, Data verified: 25.83, total heap usage at these points from Figure 6.2).

Because the Wi-Fi stack is the primary heap consumer, whereas the NDN-Lite implementation is not, offloading networking to a coprocessor may be a practical strategy for constrained devices. This aligns with the coprocessor approach used in the Arduino UNO R4 WiFi (Section 3.4), where the ESP32-S3 coprocessor handles networking.

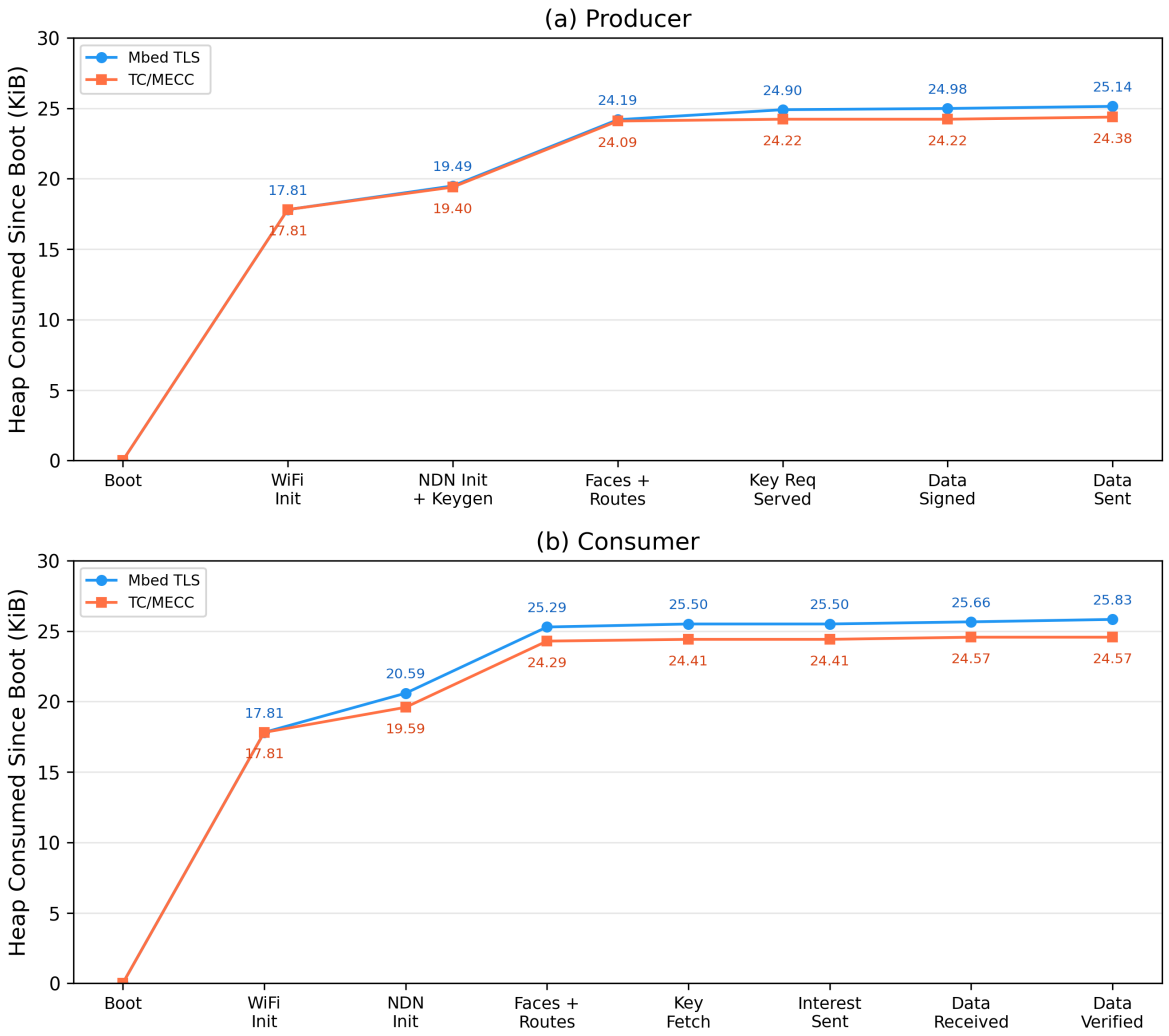


Figure 6.2: Heap consumption by stage for udp-group-producer (a) and udp-group-consumer (b) on the ESP32-P4 comparing Mbed TLS and TCC/MECC backends

6.2.1 Content Store Caching

As the CS caches data during the runtime of an application, sending Interest to data and retrieving new content increases the amount of memory used. NDN-Lites' implementation of the CS on the devkits could be benchmarked by setting its max entries to 50 and testing the insertion of 100 separate data items.

Each Data amounted to 172 bytes, and the benchmarks from Figure 6.3 showed successful operation of the CS with heap usage no longer growing after 50 entries being assigned due to old data being removed for new entries as opposed to assigning more memory.

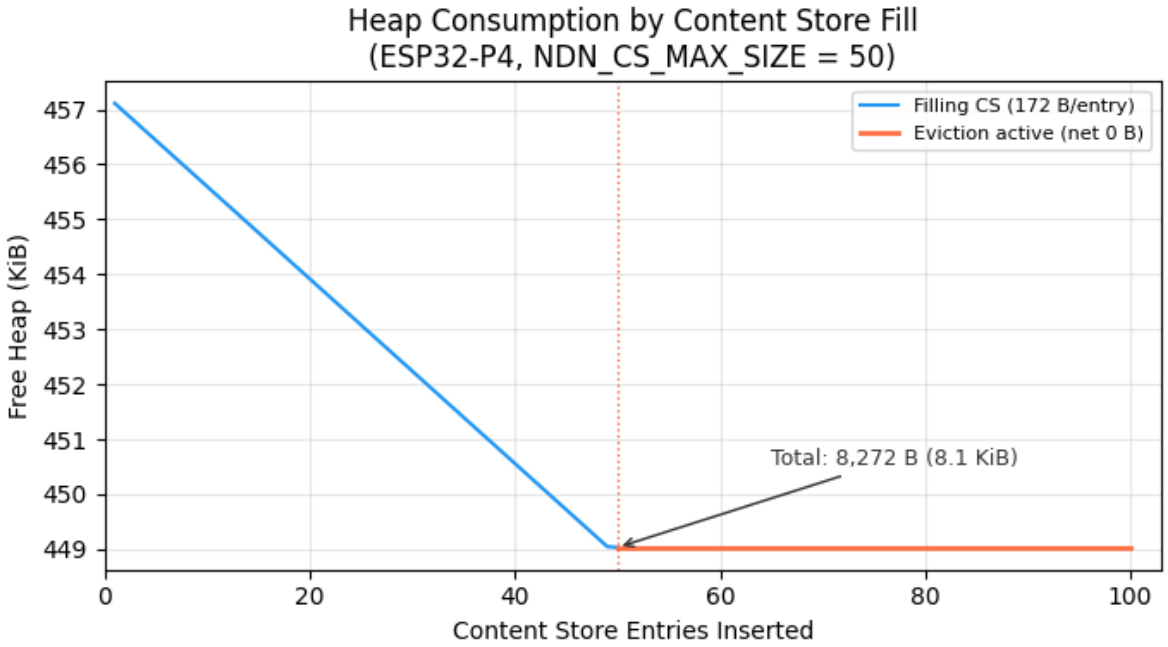


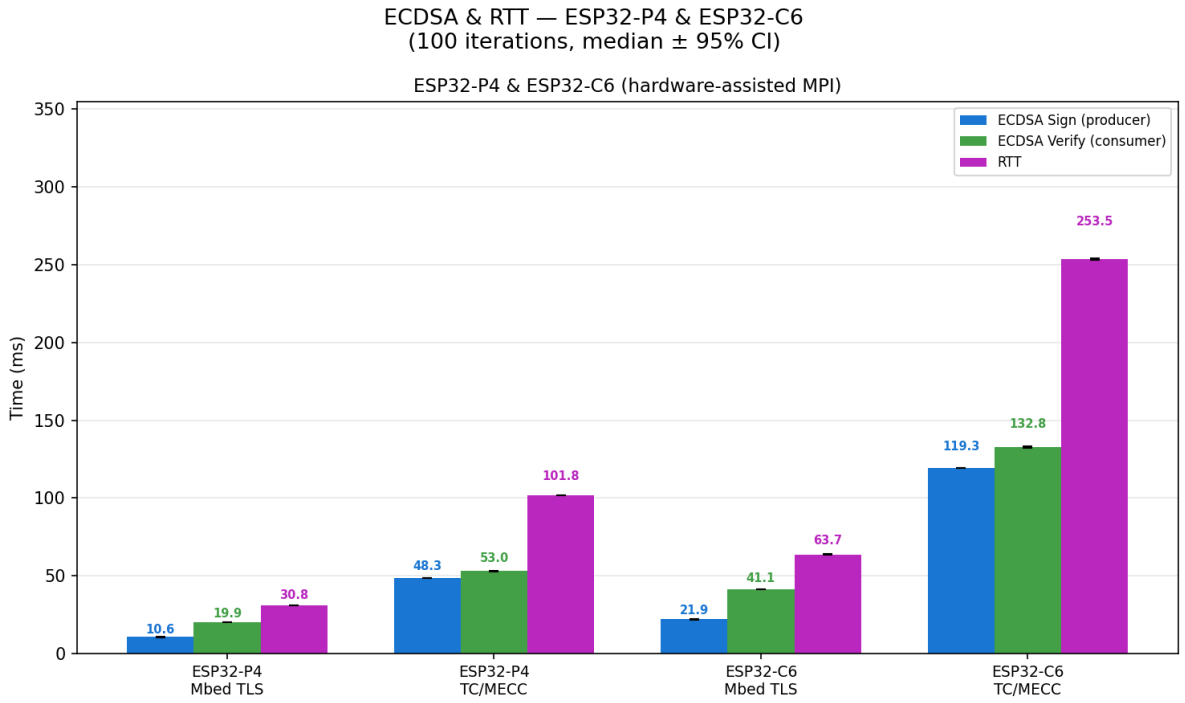
Figure 6.3: Heap consumption during Content Store fill on the ESP32-P4 with a maximum of 50 entries at 172 bytes each

6.3 Latency

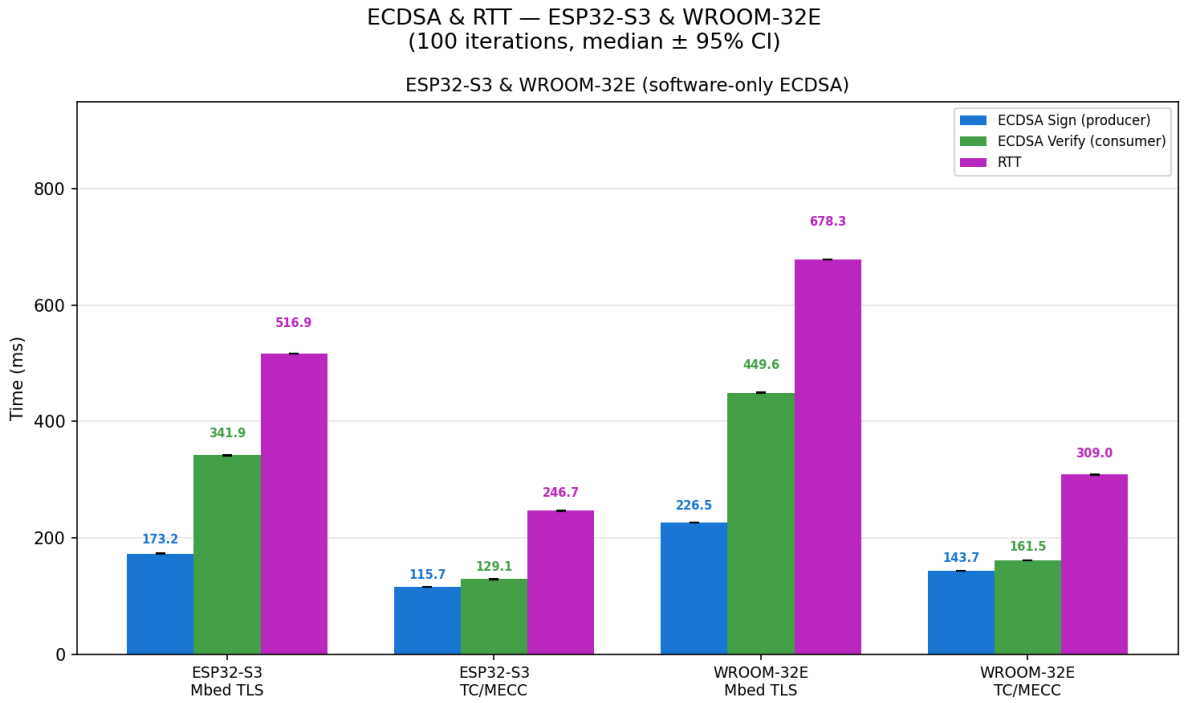
The time for Interest and Data retrieval was benchmarked, with one round trip defined as the creation of Interest, its forwarding, and successful Data retrieval. To eliminate network latency and understand the time delays for pure NDN-Lite processes the benchmarks were taken with udp-basic-consumer-producer in a loopback test.

Three separate timing measurements were taken: the producer's sign time, the consumer's verify time and one round trip, which includes both previous operations and the latency of the NDN-Lite stack itself. Median values for these measurements were computed over 100 iterations, with the CS disabled to evaluate timing without caching. A 95% confidence interval was computed for each of the collected values automatically as $\bar{x} \pm 1.96 \cdot \frac{s}{\sqrt{n}}$, where s is the sample standard deviation.

The latency on the P4 is the lowest as expected due to it possessing the greatest processing capabilities, and when using Mbed TLS, the cryptographic hardware acceleration successfully reduces the ECDSA sign latency from 48.3ms to 10.6ms, representing a 78% reduction, and ECDSA verify from 53ms to 19.9ms, a 62.45% reduction compared to the TC and MECC backend. These results are also reflected with the C6 which reduced its total RTT from 253.5ms to 63.7ms, a 74.9% reduction overall.

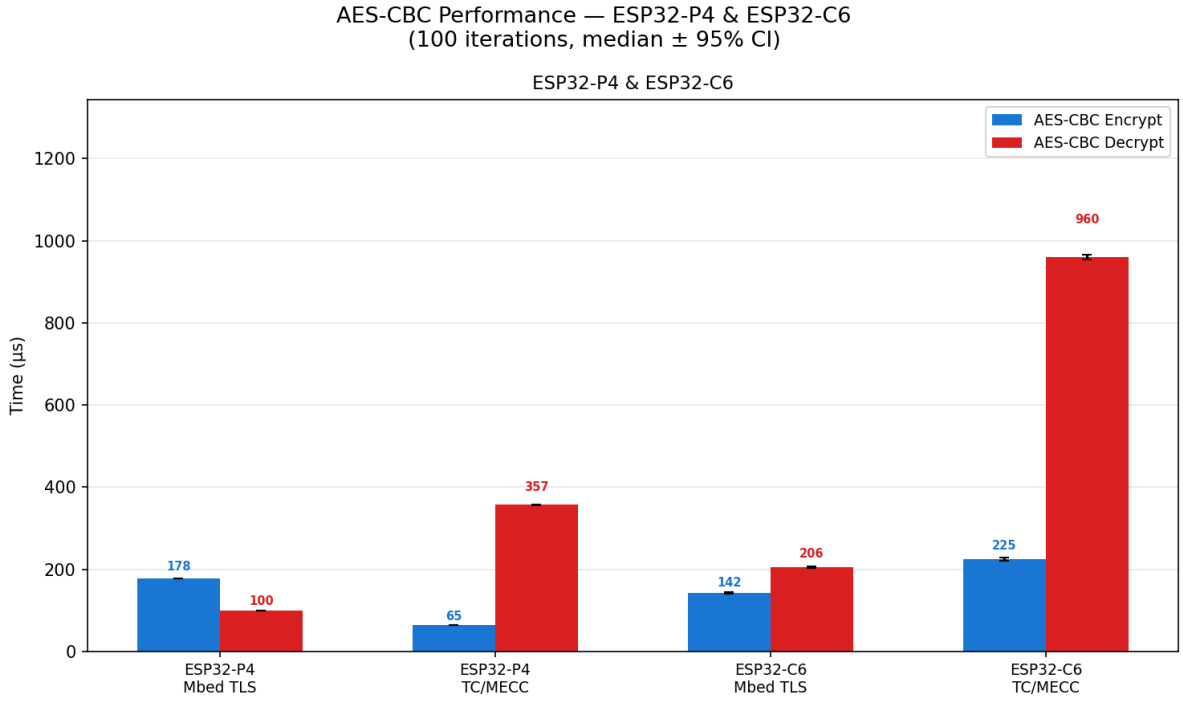


(a) Latencies on P4 and C6, Mbed TLS showing a clear performance advantage due to these devices having ECDSA hardware acceleration

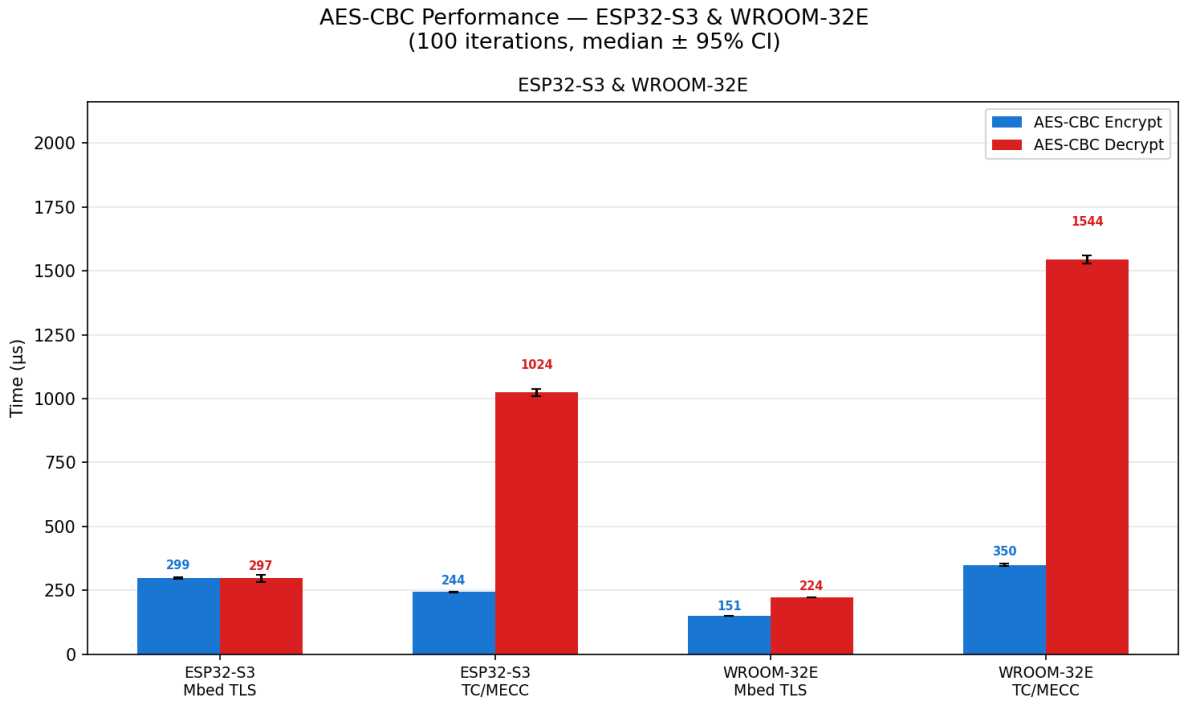


(b) Latencies on the S3 and WROOM-32E, TC and MECC showing a clear performance advantage due to these devices having no ECDSA hardware acceleration

Figure 6.4: ECDSA sign and verify latency across ESP32 variants



(a) AES-CBC-128 encrypt and decrypt latency on P4 and C6, Mbed TLS showing a clear performance advantage due to these devices having AES hardware acceleration



(b) AES-CBC-128 encrypt and decrypt latency on S3 and WROOM-32E, Mbed TLS showing a clear performance advantage due to these devices having AES hardware acceleration

Figure 6.5: AES-CBC-128 encrypt and decrypt latency across ESP32 variants

The S3 and the WROOM-32E, however, have the opposite result. ECDSA operations were completed on both with lower latencies when using TC and MECC, with a decrease of 52.3% for the S3 and 54.5% for RTT, and this is due to both of them not having any hardware support for ECDSA operations, as seen in Table 4.2, and therefore, when using Mbed TLS, they fall back on its software implementations to complete them. TC and MECC are both optimised specifically for software cryptographic computations and therefore outperform the Mbed TLS forks' software ECDSA implementations. For AES, the S3, and WROOM-32E, both rely on hardware acceleration, as do the other chips, with some small speed advantages for Mbed TLS; however, these differences are in the lower-microsecond range, as seen in Figure 6.5.

Disabling ECDSA showed that SHA-256 verification and signing could be performed with both Mbed TLS and TC/MECC, yielding results that were highly similar, as seen in Figure 6.6. Using Mbed TLS on the WROOM-32E was also slower than TC and MECC, despite the WROOM-32E's hardware acceleration for SHA (0.204 ms on Mbed TLS compared to 0.103 when using TC and MECC). The verification latencies in Figure 6.7 show little to no difference, with microsecond variance of $\pm 1000\,\mu$s.

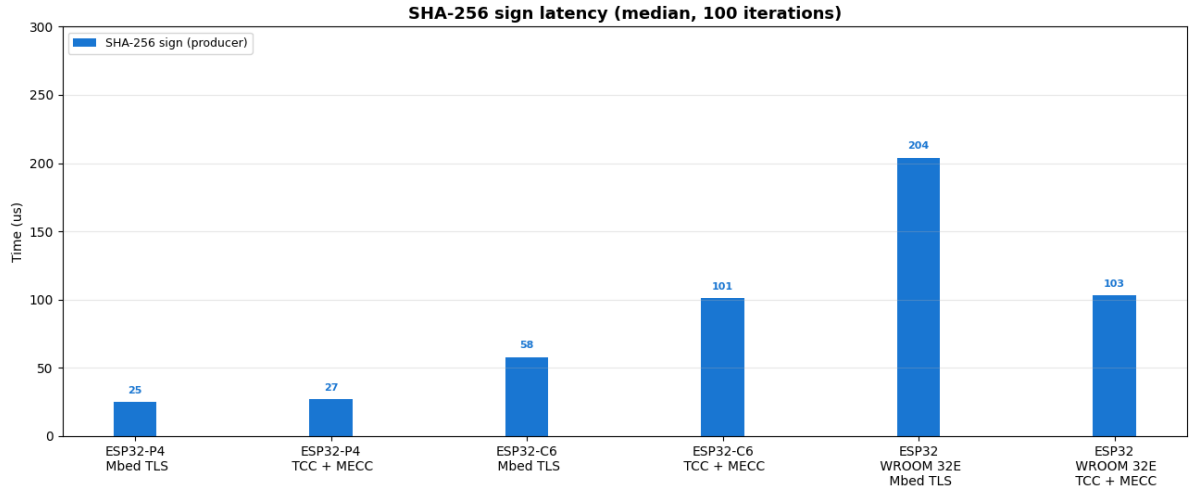


Figure 6.6: SHA-256 sign latency across ESP32 variants comparing Mbed TLS and TC and MECC backends, these latency differences are extremely small and therefore cryptographic back-end choice based on these results is not necessary

The latency tests were not performed between two boards; this was because the RTT suffered from extreme variance due to the fact that the base applications act on the reception of packets through a while loop with a delay, as seen in Figure 6.8 below.

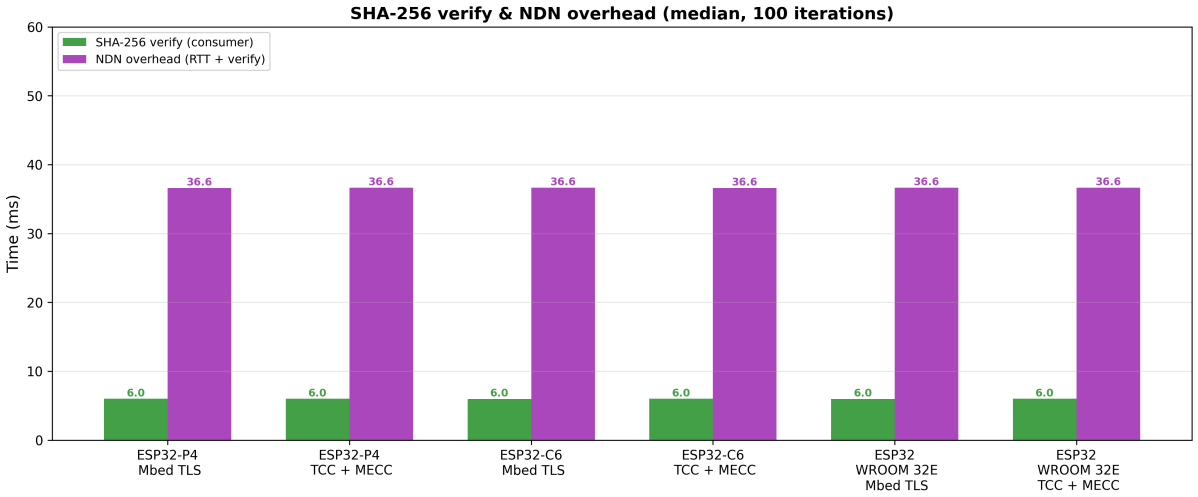


Figure 6.7: SHA-256 verify and NDN overhead latency across ESP32 variants comparing Mbed TLS and TC and MECC backends, results are nearly the same for all back-end choices

```
while (running) {
ndn_forwarder_process();
vTaskDelay(xDelay);
}
```

Figure 6.8: Basic application forwarder polling loop

The fixed `vTaskDelay` introduces periodic idle periods between forwarder cycles, contributing to the RTT. Both consumer and producer applications operate under this logic; therefore, if the period cycles are opposite, the delay will be greater. Measurements involving throughput would not be meaningful due to this logic, and a more specialised networking application would be necessary to benchmark this implementation with a different forwarding implementation, which is identified as future work.

6.3.1 Content Store Retrieval Times

Interest was sent with the parameter MustBeFresh, and the Data FreshnessPeriod was set to one second, then the same previous benchmarks were run for one hundred iterations in the loopback test with udp-basic-consumer-producer. Data was retrieved from the CS until expiry, at which point a new Interest would be successfully created. Data is then retrieved and cached, and, from Figure 6.9, it can be seen that CS entries expire approximately every 17th iteration. The RTT was approximately 31ms when data was absent (Figure 6.4) and 20 ms when data was present, which is essentially the time required to verify the Data.

When the same test was performed between two boards, the delay in the forwarding while

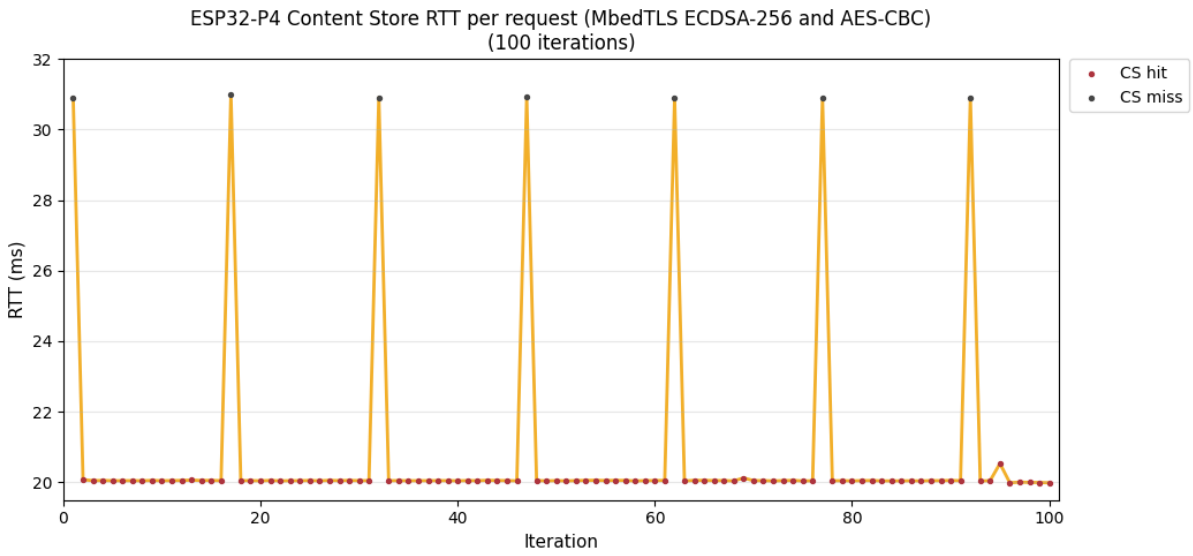


Figure 6.9: Content Store round-trip time per request on the ESP32-P4 with udp-basic-consumer-producer over 100 iterations showing CS hits and periodic cache expiry misses, due to being on the same device, the timing is synchronous.

loop caused large fluctuations in RTT variance, with maximums of 800 ms and minimums of around 40 ms, as shown in Figure 6.10. Efforts to reduce these maximums involved reducing the delay, which resulted in packets no longer being forwarded; the smallest delay possible was around 20ms before issues began to occur.

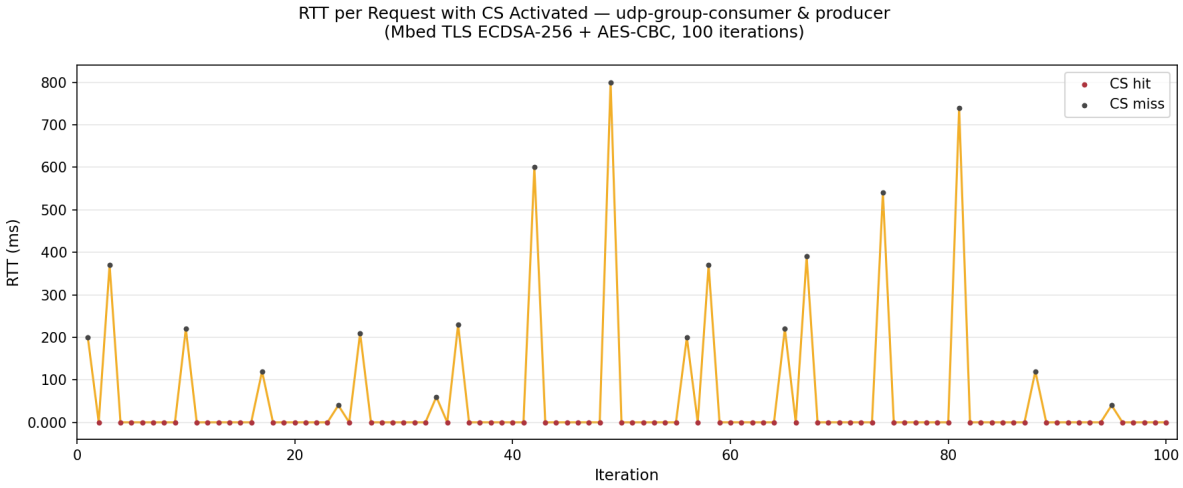


Figure 6.10: RTT between two ESP32-P4's one running udp-basic-consumer, the other running udp-basic-producer. 100 iterations showing CS hits and periodic cache expiry misses and retrieval times varying greatly due to forwarding logic.

6.4 Optimisation

When compiling, there are multiple options that alter the compilation strategy. These include optimising for size, performance and debugging or debugging without any optimisation. Differing compilation parameters can affect board operation speeds and overall performance and can be controlled via an sdkconfig variable set in the idf.py menuconfig, `CONFIG_COMPILER_OPTIMIZATION`. When optimised for size on the ESP32-P4, the udp-basic-consumer-producer loopback test reduces in size to 641KB from an original 723KB.

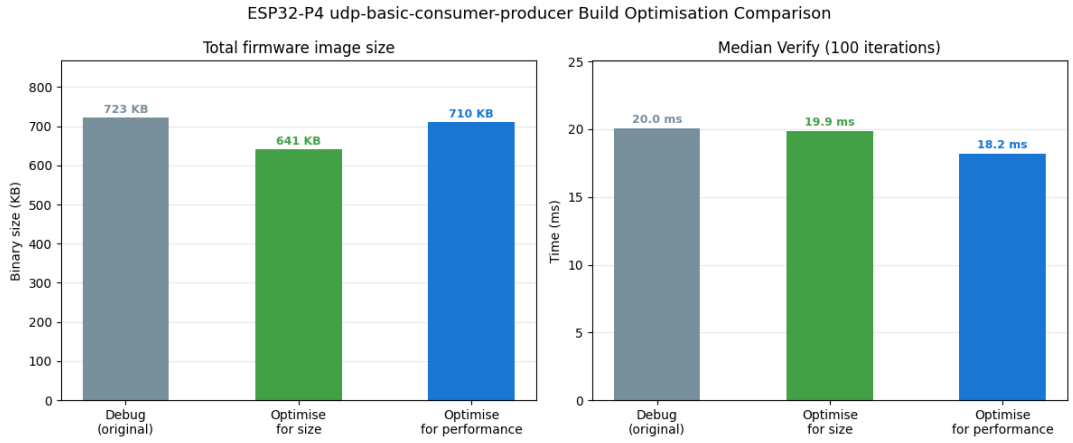


Figure 6.11: Build optimisation comparison on the ESP32-P4 for udp-basic-consumer-producer showing binary image size and median verify latency.

6.5 Power Usage

Embedded applications often need to minimise power consumption to deploy at scale without rapidly exhausting limited energy resources. This is also important for single devices running off battery power. To characterise the basic applications power usage, the processes analysed in Section 6.2 were also examined with current measurements.

6.5.1 ADALM1000

The measurements were taken with an Analog Devices ADALM1000 board (Analog Devices, 2015), which is a USB-powered source measurement unit. This has two analogue channels that can both source or measure signals up to 100kSPS. It contains a 16-bit ADC that maps over a voltage input range of 0-5V and a current range of $\pm 200\,\text{mA}$. Accurate measurements can be obtained using an ESP32C6-DevkitC-1, which provides pins for measuring current. When its J5 header is removed, the current used by the C6 is available from the exposed pins. These pins were routed to a $1\,\Omega$ resistor so that it was in series, voltage could then be measured across this resistor. It was expected that the

C6 would draw more current than the ADALM1000 board's 200 mA limit. Therefore, placing the ADALM1000 board channels in parallel with this resistor to measure voltage, and then manually converting the measured voltage to current, was used to avoid the limit.

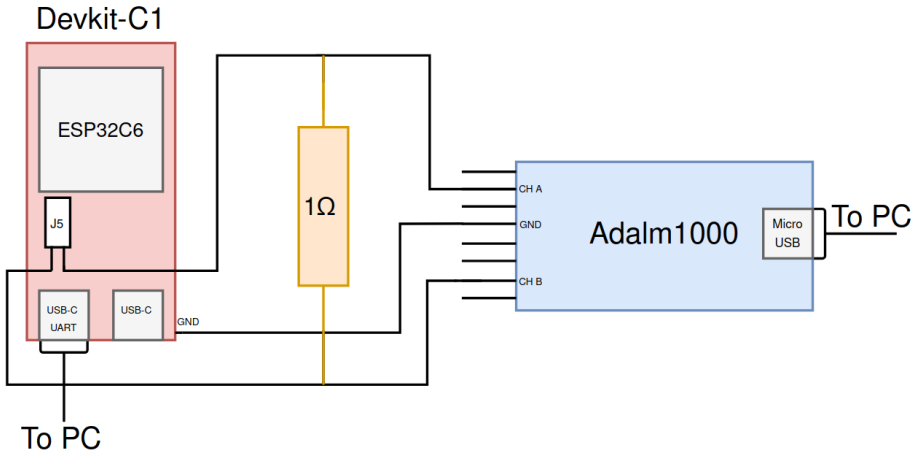


Figure 6.12: Current draw measurement circuit setup using ADALM1000 and ESP32C6-Devkit-C1 with exposed J5 header and 1 Ω resistor

Analog Devices offer their own software for use in conjunction with the ADALM1000, however for more extensive logging and real time visualisation, a custom python script was written with the `pysmu` library (Analog Devices, 2026) they provide. This offers a Python interface for the boards hardware and both channels could be configured to be in high-impedance mode (`HI_Z`) so that the device was acting as a voltmeter. The script acquires sample bursts in varying amounts, depending on whether live plotting was enabled, with the max observed sample rate at approximately 93kSPS. As the shunt resistor is 1 Ω, the current is calculated using Ohm's law, bringing the measurements close to the actual current. The error is due to the resistor's 5% tolerance and to any voltage drop across the resistor in series.

6.5.2 Loopback Power Use

Initially current and power usage was averaged over a 5 second window with approximately 65000 samples. As the device boots, the C6 draws an idle 20mA which is less than what the Espressif C6 datasheet states (Espressif Systems, 2022a) (28mA) this may be due to the removal of the J5 header disabling many peripherals when in this current measuring mode. When the Wi-Fi stack is initialised for the basic applications, the Wi-Fi module may be in TX (transmit) or RX (receive) mode. Wi-Fi TX events begin to occur as soon as the stack enters initialisation, this spikes the current draw to roughly 340mA - 350mA each time a connection attempt is made consistent with the C6's datasheet.

When this connection is successful, the board drops back to its previous idle current draw with intermittent spikes to around 80-100 mA. This is the C6 persisting the Wi-Fi connection in RX mode.

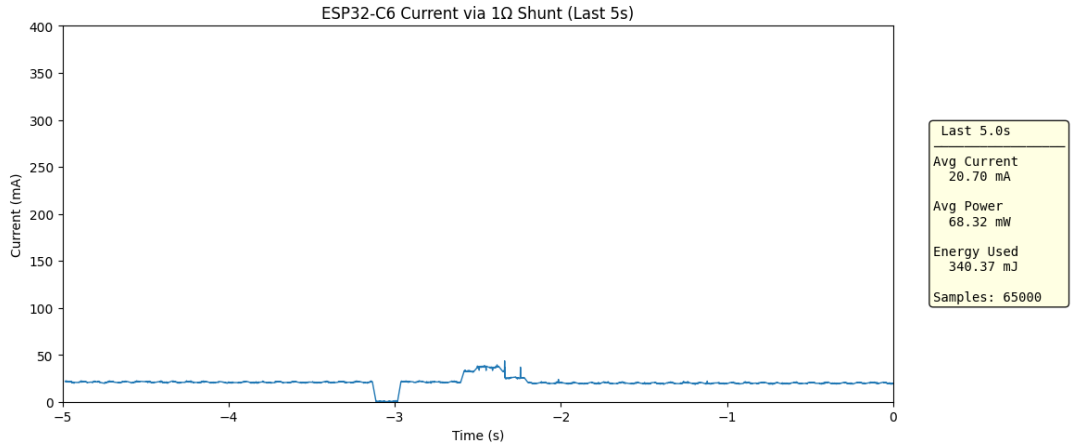


Figure 6.13: Current measurement at device boot, before Wi-Fi initialisation on udp-basic-consumer-producer, the program was paused in between stages in order to better capture current draw at each stage

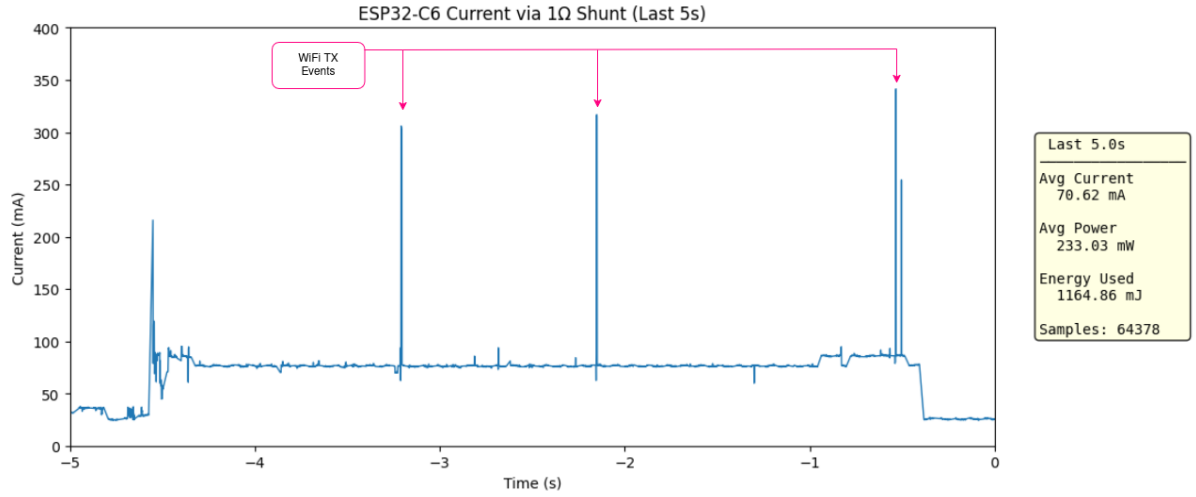


Figure 6.14: Current measurement on udp-basic-consumer-producer during Wi-Fi initialisation

The udp-consumer-producer loopback test shows the change in current usage as it signs, encrypts, decrypts and verifies packets. While the C6 is not using its radio to ensure the Wi-Fi connection is secured, the current jumps from an idle 30mA, to 40mA during the cryptography phases. This, however, is difficult to discern because multiple operations were performed in a short sequence, with Wi-Fi and intermediate events overlapping.

Multiple samples were not being captured during the window, this was due to the sample rate being dropped to as low as 30kSPS at times so that a smooth live plot could be

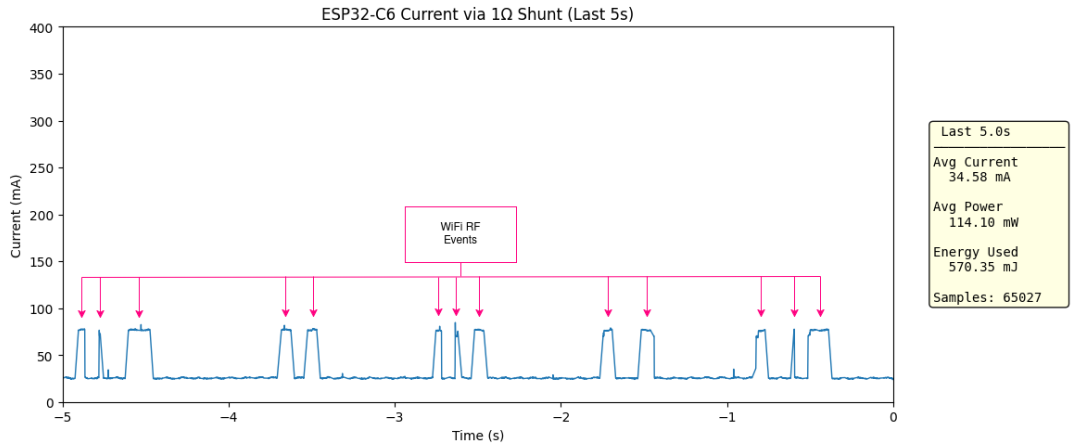


Figure 6.15: Current measurement on udp-basic-consumer-producer post Wi-Fi initialisation before any packet forwarding or receiving has begun, the Wi-Fi module will pull around 80mA during Wi-Fi RF events which is consistent with the ESP32-C6 datasheet

viewed during initial testing, the next tests increased the sample rate to 70kSPS, and captured many more of the quick Wi-Fi TX events, however not all of these were still captured, for more in depth measurements, plots could be created from the CSV logging file created during runtime.

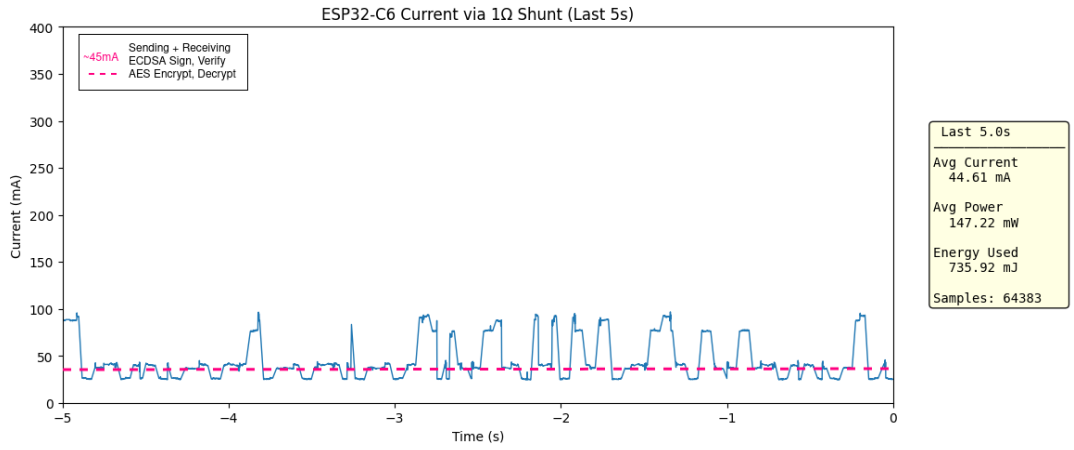


Figure 6.16: Current draw when udp-basic-consumer-producer is running, there are no Wi-Fi TX events due to the device forwarding packets to itself. The average power is 150mW as there is very little delay. Interest and Data are being forwarded constantly. This means that ECDSA signing and verification, as well as AES encryption and decryption, occur back-to-back.

6.5.3 Consumer and Producer Power Use

Consumer and Producer metrics were taken separately to see the individual cost split in a real board-to-board networking scenario. This was done with the CS disabled, and then with it enabled to see any power savings from local caching.

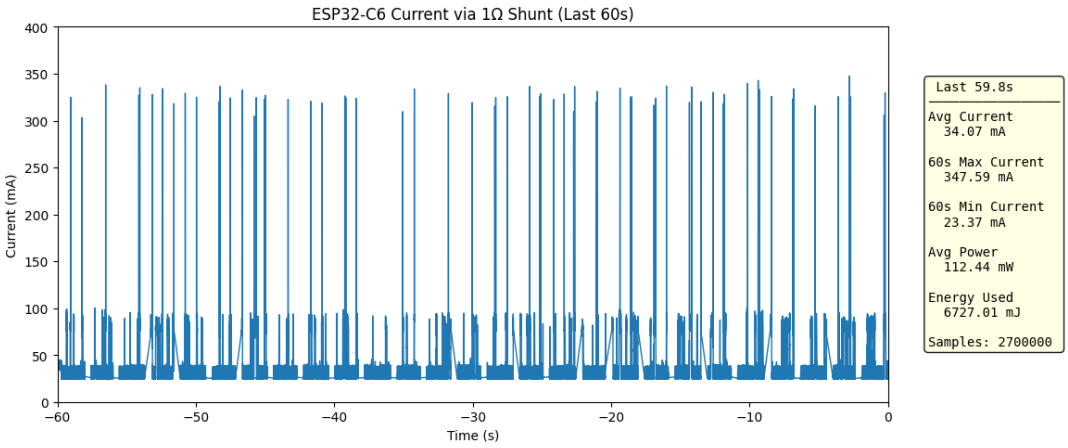


Figure 6.17: Sixty-second window of current measurement on udp-group-consumer with visible Wi-Fi TX events for each Interest forwarded, not every TX event was captured due to the fact that figures were captured live, slightly reducing sampling rate. In total 30 Interest were satisfied

udp-group-consumer ran on the C6 for 60 seconds with a corresponding C6 running udp-group-producer. Delays of 50 ms were introduced into the forwarding process on both sides to enable larger analysis windows. The tests showed 30 Data were returned during the 60-second window, and a total energy consumption calculated to be 6726mJ. Multiple 340mA bursts were recorded again; this is the application activating Wi-Fi TX events to express Interest.

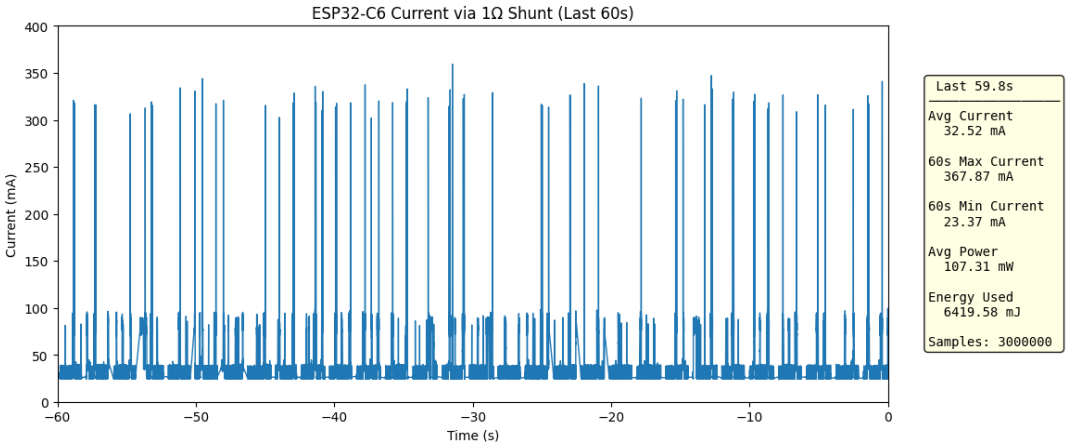


Figure 6.18: Sixty second window of current measurement on udp-group-producer with similarly visible Wi-Fi TX events for each Data returned

The setup was reversed with the ADALM1000 board reading measurements from a C6 running the corresponding producer application. Results show that less energy was used on average at 6343mJ, which may be due to the ECDSA signing requiring less time and resources than verifying which was discovered in the previous latency tests (Figure 6.4).

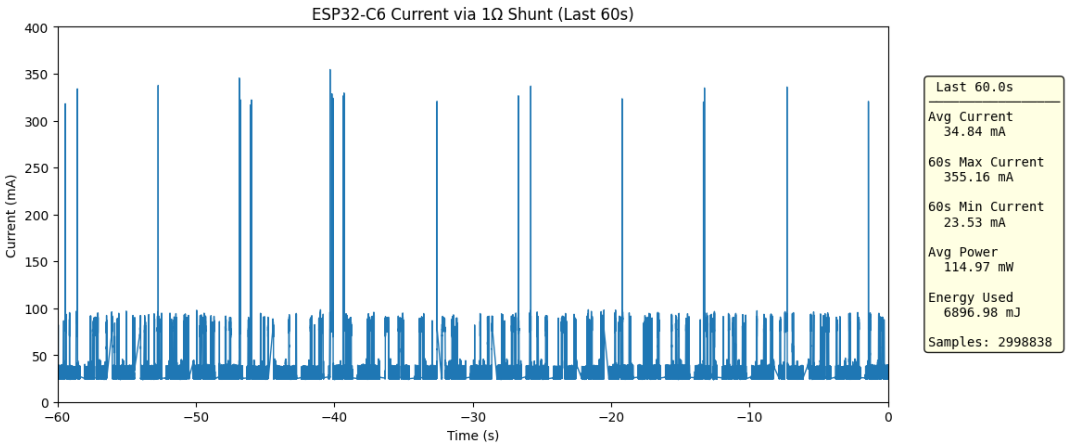


Figure 6.19: Sixty-second window of current measurement on udp-group-consumer when the CS is turned on, and Data FreshnessPeriod is set to 5000ms. This plot shows a reduction in Wi-Fi TX events but a larger amount of ECDSA verification due to Data being available in the CS

With the CS turned on, the same test was performed with measurements taken on the consumer board. The producer had Data FreshnessPeriod set to 5000ms so that the consumer would still need to intermittently send Interest to the multicast face. Figure 6.19 shows that 6896mJ of Energy was used, this is more than when the CS was disabled.

When enabled, the consumer can satisfy more Interests than when waiting for the producer to send Data back, with 70 Interests satisfied in 60 seconds with caching enabled, more than double the 30 satisfied without caching.

Normalising the energy for satisfied requests: $6896/70 \approx 98.5$ mJ per satisfied request with caching versus $6726/30 \approx 224.2$ mJ without caching shows a clear difference. This demonstrates the benefits of in-network caching for embedded applications, with potential speed and power savings. Sensor networks that do not need exact time-sensitive data may rely on such abilities to provide data. However, these tests should be extended to a larger sample to confirm these results.

6.5.4 Cryptography Power Use

To assess the power consumption of cryptographic operations, a new application was developed that performs all cryptographic operations used by both the producer and the consumer without connecting to Wi-Fi or forwarding any packets. Measurements were performed while the board looped through each operation 500 times, as the operations occurred too fast to capture individually. Individual operation usage was taken as the maximum current drawn during a batch of operations.

ECDSA key pair generation draws approximately 35 mA of current, with a maximum

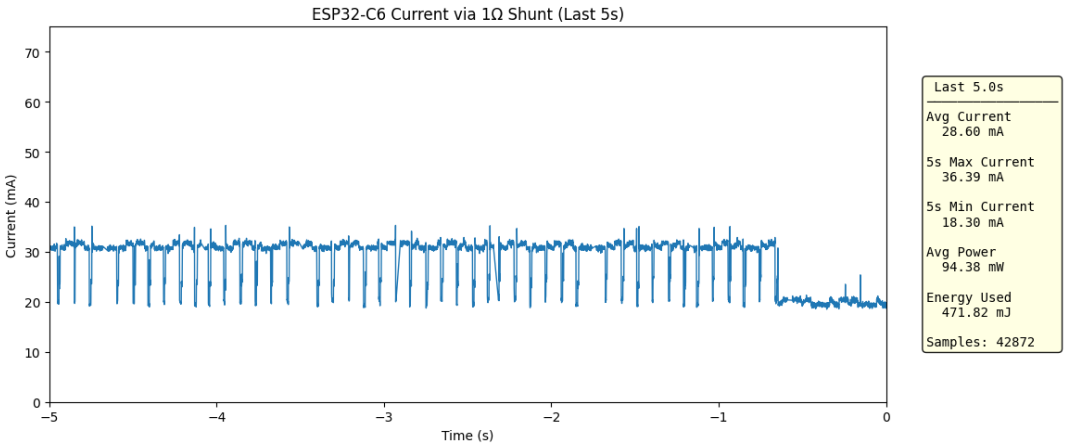


Figure 6.20: ESP32-C6 current draw during 500 iterations of ECDSA key pair generation without Wi-Fi activity

spike of 36 mA. ECDSA operations are the slowest, as shown previously; however, key generation draws less current than verification and signing which both spike current draw to approximately 40mA.

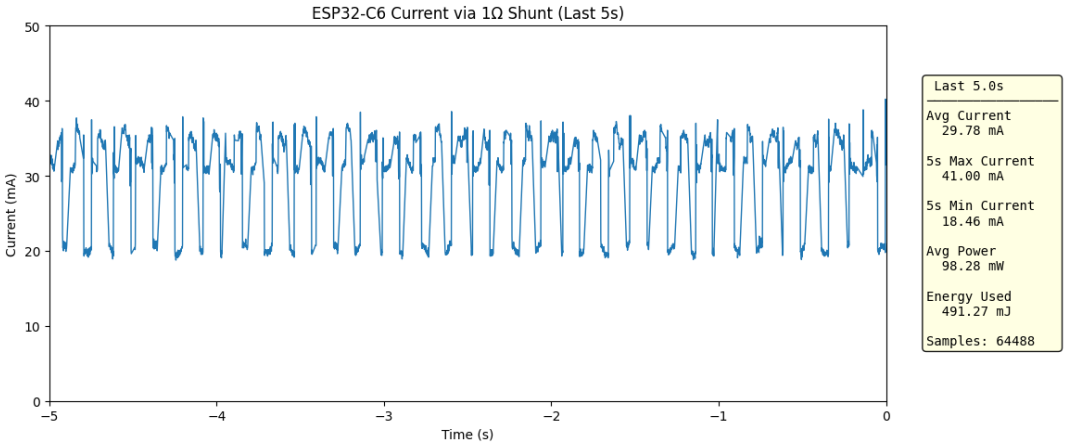


Figure 6.21: ESP32-C6 current draw during 500 iterations of ECDSA signature operations

The AES operations are much faster than ECDSA, as shown in Figure 6.5, for both encryption and decryption, the maximum current spiked to around 35mA showing they are more efficient than the ECDSA operations.

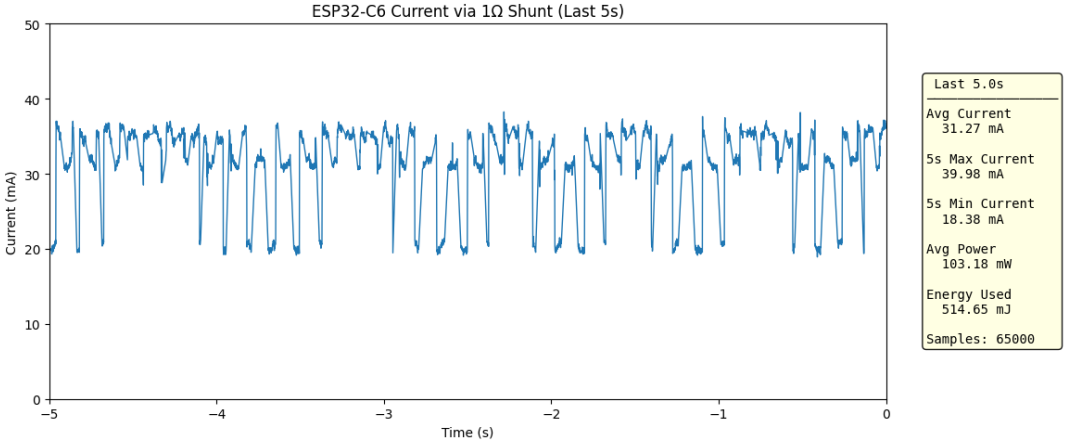


Figure 6.22: ESP32-C6 current draw during 500 iterations of ECDSA verification operations

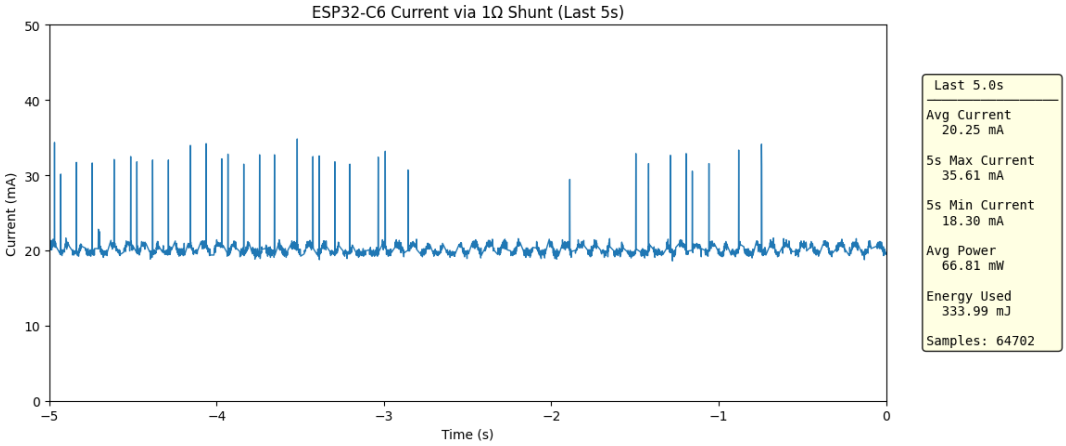


Figure 6.23: ESP32-C6 current draw during 500 iterations of AES-CBC encryption, these operations happen in a matter of microseconds, meaning many of them are missed by the ADALM1000, max values give an idea of the cost of one operation

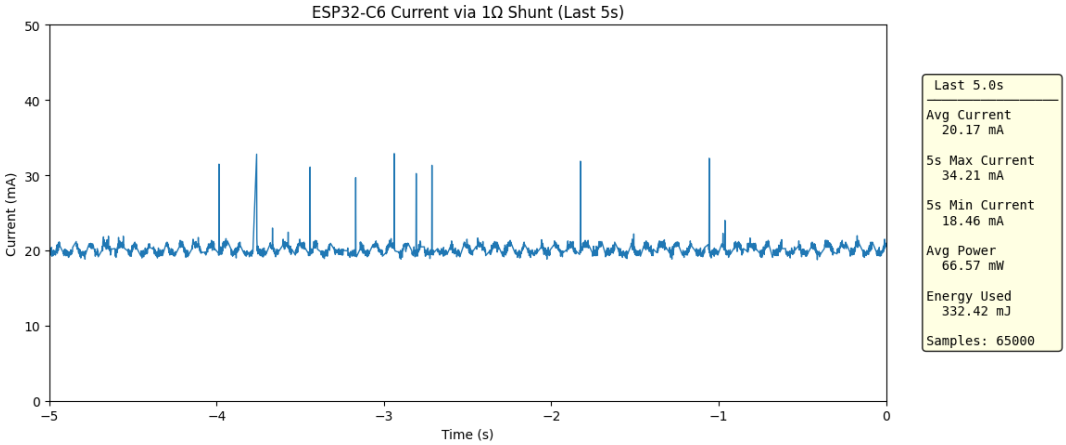


Figure 6.24: ESP32-C6 current draw during 500 iterations of AES-CBC decryption

6.5.5 Battery Life Estimation

The current draw can be used to get an estimate of battery life for deployments without constant power connection. Assuming an 18650 lithium-ion cell with 2500 mAh capacity at 3.7 v with a duty cycle where the device is mostly in the Wi-Fi idle state responding to Interest that's arriving every 50ms (approximately using 35mA), the battery lifetime may last:

$$t = \frac{2500\,\text{mAh}}{35\,\text{mA}} \approx 73.5\,\text{hours} \approx 3\,\text{days and 1 hour}$$

This is an estimate in which the device is constantly responding, and a realistic scenario would allow the device to enter deep sleep, with the idle current draw varying by device. For the ESP32-C6 the datasheet specifies it to be under 10 µA.

A deployment may enter this deep-sleep state intermittently, waking to connect, respond to an Interest, and return to sleep. For this implementation to support NDN retrieval across deep-sleep intervals, additional mechanisms would be required to coordinate Interest sending at the correct times, or to have the device send an "awake" Interest after it switches from deep sleep to active, so that devices can retrieve its content.

During each wake cycle, the device must reconnect to Wi-Fi (approximately 340 mA for 3 seconds), process the NDN Interest–Data exchange (approximately 35 mA for 2 seconds), then return to deep sleep. The average current during a single wake event is:

$$I_{wake} = \frac{(340 \times 3) + (35 \times 2)}{5} = \frac{1088}{5} \approx 217\,\text{mA over 5 seconds}$$

For a sensor network reporting at ten-minute intervals:

$$D = \frac{5}{600} \approx 0.83\%$$

$$I_{avg} = (0.0083 \times 217) + (0.9917 \times 0.01) \approx 1.81\,\text{mA}$$

$$t = \frac{2500\,\text{mAh}}{1.81\,\text{mA}} \approx 1{,}381\,\text{hours} \approx 58\,\text{days}$$

These battery estimates are power-only figures based on a simplified deep sleep schedule. In the current Wi-Fi/NDN implementation, retrieval from a deeply sleeping node was not tested directly and the estimates do not take into account that the node is not continuously reachable while asleep. More accurate estimates would account for a system

that uses synchronised wake intervals to send Interest when the device is active and no longer in this deep-sleep state.

6.5.6 Packet Inspection

To verify that the ESP32 devices produce valid NDN-TLV 0.3 packets that are compatible with NFD, network traffic was captured using Wireshark (Wireshark Foundation, 2026) with the NDN packet dissector plugin (Named Data Networking Project, 2021).

The NDN Wireshark packet dissector allows TLV packets to be decoded based on the NDN TLV v0.3 specification, with differentiation between Interest and Data and all their respective components tabled for examination.

To ignore all other traffic on the network, a filter was placed to only observe the NFD multicast group using `udp.port == 56363`.

After a device boots, advertising Interest and manager response Data packets are first observed as seen in Figure 6.25.

```
. 192.168.0.220    224.0.23.170     UDP (…      290 Data /home/SD/ADV/living-room/esp32c6-4E/params-sha256=
. 192.168.0.140    224.0.23.170     UDP (…      401 Interest /home/SD/ADV/living-room/esp32c6-4E/params-sha
. 192.168.0.220    224.0.23.170     UDP (…      291 Data /home/SD/ADV/living-room/esp32c6-4E/params-sha256=
. 192.168.0.140    224.0.23.170     UDP (…      402 Interest /home/SD/ADV/living-room/esp32c6-4E/params-sha
. 192.168.0.220    224.0.23.170     UDP (…      292 Data /home/SD/ADV/living-room/esp32c6-4E/params-sha256=
```

Figure 6.25: Advertising Interest packets and Data response captured

The transactions show communication between the manager/NFD instance with an IP address of 192.168.0.220 and the ESP32-C6 device IP of 192.168.0.140. The advertisement Interests can be seen to be forwarded over UDP and following this the manager responds with its Data which contains its public key.

Other device packets were confirmed to be forwarded just as the ones from the ESP32-C6 did:

```
192.168.0.97     224.0.23.170     UDP (…      388 Interest /home/SD/ADV/bed-room/esp32-5E/params-sha256
192.168.0.220    224.0.23.170     UDP (…      285 Data /home/SD/ADV/bed-room/esp32-5E/params-sha256=d8d
```

Figure 6.26: Advertising Interest reported from the ESP32-WROOM-32E

These captured packets show a match between the outgoing Interest names and the incoming Data in accordance with NDN standards, and, when a signed Interest is used, a SHA-256 parameter digest is appended to the name as specified by NDN TLV v0.3, as shown in Figure 6.27.

```
Interest /home/SD/ADV/bed-room/esp32-5E/params-sha256=d8d78dba61a22d0d325f489d4a1b467a38fc3df0da213c59356287b40f6ca07a
Data /home/SD/ADV/bed-room/esp32-5E/params-sha256=d8d78dba61a22d0d325f489d4a1b467a38fc3df0da213c59356287b40f6ca07a
```

Figure 6.27: Interest and Data matching in Name, in accordance with NDN TLV v0.3

Interest sent with the general /home/temperature prefix had the CanBePrefix flag set, and devices were able to return Data which had a name consisting of a longer prefix, this was reproduced in Wireshark just as was observed with the dashboard, seen in figure 6.28. Wireshark allows packets to be inspected more thoroughly, and when done so,

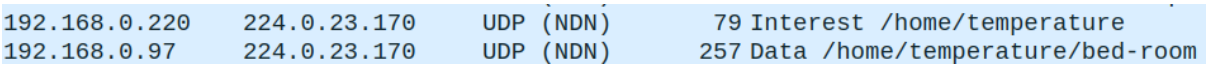


Figure 6.28: Interest with CanBePrefix parameter set and Data returned with the longer child prefix name

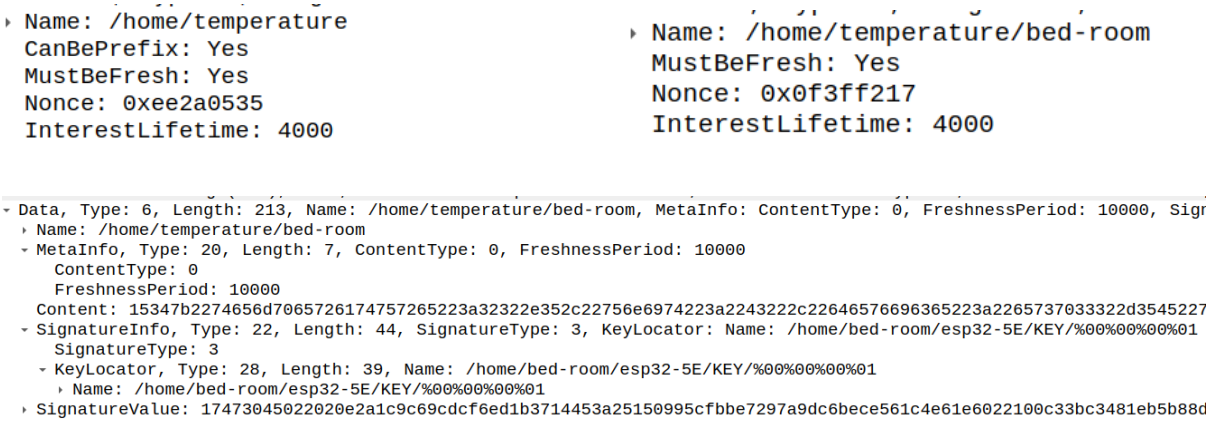


Figure 6.29: Top: Interest breakdown (left) and no-prefix (right). Bottom: Data breakdown. Both Interests return the same Data

the CanBePrefix, MustBeFresh, Nonce, and InterestLifetime were all observed to be set correctly, as seen in Figure 6.29, top left and right images.

Data packets returned for these Interest were also examined; these contained Names, a MetaInfo field containing ContentType and FreshnessPeriod, Content with the JSON sensor payload, a SignatureInfo field with SignatureValue confirming ECDSA signing. They also contained an appended KeyLocator value, as shown in Figure 6.29, which is generated automatically by the NDN-Lite signing function. This followed the specification for the naming format described in Section 2.1.4, however it is not used by the smart home system.

6.6 Summary

This chapter compared flash usage across the different devkits for basic applications, with the C6 having the largest flash due to its extended capabilities and the P4 having the smallest. Devices that ran networking on their own processor also used more RAM; however, this remained below 100 kB. The binary size could be reduced further, and some components that may not be necessary were identified, such as `esp_driver_uart`.

The NDN concepts that NDN-Lite implements were verifiably working across the board, including CS, Interest forwarding, Data retrieval, and cryptographic operations. Metainfo such as FreshnessPeriod and MustBeFresh were shown to work as described.

Although the cryptographic backends could be swapped out, their memory profiles were highly similar across static and dynamic memory. However, Expressif's optimisations to the Mbed TLS library and board-specific changes enabled interfacing with on-board hardware to accelerate cryptographic operations. This was found to be the opposite on boards that lacked this hardware for ECDSA, and, in fact, the TC and MECC backends were more favourable. Espressif's compilation parameters can reduce binary size while maintaining latency or even reducing it.

Power usage was profiled using an ADALM1000 board, and several tests were carried out with the udp-basic-consumer-producer and udp-group-consumer and producer applications. Power usage was low: an ESP32 running the NDN-Lite port's basic application could remain powered for up to 3 days when forwarding continuously or up to 58 days when utilising deep sleep; this claim needs to be confirmed, and deep sleep could substantially extend battery life. ECDSA operations were shown to be the most costly; however, they are vital, as NDN requires that all data be signed. Using the CS could reduce power consumption, as the most expensive operations were Wi-Fi TX bursts for sending Interest and Data to the producer counterpart. Using another face, such as Bluetooth LE, may enable power optimisations for deployments that require long battery life.

Packet transfer was analysed with Wireshark, and the TLV encoding could be examined using the Wireshark NDN TLV dissector, corroborating the packet transfer observed in the dashboard.

7 Conclusion

This project aimed to implement a novel port of NDN-Lite to the ESP32 series using ESP-IDF, enabling NDN communications on some of the most widely deployed Wi-Fi-enabled MCUs in IoT. Pre-existing NDN-Lite adaptations provided no quantitative evaluations of NDN on ESP32 hardware, and no implementations using ESP-IDF existed; both gaps were addressed throughout this project.

The port was verified to deploy across four different ESP32 variants with different CPU architectures (Xtensa and RISC-V) and to be compatible with boards that offload networking to separate processors on development kits. The default cryptographic backend for NDN-Lite relied on TinyCrypt and micro-ecc; these were replaced with Mbed TLS because they had been archived and lacked maintenance, posing a security risk. The advantages of this choice were evaluated, and boards with ECDSA and AES hardware cryptographic acceleration were found to perform cryptographic operations faster than boards without this acceleration. For boards without these hardware accelerators, libraries that optimise software cryptographic implementations are preferable. NDN concepts, including Interest forwarding, Data retrieval, Content Store caching, and FIB and PIT management, were confirmed to be functional and working as intended on the target devices. An adaptation layer was created that mapped NDN-Lite's platform abstractions in an ESP-IDF component, and a compile-time flag was used to alternate between the new Mbed TLS backend and the NDN-Lite-provided TinyCrypt/micro-ecc cryptographic backends.

Memory evaluation showed that binary sizes remained under 1 MB and that runtime RAM was under 100KB across all boards. RTT for loopback Interest-Data exchange was approximately 31ms on the ESP32-P4 and could be reduced to 20ms with Content Store cache hits. This showed the latency benefits of in-network caching even on a single device.

Power measurements were taken using an ADALM1000 with a shunt resistor. This allowed analysis of events such as Wi-Fi TX bursts, which could draw up to 350 mA. In comparison, cryptographic operations contributed around 35 mA and, combined with forwarding, reached up to 40 mA. Average power readings measured between 107mW

and 114mW when forwarding on both the producer and consumer nodes, with estimates based on possible duty cycles indicating the battery's lifetime could reach 58 days with 10-minute reporting intervals.

NFD interoperability was demonstrated using a smart home application that ran on all four target boards simultaneously. ECDSA-signed packets were verified by a Python manager script running alongside NFD on a PC, which confirmed that the port produces NDN-TLV 0.3-compliant packets and is therefore compatible with the broader NDN ecosystem.

The work throughout this project demonstrates that NDN communication on ESP32 hardware is possible and can be achieved with resource-constrained budgets; therefore, NDN-Lite can be successfully adapted to ESP-IDF with performance characteristics typical of IoT deployments.

7.1 Limitations

Although AES was used to encrypt and decrypt data in basic applications, no protocol was implemented to transfer these keys in the smart home system, leaving it vulnerable to security breaches and making packets susceptible to interception during transmission. Real sensor data would be preferable in an application such as this to enable it to serve as an actual smart home network that retrieves information using NDN principles. Signed Interest from the manager was not used in the smart home network, although devices can receive public keys from the manager, they did not use them to verify its Interests. This implementation served as a proof of concept of the ESP32 nodes' capabilities and interoperability with NFD on a PC, and security bootstrapping was not the main goal of this project. This left the smart home network susceptible to impersonation attacks for the manager and devices. Wi-Fi activity far exceeded the other processes in power consumption on the ESP32C6-DevkitC-1; if other faces were implemented, power usage could be reduced, and longer battery life estimates could be made.

7.2 Future Work

Several possible areas of future work were identified upon evaluation that fell outside the scope of this dissertation but represent meaningful directions for further work. These include protocol-level features that could increase throughput or contribute to a more efficient system, such as data chunking and additional face implementations, security improvements and more rigorous benchmarking methodologies. The next subsections address these areas and detail the work necessary to achieve a stronger implementation.

7.2.1 Chunking and Throughput

For more realistic networking scenarios, the adaptation needs to enable the transmission of much larger amounts of data, such as upwards of 1 MB. NDN does this by breaking data into chunks, each of which is loaded into a Data packet. All packets that combine to form the original Data will share a common prefix with an appended segment number. Consumers will retrieve the full object by expressing Interests for each segment until the last segment is retrieved, as indicated by the FinalBlockId field described in Section 2.1.1. The ESP32-S3-EYE's camera could be used as a test for this, with images captured being stored as the Data, and a consumer requesting this content; this consumer could be the dashboard, and the image could be displayed in the data retrieval panel.

This would also provide a more complete picture of NDN performance on ESP devices, with direct comparisons against protocols such as MQTT and CoAP for larger payloads.

7.2.2 New Face Implementations

Implementing BLE or IEEE 802.15.4 could give access to much lower power solutions for battery deployment scenarios. These faces could be created as a standalone face for a device, or alongside another face, such as the UDP faces used in this project. The ESP32-C6 already supports these protocols, and since BLE has a lower MTU, it could benefit from chunking.

7.2.3 SSP

The Secure Sign-On Protocol has been used in ndn-package-over-posix implementations, and much of the code from that repository can be reused for this project. Their HTML dashboard could also be used alongside it to upload the generated key pairs; this was not tested.

If implemented, it would provide a more secure network and be stronger against some of the impersonation vulnerabilities described in Section 4.3.2. The SSP method described for reducing the last exchange of a private key generated by their controller can be used by all the ESP32s selected for this project, as they are capable of generating their own keys.

7.2.4 Real Sensor Data

A sensor such as the BOSCH BME280 (Bosch Sensortec, 2026) could be used to collect three readings, temperature, humidity and barometric pressure and combined with any of the boards that provide GPIO for its SDA and SCL I2C connections. Combined with

the **on_interest** callback, this could then update the dashboard with real readings. This would allow evaluating how the data FreshnessPeriod may interact with readings that constantly change, and variable expiration periods could be applied based on the type of data to be captured.

7.2.5 Topology Benchmarks

In-network caching could be benchmarked by disseminating data across larger networks with 10+ nodes. A PC could run NFD and have a unicast connection to a node; this node could have a multicast connection to 3 more nodes, and each of these nodes could have their own unicast or multicast connections. This would create a semi-realistic networking environment and enable testing of how Data is disseminated across the nodes. Another possibility is to run NFD in the cloud and connect to one of the nodes; this could demonstrate that an NDN IoT network can be reached when away from home.

7.2.6 Stronger base applications and throughput testing

Because the base applications rely on a delay within their main while loop, they incur latency offsets while transmitting packets between each other. If an event-driven architecture were used, there would be less downtime between data reception and forwarder processing. This can be accomplished using FreeRTOS task notifications or semaphores. Such an implementation would enable accurate throughput testing and comparisons between CoAP and MQTT.

Bibliography

Afanasyev, A., Shi, J., Zhang, B., Zhang, L., Moiseenko, I., Yu, Y., Shang, W., Li, Y., Mastorakis, S., Newberry, E., Huang, Y., Abraham, J. P., Liang, T., Schneider, K., DiBenedetto, S., Fan, C., Shannigrahi, S., Papadopoulos, C., Pesavento, D., Grassi, G., Pau, G., Zhang, H., Song, T., Yuan, H., Abraham, H. B., Crowley, P., Amin, S. O., Lehman, V., Chowdhury, M., Gawande, A., Wang, L., and Gordon, N. (2021). NFD developer's guide. Technical Report NDN-0021, Revision 11, Named Data Networking.

Alexander, R., Brandt, A., Vasseur, J. P., Hui, J., Pister, K., Thubert, P., Levis, P., Struik, R., Kelsey, R., and Winter, T. (2012). RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks. Request for Comments RFC 6550, Internet Engineering Task Force.

Amazon Web Services (2026). FreeRTOS. https://freertos.org. Accessed: January 2026.

Analog Devices (2015). ADALM1000 evaluation board. https://www.analog.com/en/resources/evaluation-hardware-and-software/evaluation-boards-kits/adalm1000.html. Accessed: April 2026.

Analog Devices (2026). libsmu: Software abstractions for analog signal exploration tools. https://github.com/analogdevicesinc/libsmu. Accessed: April 2026.

Arduino (2023). UNO R4 WiFi documentation. https://docs.arduino.cc/hardware/uno-r4-wifi/#tech-specs. Accessed: March 2026.

Arduino (2026a). Arduino. https://www.arduino.cc/. Accessed: March 2026.

Arduino (2026b). UNO R3 documentation. https://docs.arduino.cc/hardware/uno-rev3/. Accessed: March 2026.

Baccelli, E., Mehlis, C., Hahm, O., Schmidt, T. C., and Wählisch, M. (2014). Information Centric Networking in the IoT: Experiments with NDN in the Wild. In *Proceedings of the 1st ACM Conference on Information-Centric Networking*, pages 77–86, Paris, France. ACM.

Bormann, C., Ersue, M., and Keränen, A. (2014). Terminology for Constrained-Node Networks. Request for Comments RFC 7228, Internet Engineering Task Force.

Bosch Sensortec (2026). Humidity sensor BME280. https://www.bosch-sensortec.com/en/products/environmental-sensors/humidity-sensors-bme280/. Accessed: April 2026.

Burke, J., Gasti, P., Nathan, N., and Tsudik, G. (2013). Securing instrumented environments over content-centric networking: The case of lighting control and NDN. In *2013 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 394–398, Turin, Italy. IEEE.

Catarinucci, L., de Donno, D., Mainetti, L., Palano, L., Patrono, L., Stefanizzi, M. L., and Tarricone, L. (2015). An IoT-Aware Architecture for Smart Healthcare Systems. *IEEE Internet of Things Journal*, 2(6):515–526.

Cimpanu, C. (2017). 14,766 Let's Encrypt SSL certificates issued to PayPal phishing sites. https://www.bleepingcomputer.com/news/security/14-766-lets-encrypt-ssl-certificates-issued-to-paypal-phishing-sites/. Accessed: March 2026.

Espressif Systems (2014). ESP8266 Wi-Fi SoC. https://www.espressif.com/en/products/socs/esp8266. Accessed: March 2026.

Espressif Systems (2016). ESP32 Wi-Fi and Bluetooth SoC. https://www.espressif.com/en/products/socs/esp32. Accessed: March 2026.

Espressif Systems (2017). ESP32-DevKitC development board. https://www.espressif.com/en/products/devkits/esp32-devkitc. Accessed: April 2026.

Espressif Systems (2022a). ESP32-C6-DevKitC-1 development board. https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c6/esp32-c6-devkitc-1/index.html. Accessed: April 2026.

Espressif Systems (2022b). ESP32-C6 Wi-Fi 6 and BLE 5 and Thread/Zigbee SoC. https://www.espressif.com/en/products/socs/esp32-c6. Accessed: April 2026.

Espressif Systems (2022c). ESP32-S3-EYE development board. https://www.espressif.com/en/taxonomy/term/890. Accessed: April 2026.

Espressif Systems (2022d). ESP32-S3-MINI-1 and ESP32-S3-MINI-1U datasheet. https://documentation.espressif.com/esp32-s3-mini-1_mini-1u_datasheet_en.pdf. Accessed: March 2026.

Espressif Systems (2022e). ESP32-S3 Wi-Fi and BLE 5 SoC. https://www.espressif.com/en/products/socs/esp32-s3. Accessed: March 2026.

Espressif Systems (2023). Espressif leads the IoT chip market with over 1 billion shipments worldwide. https://www.espressif.com/en/news/1_Billion_Chip_Sales. Accessed: March 2026.

Espressif Systems (2024). ESP32-P4 High-performance SoC. https://www.espressif.com/en/products/socs/esp32-p4. Accessed: March 2026.

Espressif Systems (2026a). ESP-IDF getting started. https://idf.espressif.com/. Accessed: March 2026.

Espressif Systems (2026b). ESP-IDF v5.5: Espressif IoT development framework. https://github.com/espressif/esp-idf/tree/release/v5.5. Accessed: April 2026.

Espressif Systems (2026c). ESP-IDF WiFi Station. https://github.com/espressif/esp-idf/blob/master/examples/wifi/getting_started/station/README.md. Accessed: April 2026.

Espressif Systems (2026d). ESP-NETIF - ESP-IDF Programming Guide v6.0. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_netif.html. Accessed: April 2026.

Espressif Systems (2026e). ESP Timer - ESP-IDF Programming Guide v6.0. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/esp_timer.html. Accessed: April 2026.

Espressif Systems (2026f). Espressif centralized documentation platform. https://documentation.espressif.com/. Accessed: April 2026.

Espressif Systems (2026g). Espressif Systems. https://www.espressif.com/. Accessed: January 2026.

Espressif Systems (2026h). Espressif systems GitHub repository. https://github.com/espressif. Accessed: March 2026.

Espressif Systems (2026i). Espressif/Mbed TLS. https://github.com/espressif/mbedtls. Accessed: April 2026.

Espressif Systems (2026j). FreeRTOS – ESP-IDF Programming Guide v4.3. https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32/api-reference/system/freertos.html. Accessed: April 2026.

Espressif Systems (2026k). IDF Size – ESP-IDF Programming Guide v6.0. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/idf-size.html. Accessed: April 2026.

Espressif Systems (2026l). Introduction to Edge AI Solutions - ESP-

Techpedia. https://docs.espressif.com/projects/esp-techpedia/en/latest/esp-friends/solution-introduction/ai/ai-solution.html. Accessed: April 2026.

Espressif Systems (2026m). Mbed TLS - ESP32-P4 – ESP-IDF Programming Guide v6.0. https://docs.espressif.com/projects/esp-idf/en/stable/esp32p4/api-reference/protocols/mbedtls.html. Accessed: April 2026.

Espressif Systems (2026n). Non-Volatile Storage Library - ESP-IDF Programming Guide v6.0. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/storage/nvs_flash.html. Accessed: April 2026.

Espressif Systems (2026o). Random number generation – ESP-IDF Programming Guide v6.0. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/random.html. Accessed: April 2026.

Espressif Systems (2026p). Support for external RAM – ESP-IDF Programming Guide v6.0. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/external-ram.html. Accessed: April 2026.

Espressif Systems (2026q). Universal asynchronous receiver/transmitter (UART) – ESP-IDF Programming Guide v6.0. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/uart.html. Accessed: April 2026.

Espressif Systems (2026r). Wi-Fi – ESP-IDF Programming Guide v6.0. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_wifi.html. Accessed: April 2026.

Fredkin, E. (1960). Trie memory. *Communications of the ACM*, 3(9):490–499.

Goldschmidt, S. (2026). lwIP: A lightweight TCP/IP stack. https://www.nongnu.org/lwip/2_1_x/index.html. Accessed: March 2026.

Guan, Z., Li, J., Wu, L., Zhang, Y., Wu, J., and Du, X. (2017). Achieving Efficient and Secure Data Acquisition for Cloud-Supported Internet of Things in Smart Grid. *IEEE Internet of Things Journal*, 4(6):1934–1944.

Gündoğan, C., Amsüss, C., Schmidt, T. C., and Wählisch, M. (2021). Reliable Firmware Updates for the Information-Centric Internet of Things. In *Proceedings of the 8th ACM Conference on Information-Centric Networking*, pages 59–70, Paris, France. ACM.

Gündoğan, C., Kietzmann, P., Lenders, M., Petersen, H., Schmidt, T. C., and Wählisch, M. (2018). NDN, CoAP, and MQTT: A comparative measurement study in the IoT. In *Proceedings of the 5th ACM Conference on Information-Centric Networking*, pages 159–171, New York, USA. ACM.

Homaei, M., Mogollón-Gutiérrez, Ó., López-Trigo, A., Carlos Sancho Núñez, J., and

Ávila, M. (2025). Load Balancing in IoT: Leveraging Learning Automata for RPL Optimization. *IEEE Access*, 13:180091–180110.

Hoque, A. K. M. M., Amin, S. O., Alyyan, A., Zhang, B., Zhang, L., and Wang, L. (2013). NLSR: Named-data link state routing protocol. In *Proceedings of the 3rd ACM SIGCOMM Workshop on Information-centric Networking*, pages 15–20, Hong Kong, China. ACM.

Infineon Technologies (2020). CYW43439 – Wi-Fi 4 (802.11n). https://www.infineon.com/part/CYW43439. Accessed: March 2026.

Intel Corporation (2019). Tinycrypt: Cryptographic library for constrained devices. https://github.com/intel/tinycrypt. Accessed: April 2026.

IoT Analytics (2025). Number of connected IoT devices growing 14% to 21.1 billion. https://iot-analytics.com/number-connected-iot-devices/. Accessed: March 2026.

Jacob, B., Wang, D., and Ng, S. (2010). *Memory Systems: Cache, DRAM, Disk*. Morgan Kaufmann.

Jacobson, V., Smetters, D. K., Thornton, J. D., Plass, M. F., Briggs, N. H., and Braynard, R. L. (2009). Networking named content. In *Proceedings of the 5th International Conference on Emerging Networking Experiments and Technologies*, CoNEXT '09, pages 1–12, New York, USA. ACM.

Li, X., Lu, R., Liang, X., Shen, X., Chen, J., and Lin, X. (2011). Smart community: An internet of things application. *IEEE Communications Magazine*, 49(11):68–75.

Li, Y., Zhang, Z., Wang, X., Lu, E., Zhang, D., and Zhang, L. (2019). A Secure Sign-On Protocol for Smart Homes over Named Data Networking. *IEEE Communications Magazine*, 57(7):62–68.

Linux Foundation (2026). Zephyr project. https://www.zephyrproject.org/. Accessed: March 2026.

MacKay, K. (2024). micro-ecc: ECDH and ECDSA for 8-bit, 32-bit, and 64-bit processors. https://github.com/kmackay/micro-ecc. Accessed: April 2026.

Mbed TLS (2026). Mbed TLS. https://github.com/Mbed-TLS/mbedtls. Accessed: April 2026.

Moin, A., Challenger, M., Badii, A., and Günnemann, S. (2022). Supporting AI Engineering on the IoT Edge through Model-Driven TinyML. In *2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 884–893, Los Alamitos, USA. IEEE.

Moy, J. (1997). OSPF Version 2. Request for Comments RFC 2178, Internet Engineering Task Force.

Named Data Networking (2019). NDN technical memo: Naming conventions. https://named-data.net/publications/techreports/ndn-tr-22-2-ndn-memo-naming-conventions/. Accessed: April 2026.

Named Data Networking (2024a). nfdc - NFD Management Tool. https://docs.named-data.net/NFD/current/manpages/nfdc.html. Accessed: April 2026.

Named Data Networking (2024b). Signature — NDN packet format specification v0.3. https://docs.named-data.net/NDN-packet-spec/current/signature.html. Accessed: April 2026.

Named Data Networking (2025). Type-length-value (TLV) encoding – NDN packet format specification v0.3. https://docs.named-data.net/NDN-packet-spec/0.3/tlv.html. Accessed: March 2026.

Named Data Networking (2026a). ndn-cxx: NDN C++ library. https://github.com/named-data/ndn-cxx. Accessed: March 2026.

Named Data Networking (2026b). python-ndn: An NDN client library with AsyncIO support. https://github.com/named-data/python-ndn. Accessed: April 2026.

Named Data Networking for IoT (2018). ndn-lite-over-riot: RIOT adaptation of NDN-Lite. https://github.com/named-data-iot/ndn-lite-over-riot. Accessed: March 2026.

Named Data Networking for IoT (2020). ndn-iot-package-over-posix documentation. https://ndn-iot-package-over-posix.readthedocs.io/en/latest/. Accessed: March 2026.

Named Data Networking for IoT (2021a). ndn-lite: A lightweight NDN protocol stack for IoT. https://github.com/named-data-iot/ndn-lite. Accessed: January 2026.

Named Data Networking for IoT (2021b). Quickstart examples – ndn-iot-package-over-posix documentation. https://ndn-iot-package-over-posix.readthedocs.io/en/latest/src/examples.html. Accessed: April 2026.

Named Data Networking for IoT (2024). ndn-iot-package-over-posix. https://github.com/named-data-iot/ndn-iot-package-over-posix. Accessed: March 2026.

Named Data Networking for IoT (2026a). Named data networking for IoT – GitHub organisation. https://github.com/named-data-iot. Accessed: March 2026.

Named Data Networking for IoT (2026b). NDN-Lite Wiki: Overview. https://github.com/named-data-iot/ndn-lite/wiki/Overview. Accessed: March 2026.

Named Data Networking Project (2021). NDN Wireshark Dissector. https://github.com/

named-data/ndn-tools/blob/master/tests/dissect-wireshark/README.md. Accessed: April 2026.

National Institute of Standards and Technology (US) (2023). Digital Signature Standard (DSS). Technical Report NIST FIPS 186-5, National Institute of Standards and Technology (U.S.), Washington, D.C.

Newberry, E., Ma, X., and Zhang, L. (2021). YaNFD: Yet another named data networking forwarding daemon. In *Proceedings of the 8th ACM Conference on Information-Centric Networking*, pages 30–41, Paris, France. ACM.

Niwa, T. (2022). 2022 wireless connectivity market analysis report summary. https://iotbusinessnews.com/download/2022-TSR-Wireless-Connectivity-Market-Report-Summary.pdf. Accessed: April 2026.

Nordic Semiconductor (2018). nRF52840 Multiprotocol Bluetooth SoC. https://www.nordicsemi.com/Products/nRF52840. Accessed: January 2026.

OASIS (2014). MQTT version 3.1.1. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html. Accessed: January 2026.

Popli, S., Jha, R. K., and Jain, S. (2019). A Survey on Energy Efficient Narrowband Internet of Things (NBIoT). *IEEE Access*, 7:16739–16776.

Python Software Foundation (2026). asyncio - Asynchronous I/O. https://docs.python.org/3/library/asyncio.html. Accessed: April 2026.

Raspberry Pi Ltd (2019). Raspberry Pi 4 Model B. https://www.raspberrypi.com/products/raspberry-pi-4-model-b/. Accessed: March 2026.

Raspberry Pi Ltd (2021). Raspberry Pi Pico. https://www.raspberrypi.com/products/raspberry-pi-pico/. Accessed: March 2026.

Raspberry Pi Ltd (2023). Raspberry Pi 5. https://www.raspberrypi.com/products/raspberry-pi-5/. Accessed: March 2026.

Raspberry Pi Ltd (2024). Raspberry Pi Pico 2. https://www.raspberrypi.com/products/raspberry-pi-pico-2/. Accessed: April 2026.

Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. Request for Comments RFC 8446, Internet Engineering Task Force.

Rivest, R. and Lampson, B. (1996). SDSI – a simple distributed security infrastructure. Technical report, MIT.

Sarker, S., Islam, M. E., Saurav, J. R., and Nahid, M. M. H. (2020). Word completion and

sequence prediction in Bangla language using trie and a hybrid approach of sequential LSTM and N-gram. In *2020 2nd International Conference on Advanced Information and Communication Technology (ICAICT)*, pages 162–167, Dhaka, Bangladesh. IEEE.

Shang, W., Bannis, A., Liang, T., Wang, Z., Yu, Y., Afanasyev, A., Thompson, J., Burke, J., Zhang, B., and Zhang, L. (2016). Named Data Networking of Things (Invited Paper). In *2016 IEEE First International Conference on Internet-of-Things Design and Implementation (IoTDI)*, pages 117–128, Berlin, Germany. IEEE.

Shelby, Z., Hartke, K., and Bormann, C. (2014). The Constrained Application Protocol (CoAP). Technical Report RFC7252, RFC Editor.

Shi, J. (2025). esp8266ndn: NDN library for ESP8266 and ESP32. https://github.com/yoursunny/esp8266ndn. Accessed: March 2026.

Svetlov, A. (2026). aiohttp: Asynchronous HTTP client/server framework. https://docs.aiohttp.org/en/stable/. Accessed: April 2026.

Waveshare (2026). ESP32-P4-Module-DEV-KIT. https://www.waveshare.com/wiki/ESP32-P4-Module-DEV-KIT-StartPage. Accessed: April 2026.

Wireshark Foundation (2026). Wireshark. https://www.wireshark.org/. Accessed: April 2026.

wolfSSL (2024). Espressif RISC-V hardware accelerated cryptographic functions up to 1000% faster than software. https://www.wolfssl.com/espressif-risc-v-hardware-accelerated-cryptographic-functions-up-to-1000-faster-than-software/. Accessed: April 2026.

Yu, Y., Afanasyev, A., Clark, D., Claffy, K., Jacobson, V., and Zhang, L. (2015). Schematizing Trust in Named Data Networking. In *Proceedings of the 2nd ACM Conference on Information-Centric Networking*, pages 177–186, San Francisco, USA. ACM.

Zhang, J. and Tao, D. (2021). Empowering Things with Intelligence: A Survey of the Progress, Challenges, and Opportunities in Artificial Intelligence of Things. *IEEE Internet of Things Journal*, 8(10):7789–7817.

Zhang, L., Afanasyev, A., Burke, J., Jacobson, V., claffy, k., Crowley, P., Papadopoulos, C., Wang, L., and Zhang, B. (2014). Named data networking. *SIGCOMM Comput. Commun. Rev.*, 44(3):66–73.

Zhang, L., Estrin, D., Burke, J., Jacobson, V., Thornton, J. D., Smetters, D. K., and Zhang, B. (2010). Named Data Networking (NDN) Project. Technical report, Xerox PARC.

Zhang, M., Lehman, V., and Wang, L. (2017). Scalable name-based data synchronization

for named data networking. In *IEEE INFOCOM 2017 - IEEE Conference on Computer Communications*, pages 1–9.

Zhang, Z., Lu, E., Li, Y., Zhang, L., Yu, T., Pesavento, D., Shi, J., and Benmohamed, L. (2018a). NDNoT: A framework for named data network of things. In *Proceedings of the 5th ACM Conference on Information-Centric Networking*, pages 200–201, New York, USA. ACM.

Zhang, Z., Yu, T., Ma, X., Guan, Y., Moll, P., and Zhang, L. (2022). Sovereign: Self-Contained Smart Home With Data-Centric Network and Security. *IEEE Internet of Things Journal*, 9(15):13808–13822.

Zhang, Z., Yu, Y., Zhang, H., Newberry, E., Mastorakis, S., Li, Y., Afanasyev, A., and Zhang, L. (2018b). An Overview of Security Support in Named Data Networking. *IEEE Communications Magazine*, 56(11):62–68.

Zhu, Z. and Afanasyev, A. (2013). Let's ChronoSync: Decentralized dataset state synchronization in Named Data Networking. In *2013 21st IEEE International Conference on Network Protocols (ICNP)*, pages 1–10, Goettingen, Germany. IEEE.

A1 Appendix

A1.1 idf.py menuconfig

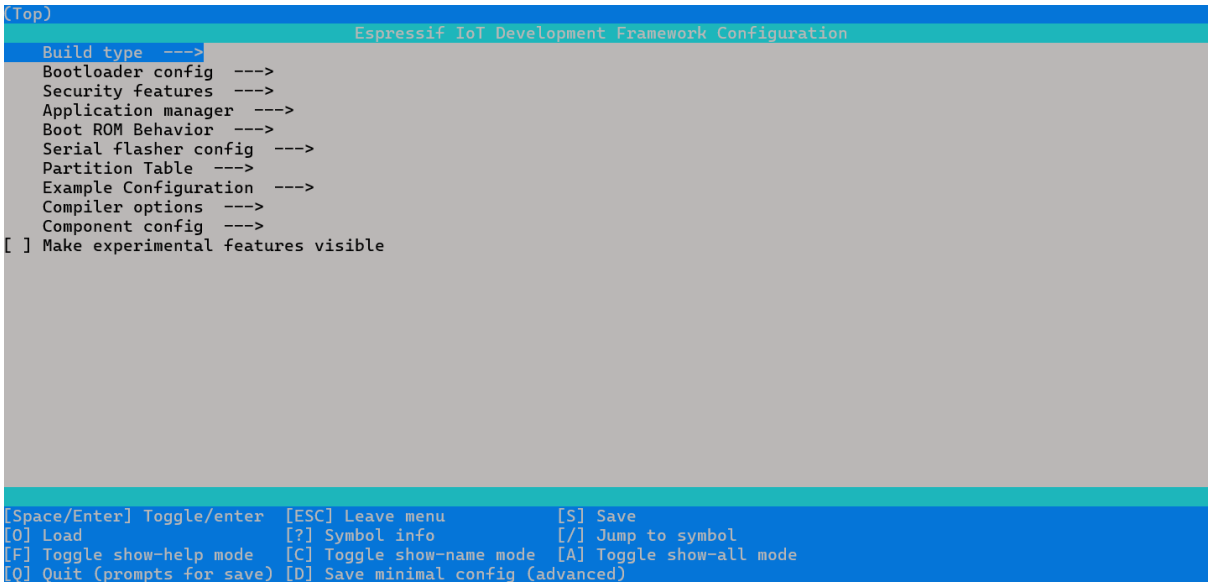


Figure A1.1: ESP-IDF menuconfig example, parameters could be set in here manually and saved to an sdkconfig file. This file could also be manually created and edited without entering this menu

A1.2 idf.py menuconfig Mbed TLS options

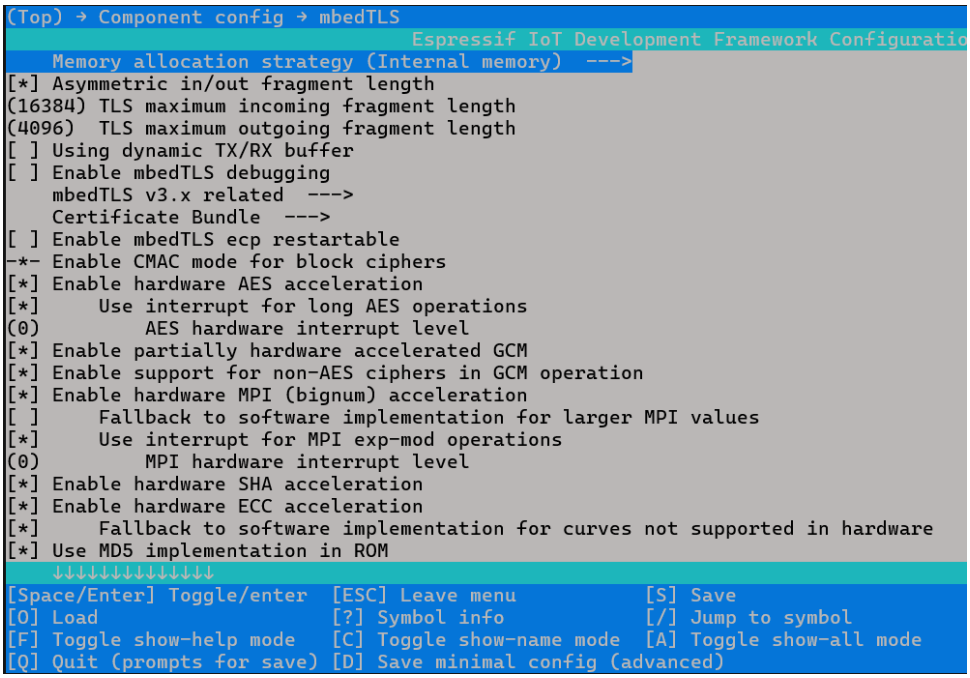


Figure A1.2: Mbed TLS hardware acceleration parameters can be altered in the menuconfig, however when using compatible devices these hardware acceleration parameters will be activated by default when using Mbed TLS

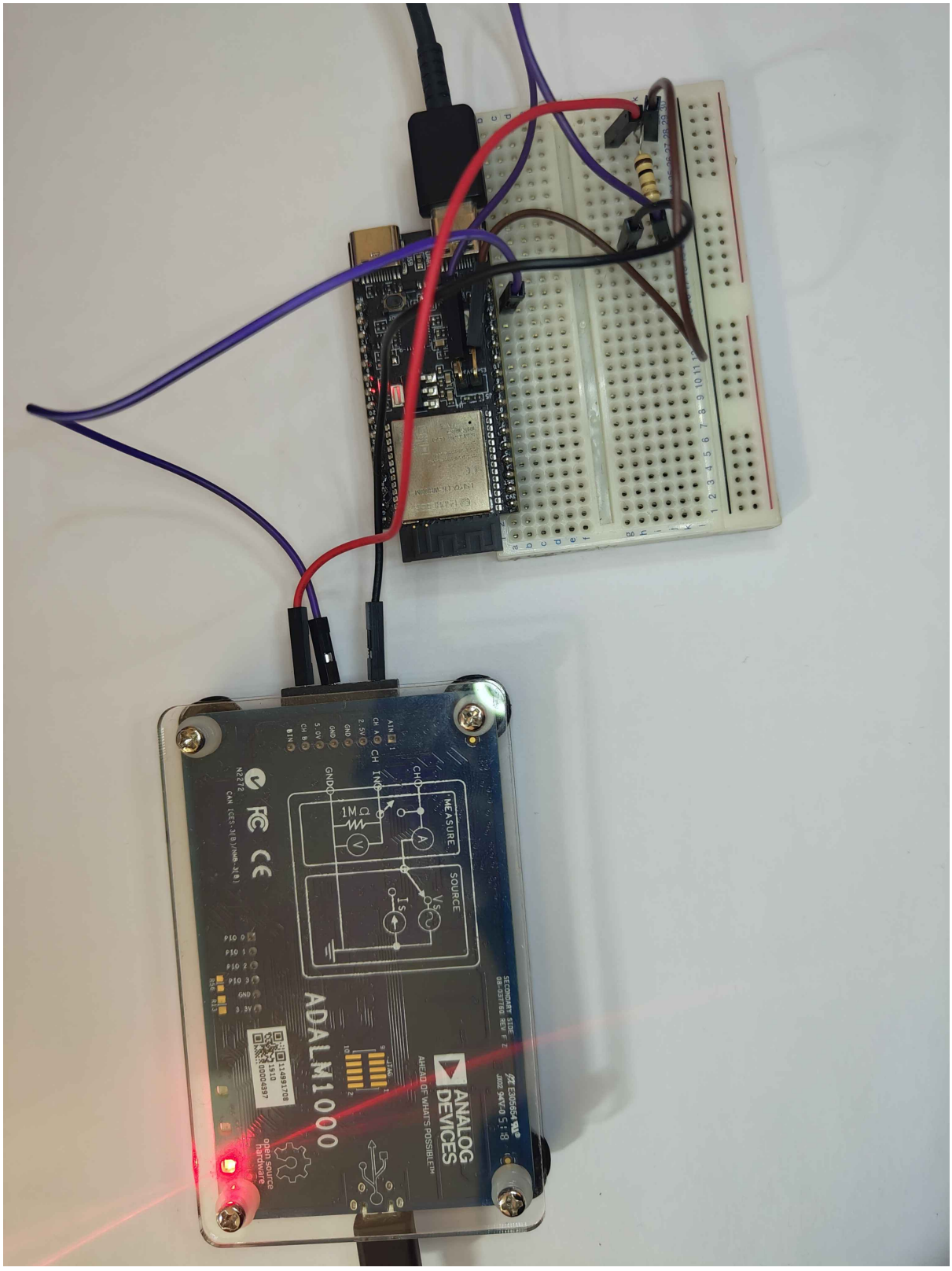


Figure A1.3: The ADALM1000 board setup with a 1 Ω shunt resistor and an ESP32-C6-DevkitC-1 with its J5 header removed to access the current measurement pins

A1.3 ADALM1000 Power measurement script

Plot Initialisation

```
import matplotlib
matplotlib.use("TkAgg")
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import csv
import time
from collections import deque
from pysmu import Session, Mode

SHUNT_R = 1.0
SUPPLY_V = 3.3
SAMPLES_PER_READ = 100000 # can change these based on performance
WINDOW_SECONDS = 5
MAX_PLOT_POINTS = 5000
CSV_FILE = "current_log.csv" # can make plots from this later if live plotting is too
    slow

session = Session()
if not session.devices:
    print("No␣ADALM1000␣found")
    exit(1)

dev = session.devices[0]
dev.channels['A'].mode = Mode.HI_Z
dev.channels['B'].mode = Mode.HI_Z

plt.ion()
fig = plt.figure(figsize=(13, 5))
gs = gridspec.GridSpec(1, 2, width_ratios=[3, 1], figure=fig)
ax = fig.add_subplot(gs[0])
ax_stats = fig.add_subplot(gs[1])
line, = ax.plot([], [], lw=1)
ax.set_xlabel("Time␣(s)")
ax.set_ylabel("Current␣(mA)")
ax.set_title(f"ESP32-C6␣Current␣via␣1␣Ohm␣Shunt␣(Last␣{WINDOW_SECONDS}s)")
ax.set_xlim(-WINDOW_SECONDS, 0)
ax.set_ylim(0, 75)
ax_stats.axis('off')
stats_text = ax_stats.text(
    0.08, 0.5, "Acquiring...",
    transform=ax_stats.transAxes,
    verticalalignment='center',
```

```
    fontsize=10,
    family='monospace',
    bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.9)
)
fig.tight_layout()
PLOT_EVERY_N = 10
timestamps = deque()
currents = deque()
```

Acquisition Loop

```
with open(CSV_FILE, "w", newline="") as csv_file:
    writer = csv.writer(csv_file)
    writer.writerow(["Timestamp", "Current_mA"])
    batch_count = 0
    try:
        while True:
            t0 = time.time()
            data = dev.get_samples(SAMPLES_PER_READ)
            t1 = time.time()
            batch_dt = t1 - t0
            print(f"Batch␣{batch_count}:␣Acquired␣{len(data)}␣samples␣"
                  f"in␣{batch_dt:.3f}s␣({len(data)/batch_dt:.1f}␣S/s)")
            n_batch = len(data)
            for i, sample in enumerate(data):
                current_mA = (sample[0][0] - sample[1][0]) / SHUNT_R * 1000
                t = t0 + (i + 1) / n_batch * batch_dt
                timestamps.append(t)
                currents.append(current_mA)
                writer.writerow([t, current_mA])
            batch_count += 1
            if batch_count % PLOT_EVERY_N != 0:
                continue
            now = time.time()
            cutoff = now - WINDOW_SECONDS
            while timestamps and timestamps[0] < cutoff:
                timestamps.popleft()
                currents.popleft()
            if len(timestamps) >= 2:
                ts_list = list(timestamps)
                cur_list = list(currents)
                n = len(ts_list)
                if n > MAX_PLOT_POINTS:
                    step = n // MAX_PLOT_POINTS
                    ts_plot = ts_list[::step]
                    cur_plot = cur_list[::step]
                else:
```

```
                ts_plot = ts_list
                cur_plot = cur_list
            rel_times = [t - now for t in ts_plot]
            line.set_data(rel_times, cur_plot)
            window_dur = ts_list[-1] - ts_list[0] if n > 1 else 0
            avg_current = sum(cur_list) / n
            power_mW = SUPPLY_V * avg_current
            energy_mJ = power_mW * window_dur
            stats_text.set_text(
                f"␣Last␣{window_dur:.1f}s␣\n"
                f"Avg␣Current\n"
                f"␣␣{avg_current:.2f}␣mA\n\n"
                f"{WINDOW_SECONDS}s␣Max␣Current\n"
                f"␣␣{max(cur_list):.2f}␣mA\n\n"
                f"{WINDOW_SECONDS}s␣Min␣Current\n"
                f"␣␣{min(cur_list):.2f}␣mA\n\n"
                f"Avg␣Power\n"
                f"␣␣{power_mW:.2f}␣mW\n\n"
                f"Energy␣Used\n"
                f"␣␣{energy_mJ:.2f}␣mJ\n\n"
                f"Samples:␣{n}"
            )
        fig.canvas.draw_idle()
        plt.pause(0.01)
except KeyboardInterrupt:
    session._close()
```

A1.4 NDN-Lite manager

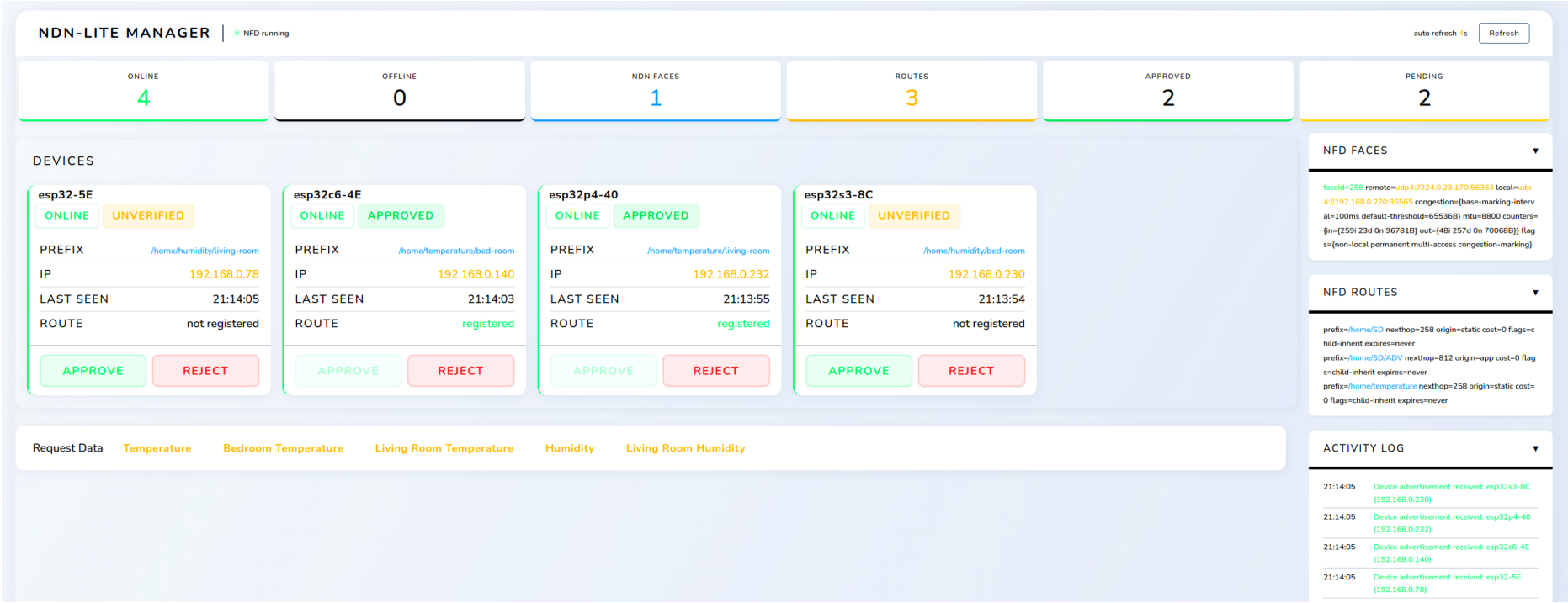


Figure A1.4: Full NDN-Lite Manager dashboard showing data retrieval panel, NFD faces, routes, and activity log.